

**A Design Methodology for Producing Highly-Adaptable and
High-Performance Simulation Frameworks**

A Dissertation Presented

by

Yuhui Bao

to

The Department of Electrical and Computer Engineering

in partial fulfillment of the requirements
for the degree of

Doctor of Philosophy

in

Electrical and Computer Engineering

**Northeastern University
Boston, Massachusetts**

December 2024

Contents

List of Figures	iv
List of Tables	vi
Abstract of the Dissertation	vii
1 Introduction	1
1.1 Why Does the Community Continually Need New and Adaptable Simulators . . .	2
1.1.1 Dynamics in the GPU Market	2
1.1.2 Accelerating Challenging Workloads	3
1.1.3 Common Issues with Simulators	5
1.2 Challenges in Designing a GPU Simulator: Implications of Previous Considerations	6
1.3 The Speed of Simulation	7
1.4 Contributions of This Thesis	9
1.5 The Organization of this Proposal	10
2 Background	11
2.1 GPU Programming Frameworks	11
2.2 Instruction Set Architecture versus Architecture versus Microarchitecture	12
2.3 GPU Microarchitecture	13
2.3.1 Kernel Dispatching	13
2.3.2 Compute Unit	14
2.3.3 The GPU Memory Hierarchy	15
2.4 Architectural Simulators	16
2.4.1 Microarchitecture Simulation	16
2.4.2 Classification of Simulators	17
2.5 Performance Evaluation Tools	18
2.5.1 perf	19
2.5.2 PAPI	19
2.5.3 pprof	19
2.6 Fully Homomorphic Encryption	19
2.6.1 The CKKS FHE Scheme	20
2.6.2 Building Blocks of the CKKS FHE Scheme	22

3	Related Work	27
3.1	GPU Simulators	27
3.1.1	Commercially-develop Simulators	27
3.1.2	Academic Research Simulators	28
3.2	Extending GPU Architectures to Accelerate FHE	29
3.2.1	Extending/Modifying Current GPU Architectures	29
3.2.2	Accelerating FHE	30
4	NaviSim	32
4.1	Design Goals	33
4.2	Simulator Development Methodology	34
4.3	RDNA Architecture	36
4.4	Implementation of NaviSim	38
4.4.1	ISA Emulation	38
4.4.2	Wavefront Dispatching	38
4.4.3	DCU modeling	39
4.4.4	Memory Hierarchy	40
4.4.5	User Interface	40
4.5	Evaluation	41
4.5.1	Simulator Validation	41
4.6	Case Studies: Understanding the RDNA Architecture Features	46
4.6.1	Impact of changes in the ISA and the introduction of the DCU	47
4.6.2	Impact of the L1 cache	49
4.6.3	Impact of increased frequency	50
5	Design Space Exploration: Accelerating FHE with Microarchitectural Extensions (GME)	52
5.1	Challenges of Accelerating CKKS FHE	52
5.2	Exploring Microarchitectural Extensions with NaviSim	54
5.2.1	cNoC: CU-side interconnect	55
5.2.2	Enhancing the Vector ALU	57
5.2.3	LABS: Locality-Aware Block Scheduler	59
5.3	Evaluation	61
5.3.1	Methodology	61
5.3.2	Results	63
6	Simulation Performance Evaluation and Optimization	68
6.1	Methodology	69
6.1.1	General simulator design	69
6.1.2	Parallel simulation scalability	71
6.1.3	Performance Metrics	72
6.2	Evaluation	72
6.2.1	Experimental Setup and Profiling Methods	72
6.2.2	Performance Analysis	75
6.3	Performance Improvement Results	83

6.4	Summary	90
7	Conclusion and Future Work	93
7.1	Conclusion	93
7.2	Future Work	94
	Bibliography	96
A	Full Synchronization Blocking Profiles	107

List of Figures

1.1	The roofline model for the FHE operation with size of the ciphertext $N = 2^{16}$ on AMD MI100 GPU	4
1.2	A comparison of a simulator's reported kernel execution time and the native RX 5500 XT GPU execution time when running the BICG benchmark from PolyBench [1].	5
1.3	Simulation slowdown of benchmarks running on NaviSim under emulation mode and timing simulation mode	8
2.1	The Microrchitecture of CDNA GPUs	14
2.2	The Microrchitecture of CDNA Compute Unit	15
2.3	FHE offers a safeguard against online eavesdroppers as well as untrusted cloud services by allowing direct computation on encrypted data.	20
2.4	The operation flow of the CKKS scheme.	21
2.5	Polynomial Multiplication.	23
4.1	A Comparison between the GCN Architecture [2] and the RDNA Architecture [3].	37
4.2	The instruction pipeline model in a NaviSim DCU. Each DCU has four copies of the structure shown.	39
4.3	Comparison of the Pointer Chasing microbenchmark between NaviSim and the RX 5500 XT hardware.	45
4.4	Simulator validation against the AMD RX 5500 XT GPU. The x-axis plots the problem size and the two y-axes plot the kernel execution time and relative error. The numbers in the parentheses represent the average difference between NaviSim simulation and the hardware execution for each application.	46
4.5	Simulator validation against the AMD Radeon Pro W6800 GPU. The x-axis plots the problem size and the two y-axes plot the kernel execution time and relative error. The numbers in the parentheses represent the average difference between NaviSim simulation and the hardware execution for each application.	47
4.6	The effect of changing from CUs to DCUs and the ISA from GFX803 (GCN3) to GFX1010 (RDNA) on execution time.	48
4.7	The effect of adding the new write-evict L1 cache on benchmark execution time.	49
4.8	The cache hit rate of each level of cache, before and after the L1 cache is added.	50
4.9	The effect of the increased core frequency on the overall performance. As we increase the core frequency by $1.845\times$, the performance improved by $\approx 1.5\times$	51

5.1	NTT kernels profile of ALU utilization, cache hit rates and memory intensity . . .	53
5.2	Inter-CU communication: Traditional (left) versus proposed on-chip network. . . .	55
5.3	Proposed hierarchical on-chip network featuring a concentrated 2D torus topology.	56
5.4	Influence of individual proposed micro-architectural extension on architectural performance metrics. Metrics illustrate a cumulative profile where each enhancement builds upon the preceding set of improvements	64
5.5	Speedup achieved from each micro-architectural extension. The baseline refers to a vanilla MI100 GPU. The reported speedup is cumulative, with each micro-architectural enhancement building upon the previous ones	65
5.6	Exploring the impact of on-chip memory size on FHE workload performance. . . .	67
6.1	The CPU profile from the Go pprof tool, while running the FIR benchmark timing simulation.	76
6.2	The speedup achieved through parallel simulation when running emulation mode. .	77
6.3	Execution time breakdown in goroutine analysis view, using the Go trace tool. . . .	78
6.4	The synchronization blocking profile from the Go trace tool, while running the ReLU benchmark parallel emulation.	79
6.5	The speedup achieved through parallel simulation when running timing simulation.	80
6.6	The synchronization blocking profile from Go trace tool for ReLU benchmark parallel timing simulation	81
6.7	Execution performance comparison comparing the baseline and improvements made to the parallel engine.	82
6.8	The synchronization blocking profile after applying the reversing-order strategy. . .	84
6.9	Performance comparison between the baseline, the modified Compute Unit and the new Workgroup Dispatcher.	86
6.10	Emulation performance comparison between the original ALU and the new ALU implementations.	87
6.11	Timing performance comparison between the original ALU and the new ALU implementations.	88
6.12	The speedup achieved through parallel simulation when running emulation mode on platform1.	89
6.13	The speedup achieved through parallel simulation when running emulation mode on platform2.	90
A.1	The synchronization blocking profile from Go trace tool for ReLU benchmark parallel emulation.	108
A.2	The synchronization blocking profile from Go trace tool for ReLU benchmark parallel timing simulation.	109
A.3	The synchronization blocking profile after applying the reversing-order strategy. . .	110

List of Tables

2.1	The HE building blocks using CKKS.	24
2.2	The CKKS parameters and descriptions.	26
2.3	Practical parameters for our FHE operations.	26
4.1	The list of microbenchmarks used to calibrate NaviSim.	35
4.2	The configurations of validation platforms.	41
4.3	Simulator configuration	42
4.4	Full Benchmarks	44
5.1	Cycle counts for 64-bit modulus instructions comparing MOD and WMAC features	59
5.2	MI100 GPU Parameters	62
5.3	Performance of various FHE building blocks	63
6.1	Benchmarks	73
6.2	Simulator configuration	74
6.3	Evaluation platform	74
6.4	Steps to tune the performance of a simulator	91

Abstract of the Dissertation

A Design Methodology for Producing Highly-Adaptable and High-Performance Simulation Frameworks

by

Yuhui Bao

Doctor of Philosophy in Electrical and Computer Engineering

Northeastern University, December 2024

Dr. David Kaeli, Advisor

Computer architecture simulators play an essential role in the development and optimization of computer hardware. A variety of simulators have been developed to explore the design space of CPUs, GPUs, and customer accelerators. As GPUs continue to grow in popularity for accelerating demanding applications, such as high-performance computing and machine learning, GPU architects have been pushing the envelope of GPU performance in every new GPU generation. GPU vendors (e.g., NVIDIA and AMD) have been introducing subsequent generations of GPU architectures and products with updated instruction set architectures (ISAs) and new microarchitectural features every 2-3 years. Modeling the state-of-the-art architecture is a crucial feature of GPU simulators, which are used to characterize and accelerate challenging workloads facilitating performance evaluation and design exploration. However, the effort required to design and construct an accurate and performant simulator is huge. Due to the rapid rate of innovation in GPU technology, any simulator that is over-customized to capture the design of a specific architecture will quickly become outdated. Thus, we need to develop a design methodology for simulators that can guard against this trend, embracing future architectures.

In this dissertation, we propose a design methodology for producing highly-adaptable and high-performance simulation frameworks. We aim to design simulators featuring high adaptability, being able to accommodate future alterations or extensions, high performance and high fidelity. We leverage the Akita simulator framework to enable the modular and extensible design of various GPU components. To fulfill the goal of high fidelity, we design a set of microbenchmarks to evaluate individual GPU subsystems. We demonstrate how we follow our design methodology to achieve a highly-adaptable and accurate simulator — NaviSim, which provides the flexibility to support simulation of three different ISAs. To demonstrate the full utility of the NaviSim simulator, we

conduct a performance study of the impact of individual architecture features revealing the high flexibility and configurability of NaviSim. In addition, we showcase how NaviSim’s high adaptability contributes to design space exploration, offering solutions to enhance the performance of real-world demanding applications.

Fast simulation speed is one of the key requirements of any simulators. NaviSim is designed to support multi-threaded execution, which is able to leverage the parallel capabilities offered by today’s multi-core CPUs, enabling parallel simulation. In this thesis we identify key performance bottlenecks in terms of both serial and parallel simulation execute modes and optimize simulation speed. We also present lessons learned about efficient simulator design and provide guidance for future simulator developers.

Chapter 1

Introduction

Over the past 20 years, GPU (Graphics Processing Units) has become an essential power that drives the rapid growth of many scientific and engineering applications, revolutionizing the landscape of scientific and engineering advances. Compared with the traditional multi-core Central Processing Unit (CPU), GPUs are considered to be throughput-optimized device with thousands of compute cores and high memory bandwidth. Applications that exhibit a high degree of data-level parallelism can benefit greatly from the impressive performance scaling delivered by GPUs. This is especially in the artificial intelligence (AI) [4], where GPUs have been used to drastically reduce training times for DNNs (Deep Neural Networks) [5]. GPUs are not only an effective accelerator for AI workload, many data-driven applications have been accelerated using a GPU. As a result, GPUs have been widely deployed as part of 168 supercomputers found in the Top500 supercomputer list [6]. GPUs have also contributed to the widespread deployment of numerous edge computing platforms, embedded systems, and portable personal devices.

Tracing back the history of GPU, the term GPGPUs (General Purpose GPUs) marked a significant milestone in efforts to broadly perform general-purpose computation leveraging the parallel processing power of GPUs. GPUs were originally designed to render 3D graphics, though they have been re-architected to support general-purpose computation as high-performance accelerators for a growing range of markets. This widespread adoption of GPUs as the accelerator of choice has been partly driven by the development of more user-friendly and efficient programming frameworks such as CUDA [7], HIP [8, 9] and OpenCL [10]. These high-level programming languages provide a well-defined syntax, as well as optimizing compilers, drivers, and runtimes to allow developers to effectively exploit the parallel architecture of GPUs.

As GPUs evolved beyond serving only as graphics device, they have been extensively used

CHAPTER 1. INTRODUCTION

for various types of compute-intensive workloads that can benefit from their parallel processing power. GPU-accelerated linear algebra libraries, such as cuBLAS in CUDA for NVIDIA GPUs, and hipBLAS from AMD’s ROCm libraries, are widely used in compute-intensive applications supporting acceleration across a range of fields. In addition, scientific simulations often utilize GPUs for complex computations. Many scientific simulation workloads, such as weather forecasting, fluid dynamics, and molecular modeling, involve the execution of a large number of mathematical operations, working with extensive datasets, running complex algorithms that consume a considerable amount of processing time and power. GPUs can accelerate these computations by executing thousands of computations simultaneously, while streaming data concurrently, thus reducing the overall processing time.

Computer architecture simulators play an essential part in the development and optimization of computer hardware [11]. Due to the huge cost associated with building real hardware for testing and verification, computer architecture researchers often rely on simulation and modeling techniques to evaluate different design options. Architectural simulators are commonly used to guide early design space exploration, performance optimization, and pre-silicon verification, supporting more efficient and cost-effective development cycles. Having access to an architectural simulator that delivers high fidelity, high flexibility, and high performance is essential for GPU architecture research. However, building an accurate and performant simulator is a very challenging task. Designing an architectural simulator that can accurately reproduce the behavior of the hardware being modeled, is easy to extend or modify, and can satisfy simulation performance requirements is a major effort.

1.1 Why Does the Community Continually Need New and Adaptable Simulators

1.1.1 Dynamics in the GPU Market

Every 2-3 years, GPU vendors (e.g., NVIDIA and AMD) have been introducing subsequent generations of GPU architectures and products, each equipped with a variety of innovative designs. While some GPU generations maintain the nearly the same instruction set architecture, others introduce major architectural overhauls, changing many aspects of the GPU’s instruction set and system organization. Take for example AMD’s Graphics Core Next (GCN) architecture [2], which has been a dominant GPU architecture for the last decade (2011-2019). GCN continued to evolve through five generations, with each generation introducing a few new features in each generation. In 2019,

CHAPTER 1. INTRODUCTION

AMD moved away from the GCN architecture and started to roll out a new RDNA architecture [3]. One year later, AMD announced their CDNA architecture [12], equipped with a Matrix Core Engine to support AI and machine learning applications.

AMD’s RDNA architecture was a major redesign, as AMD modified nearly every aspect of the GPU’s architecture. This included changes to the instruction set architecture (ISA), core architecture, and memory hierarchy. In the cores, the RDNA architecture reduced the number of co-scheduled threads (i.e., the wavefront) from 64 to 32 to cope with a higher level of thread divergence in modern workloads. The memory hierarchy was also extended, adding an extra layer of cache between the original L1 and L2 caches. This reduced the burden of the L2 caches and simplified the massive L1 to L2 network. However, not all exact details of industrial designs are disclosed and some hidden enhancements were also made under the hood. In comparison, the CDNA architecture mostly inherits the design of GCN architecture, but with a brand-new instruction set architecture and augmented processing units targeting specialized workloads.

1.1.2 Accelerating Challenging Workloads

Although GPUs provide great potential to accelerate a variety of workloads, some workloads face performance issues where they cannot efficiently utilize the parallel computing resources provided on the GPU. These performance issues can be attributed to factors such as thread divergence, irregular memory access patterns, and other workload characteristics, impacting our ability to optimize the performance of these workloads.

Any workload that includes inherent parallelism across vectors or threads, and can be executed concurrently, can leverage the capabilities of GPUs. However, the amount of effort required to modify an algorithm to expose this parallelism targeted for GPU execution can vary greatly. One class of workload that can map naturally to a GPU involves embarrassingly parallel code. However, navigating real-world workloads proves to be a challenging task, as achieving the anticipated ideal performance gain is not straightforward. This complexity is exemplified when considering intricate workloads such as graph neural networks (GNNs) and homomorphic encryption (HE). In these instances, the inherent characteristics of the workloads present challenges during mapping to a GPU. Let us focus on HE workloads more closely.

Fully homomorphic encryption (FHE) offers ideal security by allowing computations to be run directly on encrypted operands. FHE adopters can offload their encrypted private data to third-party cloud service providers while preserving end-to-end privacy. However, one challenge in

deploying FHE in real-world systems is the high computational costs associated with FHE operations. Computations on data encrypted via state-of-the-art HE schemes, such as CKKS [13] and TFHE [14], experience a slowdown of 4–6 orders of magnitude, as compared to running the same computation on unencrypted data [15].

Many prior studies have tried to accelerate FHE kernels by developing GPU-based implementations and applying a range of optimizations, including microarchitectural improvements [16], low-level GPU kernel optimization techniques [17], and multi-GPU optimization [18]. Even though these implementations outperform CPU-based solutions, the performance benefits over an out-of-the-box GPU implementation is still limited. We show a roofline model [19] for one instance of the FHE when ran on an AMD MI100 GPU. We can see that there is considerable room for performance improvement (See Figure 1.1). Based on the specification of the AMD MI100 GPU, we plot its

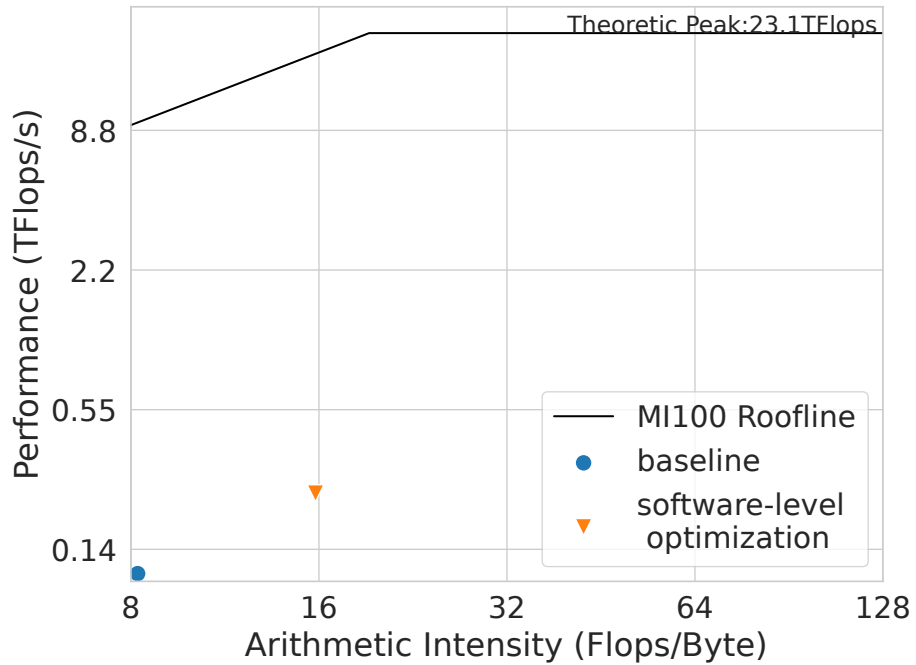


Figure 1.1: The roofline model for the FHE operation with size of the ciphertext $N = 2^{16}$ on AMD MI100 GPU

roofline model. In the roofline model plot, we indicate the performance in tflops/s and the arithmetic intensity(AI) in flops/byte measurements of the baseline and optimized GPU implementation of one FHE operation.

Despite the potential benefits of parallel processing on a GPU, FHE operations, which is a

memory-bound workload, inherently faces limitations to exploit the massive computation resources and high memory bandwidth. With closer examination of the FHE workload, we can characterize the inherent bottlenecks (e.g., large (dozens of MB) ciphertexts, a massive volume of integer arithmetic operations, and irregular memory access patterns). These issues present a challenge when mapping to existing GPU architectures. Historically, GPUs have been designed to execute thousands of threads in parallel, exploiting regular memory access patterns and rich floating-point computations.

1.1.3 Common Issues with Simulators

Inability to simulate new architectures Most simulators are designed to simulate a specific instruction set architecture, typically supporting only a single microarchitecture.

This lack of up-to-date simulation tools within academia is potentially harmful to the computer architecture research community. Research papers often describe their research methodology using stating that “we modified simulator *S*, which was originally validated against product *A*, to model product *B*, changing the number the number of cores and the cache sizes.” Typically, this product *B* is a few generations later than product *A*, and the study implicitly assumes that the simulator can still correctly model the new architecture. However, the nuances in the architecture design and parameter selection may significantly impact the performance characteristics.

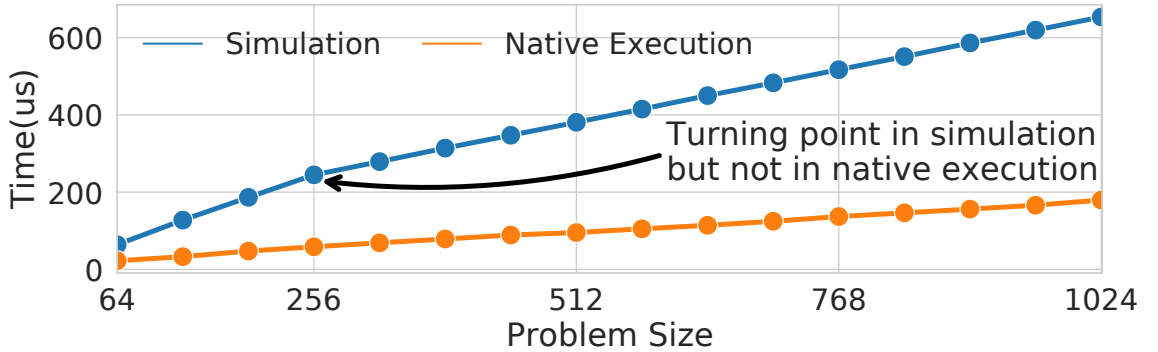


Figure 1.2: A comparison of a simulator’s reported kernel execution time and the native RX 5500 XT GPU execution time when running the BICG benchmark from PolyBench [1].

Looking at an example, simply modifying the publicly available parameters (core count, frequency, cache sizes, DRAM bandwidth, etc.) of MGPUSim [20], which was validated against a 2015-released R9 Nano GPU to model the RX 5500 XT GPU (RDNA architecture), as shown in Figure 1.2, can result in simulation errors as large as 318% when running the BICG benchmark

CHAPTER 1. INTRODUCTION

from PolyBench [1]. Not only are the performance discrepancies high, but the performance does not follow a trends we have measured on RX 5500 XT hardware. The major inaccuracies reported above suggest that simply adjusting publicly known parameters is insufficient to model a brand-new architecture and can lead to misguided conclusions. A careful redesign and re-calibration of the simulator infrastructure is necessary to provide a trustworthy baseline model for the next generation of GPU architecture research.

Lack of Modularity When designing a simulator, it is important to utilize high modularity of the code, as this will allow users to more easily modify and extend the simulator to fit the needs of their intended study. Without modularity, implementing extensions to a simulator can be challenging and error prone. The main problem will be the lack of independence between simulator components. Users will find it nearly impossible to replace an existing simulator component with their own component, without causing any side-effects in other parts of the simulator.

This lack of modularity will impact a user’s ability to quickly and effectively modify or extend the GPU simulator. Additionally, hardware architects will have difficulty performing design space exploration when introducing new features to the current design. They will need to frequently modify the simulator to implement and verify their ideas.

1.2 Challenges in Designing a GPU Simulator: Implications of Previous Considerations

In light of the previous discussion, we need to design and formulate a GPU simulator characterized by a high degree of adaptability. This simulator should demonstrate the capability to support multiple existing GPU instruction set architectures and microarchitectures. The true value of the simulator will lie in its ability to accommodate future alterations or extensions.

We need to develop a strategy to guide simulator design and development, with the goal of producing tools that possess a high degree of adaptability. We focus on the high modularity of the code and the ability to extend the simulator without modifying it. To ground our discussion in reality, we have developed NaviSim [21], a novel GPU simulator that models the AMD RDNA and CDNA architectures. Reflecting on the design and development process we used for NaviSim, we highlight the simulator’s flexibility to support simulations of three different ISAs and two different microarchitectures of three GPU architectures. NaviSim also features high extensibility, allowing developers to add, not only new components, including new function units and interconnects, and

new types of instructions, but also new ISA emulators and new microarchitectural features, without requiring major modifications. In addition, we showcase how NaviSim’s high adaptability contributes to design space exploration, offering solutions to enhance the performance of demanding applications. NaviSim can be used to identify hardware bottlenecks during workload execution by identifying the microarchitecture-level bottlenecks and performing design space exploration on different architectural features fulfilling the special needs of the intricate workloads.

1.3 The Speed of Simulation

Over the past decade, the rapid evolution of GPU architectures has been driven by the demands posed by high-performance computing (HPC) and machine learning (ML) workloads. Given the nature of these workloads (long running and potentially highly memory intensive), they pose challenges when designing a GPU simulator. Ideally, we would like to be able to quickly explore a wide design space. Rapidly evaluating the impact of a variety of new features across a diverse set of workloads is necessary for GPU architects to quantify the potential of their architectural innovations for future GPUs. A widely acknowledged problem encountered with architectural-level or cycle-level simulators is their slow simulation speed [22]. Usually, simulating even a single cycle requires much more resources and time than executing one cycle on the hardware, causing a significant slowdown compared with actual hardware. Thus, simulating a single second of execution could potentially take days, weeks, or even years. We quantify and show the slowdown of running different benchmarks in NaviSim in Figure 1.3. Slowdown, as a metric, is defined as the wall clock time of the simulation over the simulated kernel execution time. This result suggests that the simulation slowdown factor varies among different benchmarks and is impacted by the design and implementation of the simulation. On average, emulation exhibits a slowdown of roughly 5 orders of magnitude and a timing simulation experiences nearly a slowdown of 6 orders of magnitude.

Ideally, a simulator should at least satisfy three desirable properties: it should be accurate, fast, and execute a wide range of workloads [11]. While *fast* is not as clearly defined as other properties, due to the lack of a set of performance metrics to evaluate the architectural simulator in terms of execution speed. Prior studies targeting the speed issue focused on employing a range of approaches, including parallel or distributed simulation [23, 24], sampling-based techniques [25, 26, 27], and trading off simulator component fidelity for speed [28]. However, these approaches often fail to consider the opportunities hidden in the simulator design, overlooking potential software optimizations that could greatly enhance simulation speed.

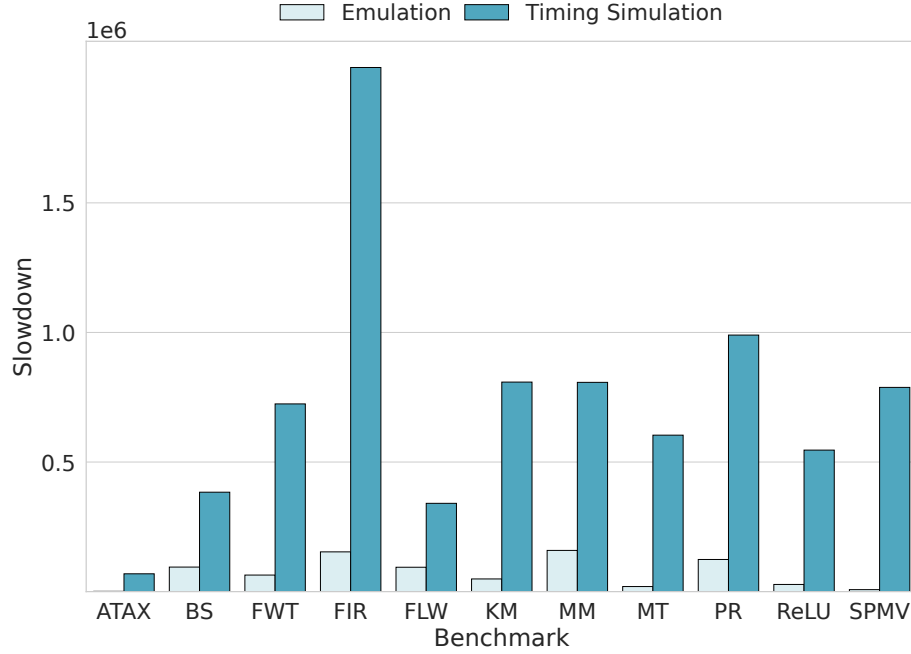


Figure 1.3: Simulation slowdown of benchmarks running on NaviSim under emulation mode and timing simulation mode

In this thesis, we will develop a methodology for evaluating the performance of architecture simulators. Equipped with our methodology, we apply our methods to optimize the simulation speed NaviSim. To achieve this goal, we define a set of performance metrics that help evaluate the simulation speed in terms of various simulated metrics (e.g., kernel execution time, number of simulated instructions, number of memory transactions). We believe an extensive evaluation of the simulator speed across representative configurations can provide useful guidance to better understand any bottlenecks present in the simulator, as well as how to remedy them. Next, we conduct a software-based profiling, understand the impact of various workloads on simulator throughput. The profiling results help us to examine better modeling strategies and software engineering techniques that we can employ to improve simulator throughput. Based on our profiling results, we perform performance tuning of the NaviSim simulator from a software perspective. Since today’s simulators are run on multi-core CPU system, we study the parallel simulation mode of NaviSim (NaviSim is one of the few simulators that is multi-threaded) and improve the parallel simulation scalability. We summarize lessons learned about efficient simulator design and attempt to provide guidance for future simulator developers.

1.4 Contributions of This Thesis

The goal of this thesis is to propose a design methodology for producing highly-adaptable and high-performance simulation frameworks. We develop a state-of-the-art GPU simulator to demonstrate how we follow this methodology to design and develop a simulator achieving high-adaptability and high performance. We also develop metrics and methods for evaluating the performance of architectural simulators. The contributions of this thesis include:

- We present a simulator development methodology that can be generalized to design any architectural simulator that aligns with the goals of high modularity, extensibility, and fidelity. To provide a concrete example of the general design methodology, we architect a new GPU simulator named NaviSim [21], which can simulate three instruction sets and two different microarchitecture implementations of three GPU architectures. Given the inherent flexibility in our emulator design, NaviSim enables the modeler to mix and match instruction sets with different microarchitecture implementations. On top of this feature, we conduct a performance study of the impact of individual architecture features, providing a highly flexible and configurable framework.
- We demonstrate the adaptability of NaviSim to enable design space exploration through a use case of accelerating FHE workloads on AMD CDNA GPU architecture. Leveraging the high flexibility NaviSim provides, we design and develop a set of GPU-based microarchitectural extensions (GME) to the existing CDNA architecture.
- We present a methodology for examining the key simulator design decisions that simulator developers need to consider to minimize unnecessary simulation overhead and improve parallel simulation efficiency. We introduce a set of metrics and methods to evaluate the simulation speed in terms of various simulated metrics. Based on the profiling results of NaviSim codebase, we conduct a thorough analysis of the factors that impact the simulation speed to better understand the characteristics of detailed, software-based architectural simulation. Based on the evaluation and analysis, we propose and develop software-based solutions to improve simulation speed.

1.5 The Organization of this Proposal

This dissertation is organized as follows. Chapter 2 presents background knowledge associated with this dissertation, including: 1) GPU programming frameworks and architectures; 2) the basics of architectural simulators; 3) performance evaluation approaches and associated profiling tools; 4) an introduction to fully homomorphic encryption (FHE). Chapter 3 presents prior work related to developing state-of-the-art GPU simulators and emulators, and extending GPU architecture to accelerate challenging workloads. Chapter 4 present our design methodology and the NaviSim GPU simulator. Chapter 5 presents a use case design space exploration using NaviSim. Chapter 6 presents our methodology for designing and optimizing high-performance simulators. In Chapter 7, we conclude this dissertation and discuss some directions for future work.

Chapter 2

Background

In this chapter, we provide important background that will help the reader better understand the contributions of this thesis. We begin by describing GPU programming frameworks and GPU architectures, covering how GPU hardware architectures support GPU software programming models. We then introduce the basics of architectural simulators. Next, we cover some performance evaluation approaches and associated profiling tools. Finally, we introduce fully homomorphic encryption (FHE), a powerful technique in the privacy-preserving computation domain, which is used as an example application in this work.

2.1 GPU Programming Frameworks

A GPU program typically can be separated into the GPU portion and the CPU portion. The portion of the program that executes on the GPU is called a *kernel*. The portion of the program that runs on the CPU (i.e., the host program) launches kernels using vendor-provided runtime APIs. A set of GPU-controlling APIs and a GPU programming language compose a GPU programming framework. CUDA, HIP, and OpenCL are three of the most popular GPU programming frameworks. These frameworks share a similar idea of exposing the abstraction of a Single Program Multiple Data [29] (SPMD) model to let users leverage the underlying parallel compute engines in GPUs. We introduce the programming model using OpenCL terminology, as it compiles and runs on NVIDIA, AMD and Intel GPUs, and other programming models (e.g., CUDA or HIP) use similar semantics.

A kernel consists of a number of work-items. Work-items are similar to CPU threads; they execute the same program concurrently, but work on different data. Work-items can be grouped into work-groups. Multiple work-groups compose a kernel in the form of NDRange [10], which can

CHAPTER 2. BACKGROUND

be arranged in 1D, 2D, or 3D orientation. The work-items in a work-group can be synchronized using barriers and can share a small, but fast, local data share (LDS) memory. All the work-items of a kernel can access the global memory but there is no way to synchronize all the work-items. This indicates that kernel-level synchronization requires the host to stop the kernel and relaunch the kernel.

On a GPU, a subset of work-items (typically 32 to 64) in a work-group are organized in a wavefront. AMD GPUs issue instructions at a wavefront granularity—every time the instruction scheduler issues one instruction, the ALU repeats the operation for each work-item within the wavefront. All the work-items in a wavefront run in *lock-step* manner. With lock-step execution, all the work-items in a wavefront have to execute the same instruction, even if only part of the wavefront (i.e., some of the threads) takes a different execution path (i.e., due to a branch or thread divergence). Thread divergence negatively impacts GPU performance, as only part of the work-item execution will take effect.

GPU frameworks provide APIs to support data movement between the host and devices. OpenCL uses *command queues* to manage tasks such as memory copies and compute kernels. The command queue is created and managed by the GPU driver, and it serves as a communication channel between the CPU and the GPU. It maintains a sequence of GPU commands that the GPU will execute in order. Commands from different queues can execute in parallel.

2.2 Instruction Set Architecture versus Architecture versus Microarchitecture

In computer architecture, *Instruction Set Architecture (ISA)*, *architecture*, and *microarchitecture* are commonly used when designing hardware components and hardware/software interfaces to build a computing system. They refer to different aspects of the design and organization of a computer.

Instruction Set Architecture (ISA): ISA defines the set of instructions that the computing hardware can execute. It is the programmer's view of a computer and is concerned with the interface between software and hardware. It is defined by the instruction set (language) and operand location (register and memory).

Architecture: Architecture encompasses the entire design and organization of a computer system, including its components, interconnections, memory hierarchy, input/output systems, and

CHAPTER 2. BACKGROUND

more. It is a broader concept that includes ISA as a superset.

Microarchitecture: Microarchitecture refers to the implementation details of the architecture at a lower level. It focuses on the specific arrangement of registers, ALUs, memories, and other building blocks to form a microprocessor. It translates the high-level architecture into a specific, efficient implementation by providing organization of the dataflow and controls, the logic design, and the physical implementation.

In summary, ISA defines the instruction set as an interface to the programmer, architecture encompasses the overall system design, and microarchitecture deals with the internal implementation details of the processor for efficient instruction execution. The concept of architecture overlaps with ISA and microarchitecture and it is mostly used, but ISA and microarchitecture focus on the interface and physical implementation details, respectively.

2.3 GPU Microarchitecture

Compared with a CPU, a GPU provides much higher computational throughput and memory bandwidth, primarily based on differences in the organization and number of different functional units. We provide an overview of AMD GPU microarchitecture, using CDNA architecture [12] as an example.

The CDNA architecture (see Figure 2.1) adopts a highly modular design that incorporates a Command Processor, Shader Arrays (including Compute Units and L1 caches), an on-chip network connecting the core-side L1 caches and the memory-side L2 caches, and DRAM.

2.3.1 Kernel Dispatching

When a GPU program needs to launch a kernel on a GPU, the program communicates with the driver to send a command to the GPU. The GPU employs a Command Processor (CP) to process all the incoming commands from the CPU, including memory copying and kernel launches. The CP allocates an Asynchronous Compute Engine (ACE) to work as a work-group dispatcher to perform kernel dispatch. The ACE unit is responsible for breaking down kernels into work-groups and wavefronts, as well as assigning the work-groups to the available Compute Units (CUs). All work-items from one work-group must execute on the same CU, although a CU can concurrently process work-items from multiple work-groups. In general, an ACE unit can use different dispatching algorithms (e.g. round robin or greedy), to fill up each CU with a number of work-groups. In general,

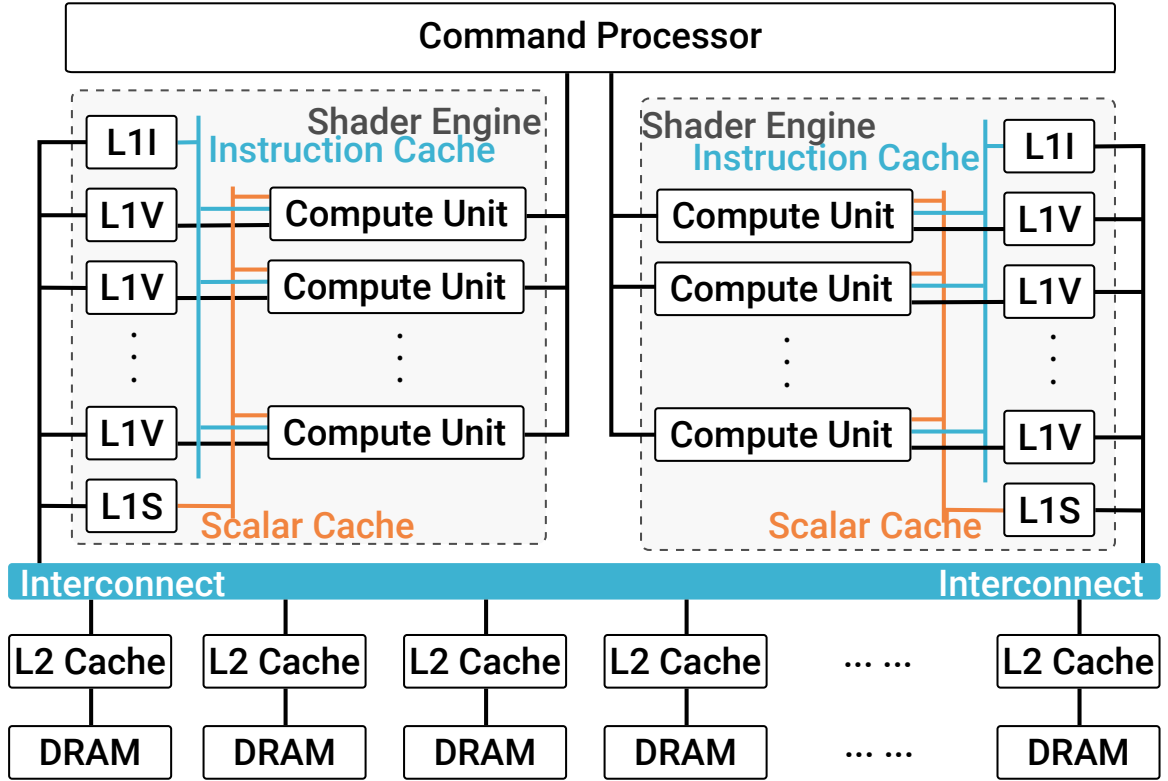


Figure 2.1: The Microarchitecture of CDNA GPUs

the number of work-groups in a kernel is larger than the number of CUs on a GPU can handle at the same time. In this case, the ACE fills all the CUs to their maximum capacity first and then pauses dispatching. Once any CU completes the execution of a work-group, the ACE then dispatches another work-group to that CU. Due to this staggered execution of work-groups within one kernel, the GPU lacks the capability to synchronize all work-items simultaneously. There are multiple ACEs in one GPU, allowing a GPU to run multiple kernels at the same time.

2.3.2 Compute Unit

The Compute Unit (CU) is responsible for instruction execution and data processing. As shown in Figure 2.2, a CU is equipped with a scheduler, a set of instruction decoders, a set of register files, a set of ALUs (4 SIMD units and a scalar unit), and memory interfaces. The scheduler decides which wavefront can fetch instructions and which wavefront can issue its instructions to be decoded. Up to 40 wavefronts can be fetched by the scheduler and stored in wavefront instruction buffers. There are 6 different execution units, including a branch unit, a scalar unit, a Local Data Share

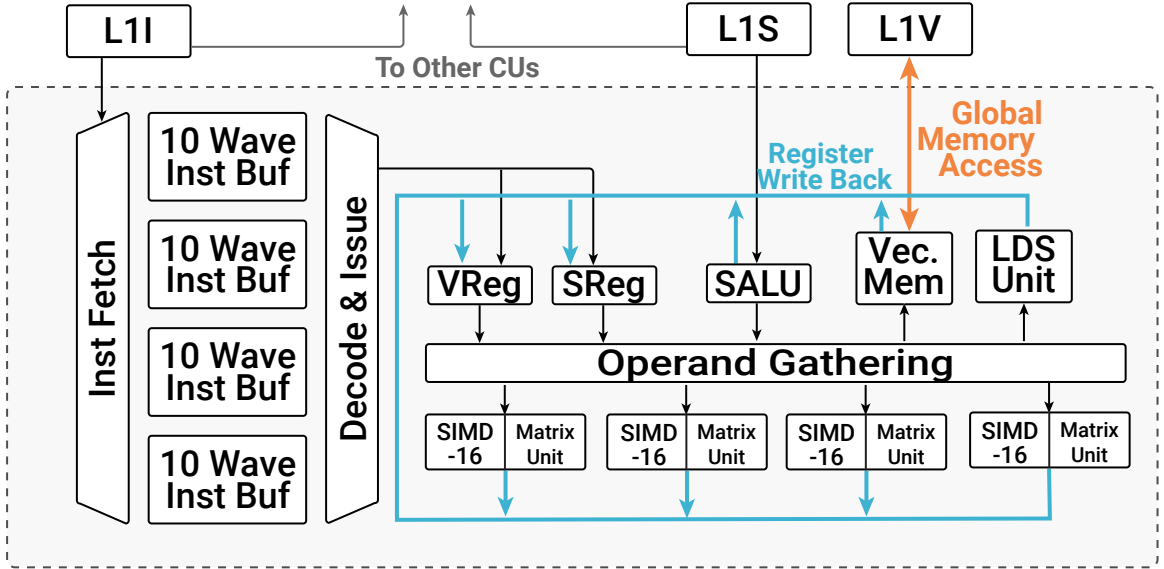


Figure 2.2: The Microarchitecture of CDNA Compute Unit

(LDS) unit, a vector memory unit, four Single-Instruction Multiple-Data (SIMD) units, and four matrix core engines. The scalar unit also serves as a scalar memory unit, responsible for executing instructions that manipulate and load data shared by work-items in a wavefront. The LDS unit executes Local Data Share instructions and interacts with the local memory space. The vector memory unit is responsible for loading and storing multiple data (64 data elements in one wavefront in CDNA) in one instruction. Each SIMD unit is responsible for executing vectorized floating-point instructions for 10 out of the 40 wavefronts managed by the scheduler. Each SIMD unit is equipped with 16 single-precision Arithmetic Logic Units (ALUs). Therefore, each 64-work-item wavefront takes 4 cycles to finish the execution of one instruction. Each matrix engine handles a family of wavefront-level instructions—the Matrix Fused Multiply Add (MFMA). MFMA instructions perform mixed-precision arithmetic and operate on matrices.

2.3.3 The GPU Memory Hierarchy

The CDNA microarchitecture has a two-level cache hierarchy (see Figure 2.1). The L1 cache can be divided into the L1 scalar cache (mainly used for storing constant data, such as kernel arguments and pointers), an L1 instruction cache, and an L1 vector cache (a write-through cache that stores most of the data required by a CU). They are shown as L1S, L1I, and L1V in the figure. Each CU has a dedicated L1 vector cache. CUs in a Shader Array share an L1 scalar cache and

CHAPTER 2. BACKGROUND

an L1 instruction cache. All the L1 caches fetch data from L2 caches (L2s are write-back caches). Each L2 cache interfaces to a DRAM controller (typically implemented in HBM [30] or GDDR technology [31]). The L2 caches and the DRAM controllers are banked. Each L2 cache serves a memory bank of a certain address range. In total, there are 4 to 8 L2 caches that cover the whole address range of the DRAM. The L1 caches and the L2 caches are connected with a crossbar. The crossbar design can provide low-latency and high-throughput communication channels for the L1 and the L2 caches.

2.4 Architectural Simulators

Architectural simulation plays an important role in the realm of computer architecture design and research, offering powerful ways to model and analyze the intricate behavior of computer systems. At its core, architectural simulation involves the creation of software-based models that mimic the behavior of hardware components within a computer architecture. This enables researchers, engineers, and system designers to experiment with and evaluate different architectural configurations without the need for physical hardware (i.e., pre-silicon).

2.4.1 Microarchitecture Simulation

The two main goals of microarchitectural simulation are: i) to model new design ideas for some parts of a computer system and ii) to enable the evaluation, prediction, and analysis of the cycle-level performance, area, and power/energy [32]. To achieve these goals, software-based simulation methods are used. Each simulator is developed in a high-level programming language. The goal is to model the behavior of the hardware. Some of the behavior of the hardware is modeled at a high level of abstraction (i.e., the exact time spent on wires is estimated, though the system behaves deterministically). The purpose for this abstraction is to simplify the execution of the simulator (i.e., not every cycle is modeled precisely). This simplification helps to speed up the overall runtime of the simulation.

In terms of designing a simulator, a critical decision to make is selecting the right level of abstract appropriate for the model. This may impact both the speed of the simulator (i.e., the more detailed a simulator, the greater the slowdown) and the accuracy of the simulation (a more detailed simulator should produce more accurate simulation results).

CHAPTER 2. BACKGROUND

An architectural simulator is a detailed software modeling tool that fulfills these goals. Discrete event simulation [33] (DES) is one of the most commonly and widely used methods of simulation in the computer architecture field. Discrete event simulation involves modeling the system as a sequence of events that occur at distinct points in time, with the state of the system changing only at these event points. Key characteristics of discrete event simulation include:

Event-based Modeling: The simulation model is built on events that represent the work or actions that the simulator needs to do in a certain future time. These events get handled at specific points in time and drive the simulation forward.

State Changes: The state of the system changes only when events occur. Between events, the system remains in a stable state. This makes DES particularly suitable for modeling systems with dynamic and variable behaviors.

Concurrency and Synchronization: DES allows for the modeling of concurrent processes and the synchronization of events, making it well-suited for systems where multiple entities interact or compete for resources. The Parallel Discrete Event Simulation (PDES) technique [34] is used to simulate different components in parallel and conservatively or optimistically synchronize them in fixed intervals.

Flexibility: DES inherently supports flexible and extensible designs, given that there are minimal dependencies between components within the simulator. DES allows customized definitions of events and integrates seamlessly with other software techniques that enhance adaptability.

Real-Time Simulation: DES is often used for real-time simulation [35], where events are modeled to occur at specific, discrete points in time, making it well-suited for modeling computer systems operating in cycles.

2.4.2 Classification of Simulators

Simulators can be divided into various types depending on the level of simulation detail, the input to the simulator and whether cycles can be skipped.

Functional versus Timing Simulators: Functional simulators model the functionality of the target device by recreating the hardware output. Functional simulators can provide basic program execution information such as instruction counts and cache hit/miss rates. In contrast, timing simulators model microarchitectural details, attempting to mimic the operations as they occur on

CHAPTER 2. BACKGROUND

the hardware. Timing simulators keep track of all of the cycles to provide detailed timing-related statistics.

Trace-Driven versus Execution-Driven Simulators: Trace-driven simulators use trace files as their inputs. These files contain recorded streams of instructions and/or addresses captured during a previous execution on the target hardware. The simulator reads a trace of instructions or addresses that were previously executed, so there is no need to decode the ISA of the target, which makes them relatively simple. Trace-driven simulators are constrained to studying specific scenarios and conditions, and are unable to support new designs that rely on dynamic information. In an execution-driven simulator, instructions are first decoded and executed on the simulated machine directly. Execution-driven simulators provide greater flexibility, as there is no need to execute the application on real hardware first. Execution-driven simulation supports more detailed modeling of complex interactions, instead of replaying pre-recorded system behavior. However, execution-driven simulators are more complex compared to trace-driven simulators (e.g., these simulators may require to model an instruction decoder and instruction caches) and may take longer to simulate.

Cycle-based versus Event-based Simulators: In a broader sense, a cycle-based simulator is considered a DES. It is a simplified DES having events occur on every clock cycle. The main difference between cycle-based and event-based simulators lies in the fact that cycle-based simulators can not skip any cycle, even if there is no work to do in that cycle, which increases simulation time unnecessarily. Many simulators combine these two by modeling some parts of the simulated target on a cycle-by-cycle basis and others on an event-driven basis.

2.5 Performance Evaluation Tools

Measuring and analyzing the performance, specifically timing information, of an application involves collecting performance counter information from various hardware and software components. Various tools and techniques are available to support performance tuning, including profiling tools provided by the operating system (e.g., `perf` [36] and `gprof` [37] for Linux and Windows Performance Toolkit(WPT) [38] for Windows), specific vendors (e.g., Intel, AMD), and cross-platform low-level libraries. Some tools focus on user-level code in specific languages, such as `gprof` [37] for C and Fortran, `pprof` [39] for Golang.

CHAPTER 2. BACKGROUND

2.5.1 perf

perf [36] is a Linux command-line tool that operates at the system level. It provides a wide range of performance monitoring features, including CPU utilization, memory access patterns, cache misses, and other hardware-level events. Perf is not limited to profiling only user-level code, but extends to system-wide analysis, allowing you to investigate the performance of the entire system. It is often used for analyzing low-level system performance, kernel-level events, and hardware performance counters. perf comes pre-installed on many Linux distributions.

2.5.2 PAPI

Performance Application Programming Interface [40] (PAPI), is a library and set of tools designed to provide a consistent interface and methodology for collecting performance counter information from the CPU and other hardware components. It allows developers to access low-level hardware performance counters in a platform-independent way, making it easier to gather detailed performance metrics across different architectures. PAPI includes a set of APIs that developers can use to integrate performance monitoring capabilities into their applications. These APIs support C and Fortran programming languages.

2.5.3 pprof

pprof is a profiling tool associated with the Go programming language. It is used for collecting and analyzing profiling data from Go applications. pprof is a part of the Go standard library and provides built-in support for profiling Go applications. It allows developers to gather insights into the runtime behavior of their Go code. Pprof supports CPU timing and memory profiling. It helps identify the functions and code paths where most of the CPU time is spent during the execution of a Go program. It also helps developers understand memory allocation patterns, identify memory leaks, and optimize memory usage in Go programs.

2.6 Fully Homomorphic Encryption

Fully homomorphic encryption (FHE) is a groundbreaking cryptographic technique that emerged as a solution to a longstanding challenge in the field of secure data processing and privacy. This form of encryption allows computations to be performed on encrypted data without ever revealing the underlying information(See Figure 2.3). This further paves the way for a wide range of practical

CHAPTER 2. BACKGROUND

applications, from secure outsourced computation and private data analysis to privacy-preserving machine learning.

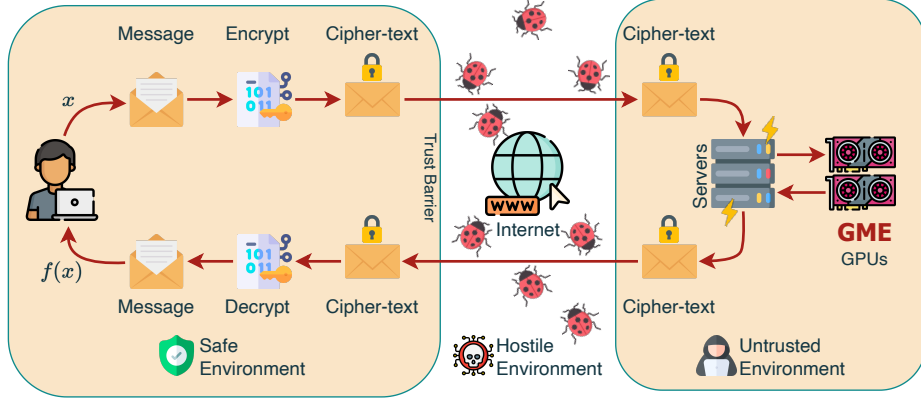


Figure 2.3: FHE offers a safeguard against online eavesdroppers as well as untrusted cloud services by allowing direct computation on encrypted data.

The idea of homomorphic encryption was proposed in 1978 by Whitfield Diffie and Martin Hellman. A few earlier homomorphic encryption schemes are partial homomorphic encryption schemes, because they can support only limited operations and have limited usability. Paillier’s cryptosystem [41] could not support multiplication operations, only addition operations are supported, while the RSA cryptosystem [42] only supports multiplication, but not addition. Then in 2009, Gentry presented, in his seminal paper [43], a new approach to FHE that overcame some of the key challenges associated with earlier proposals. Gentry’s approach uses lattice-based cryptography and enables both addition and multiplication operations. The initial version of the homomorphic encryption scheme was incomplete because, while it could support any number of additions, it could only support a limited number of multiplications. Then later on, Gentry made some improvements by introducing an operation known as bootstrapping. This improvement elevated homomorphic encryption to a new level. This new approach to FHE was called *fully* homomorphic encryption because it could support any number of additions and any number of multiplications, turning the theoretical concept of fully homomorphic encryption into a practical reality.

2.6.1 The CKKS FHE Scheme

In this thesis, we will utilize the Cheon-Kim-Kim-Song (CKKS) FHE scheme to demonstrate the flexibility of our simulation infrastructure to be adapted to accelerate this complex scheme. CKKS can support a wide range of privacy-preserving applications by allowing operations on

CHAPTER 2. BACKGROUND

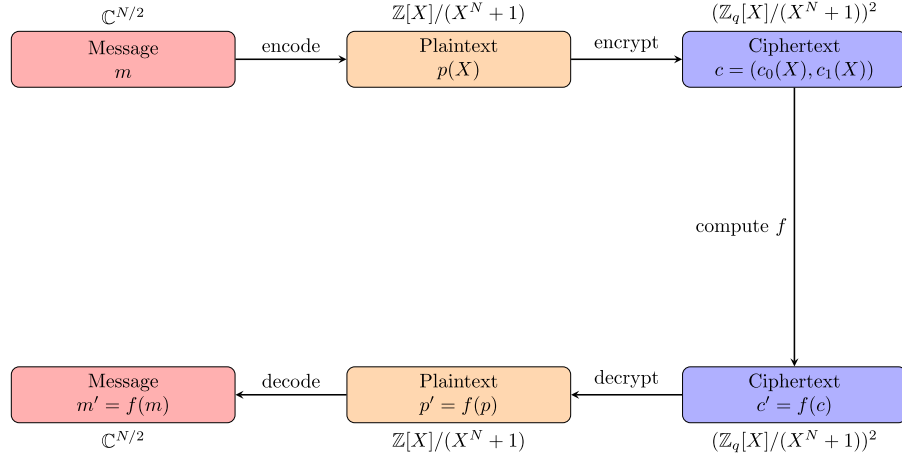


Figure 2.4: The operation flow of the CKKS scheme.

floating-point data. Figure 2.4 provides a high-level view of CKKS showing the flow of the operations performed in the scheme. We begin with a message m . The message space is defined as a space of complex numbers. Basically, this is a vector of complex values on which we want to perform certain computation. The first step is to encode this message to generate what we call plaintext. The plain text space is constrained by an irreducible polynomial and the values are the coefficients of this plaintext polynomial $\mathbb{Z}[x]$. After encoding, we encrypt the plaintext, which now is represented as a single polynomial, into a pair of polynomials to generate the ciphertext using a public key. The coefficients of the encrypted polynomial are defined as $\mathbb{Z}_q[x]$. We introduce the polynomial modulus Q to constrain the size of each coefficient. We can examine the ciphertext further to understand its content. Basically, the ciphertext consists of a pair of elements in the polynomial ring $R_Q = \mathbb{Z}_Q[x]/(x^N + 1)$. Each element of this ring is a polynomial $\sum_{i=0}^{N-1} a_i x^i$ with “degree-bound” $N - 1$ and coefficients a_i in $\mathbb{Z}_Q$. For a message $\mathbf{m} \in \mathbb{C}^n$, we denote its encryption as $\llbracket \mathbf{m} \rrbracket = (A_{\mathbf{m}}, B_{\mathbf{m}})$ where $A_{\mathbf{m}}$ and $B_{\mathbf{m}}$ are the two polynomials that comprise the ciphertext. After the message is encoded and encrypted, we can perform homomorphic encryption operations (denoted as f). Then we get a result in ciphertext, represented as c' . We need to decrypt it to plaintext and then decode it back to a result message m' . The basic datatype in CKKS is a vector with length N in both plaintext and ciphertext, and each element in the vector is of size $\log Q$ bits. N and Q are the main parameters, which define the size of the ciphertext, and also govern the size of the working data set that is required to be present in the on-chip memory.

2.6.2 Building Blocks of the CKKS FHE Scheme

Since CKKS scheme deals with the Learning with Errors (LWE) [44] problem, one of the challenges is how to handle the problem that the error introduced by homomorphic encryption (HE) operations grows as we perform a sequence of these operations. For example, the addition operation doubles the noise, and multiplication squares the noise. The noise keeps growing if we perform multiple levels of addition or multiplication operations then at one point the error is so large that we are no longer able to decrypt it. There are two solutions to this problem. One is we can choose large enough parameters so that we have a large space for the noise to grow. Another solution is that we can perform a bootstrapping operation.

Choosing a large enough value for $\log Q$ will help us preserve the underlying message because it allows more space for the error to grow, without losing precision. Also, there is a correlation between the value for $\log Q$ and the value of N . A large enough $\log Q$ value requires a large enough value of N so that we have enough security bits for the underlying encryption scheme. For 128-bit security, typical values of N range from 2^{16} to 2^{17} , and $\log Q$ values range from 1700 to 2200 bits for practical purposes. These large sizes of N and $\log Q$ are required to maintain the security of the underlying *Ring-Learning with Errors* assumption [44].

However, there are no commercially available compute systems that have a hundred-bit wide or thousand-bit wide ALU, which are necessary to process these large coefficients. A common approach for implementing the CKKS scheme on hardware with a much smaller word length is to choose Q to be a product of distinct word-sized primes $q_1, \dots, q_\ell$. Then $\mathbb{Z}_Q$ can be identified with the “product ring” $\prod_{i=1}^{\ell} \mathbb{Z}_{q_i}$ via the Chinese Remainder Theorem [45]. In practice, this means that the elements of $\mathbb{Z}_Q$ can be represented as an ℓ -tuple $(x_1, \dots, x_\ell)$, where $x_i \in \mathbb{Z}_{q_i}$ for each i . This representation of elements in $\mathbb{Z}_Q$ is referred to as the *Residue Number System* (RNS) and is commonly referred to as the limbs of the ciphertext.

The CKKS FHE scheme supports floating-point messages, so all encoded messages must include a scaling factor Δ with the purpose of reducing the precision loss. CKKS deals with real and complex numbers, which are continuous and have infinite precision. In contrast, digital computers work with finite precision. The scale factor is used to map real or complex numbers to a finite and discrete set of values. It is crucial to constrain error growth because it helps ensure that ciphertexts remain proportional to the scale factor, while the error remains the same. This scaling factor is typically the size of one of the limbs of the ciphertext, so its value Δ is typically equivalent to $\log q$.

As we can see, since ciphertexts are represented by polynomials, polynomial multiplication

CHAPTER 2. BACKGROUND

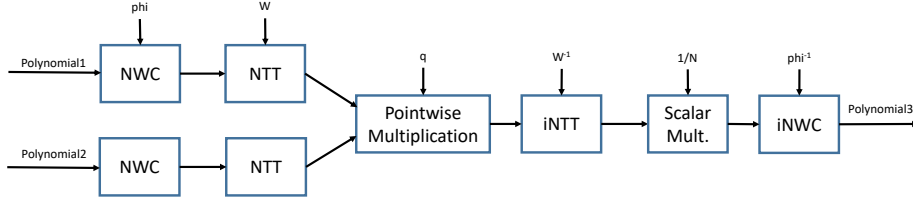


Figure 2.5: Polynomial Multiplication.

becomes one of the main operations in the CKKS scheme, and more generally, in lattice-based cryptography. Polynomial multiplication is a very expensive operation and in order to enable fast polynomial multiplication, by default, we represent polynomials as a series of N evaluations at fixed roots of unity[46]. We refer to this polynomial representation as the *evaluation representation*. Moving between the two polynomial representations requires a number-theoretic transform (NTT) or inverse NTT, which is the finite field version of the fast Fourier transform (FFT)[43]. After performing NTT on polynomials we can perform point-wise multiplication of the corresponding coefficients of the two polynomials. This allows polynomial multiplication to occur in $O(N)$ time instead of $O(N^2)$ time. Figure 2.5 shows the steps involved in polynomial multiplication. It consists of performing a negative wrapped convolution operation (NWC), followed by NTT operations, which is then followed by a point-wise multiplication. NWC is a multiplication with ϕ which is generated from the root of unity. ω and ϕ are N and $2N$ root of unity respectively. After the point-wise multiplication, we can perform an inverse NTT, then do a scalar multiplication with $1/N$, and then perform an inverse negative wrapped convolution (iNWC).

We list all the building blocks in the CKKS scheme in Table 2.1. All of the operations that form the building blocks of the CKKS scheme involve polynomial addition and multiplication, which further reduce to scalar modular additions and scalar modular multiplications. The ScalarAdd and ScalarMult are the two most basic building blocks that add and multiply a scalar constant to a ciphertext. PolyAdd and PolyMult add and multiply a plaintext polynomial to a ciphertext. We define separate ScalarAdd and ScalarMult operations (in addition to PolyAdd and PolyMult) because the scalar constant values can be fetched directly from the register file, which can help save expensive main memory accesses. Note that the PolyMult is followed by an HERescale operation to restore the scale of a ciphertext to Δ from scale Δ^2 . As we mentioned before, when multiplying messages together, this scaling factor grows as well. The scaling factor must be shrunk down in order to avoid overflowing the ciphertext coefficient modulus.

Table 2.1: The HE building blocks using CKKS.

Block	Computation	Description
ScalarAdd($\llbracket \mathbf{m} \rrbracket, c$)	$\llbracket \mathbf{m} + \mathbf{c} \rrbracket = (\mathbf{B}_{\mathbf{m}} + \mathbf{c}, \mathbf{A}_{\mathbf{m}})$	Add a scalar c to a ciphertext where, $\mathbf{c}$ is a length- N vector with element c
ScalarMult($\llbracket \mathbf{m} \rrbracket, c$)	$\llbracket \mathbf{m} \cdot \mathbf{c} \rrbracket = (\mathbf{B}_{\mathbf{m}} \cdot \mathbf{c}, \mathbf{A}_{\mathbf{m}} \cdot \mathbf{c})$	Multiply a scalar by a ciphertext
PolyAdd($\llbracket \mathbf{m} \rrbracket, \mathbf{P}_{\mathbf{m}'}$)	$\llbracket \mathbf{m} + \mathbf{m}' \rrbracket = (\mathbf{B}_{\mathbf{m}} + \mathbf{P}_{\mathbf{m}'}, \mathbf{A}_{\mathbf{m}})$	Add an unencrypted polynomial to a ciphertext
PolyMult($\llbracket \mathbf{m} \rrbracket, \mathbf{P}_{\mathbf{m}'}$)	$\llbracket \mathbf{m} \cdot \mathbf{m}' \rrbracket = (\mathbf{B}_{\mathbf{m}} * \mathbf{P}_{\mathbf{m}'}, \mathbf{A}_{\mathbf{m}} * \mathbf{P}_{\mathbf{m}'})$	Multiplying an unencrypted polynomial with a ciphertext
HEAdd($\llbracket \mathbf{m} \rrbracket, \llbracket \mathbf{m}' \rrbracket$)	$\llbracket \mathbf{m} + \mathbf{m}' \rrbracket = (\mathbf{B}_{\mathbf{m}} + \mathbf{B}_{\mathbf{m}'}, \mathbf{A}_{\mathbf{m}} + \mathbf{A}_{\mathbf{m}'})$	Add two ciphertexts
HEMult($\llbracket \mathbf{m} \rrbracket, \llbracket \mathbf{m}' \rrbracket, \mathbf{evk}_{\text{mult}}$)	$\llbracket \mathbf{m} \cdot \mathbf{m}' \rrbracket = \text{KeySwitch}(\mathbf{A}_{\mathbf{m}} * \mathbf{A}_{\mathbf{m}'}, \mathbf{evk}_{\text{mult}}) + (\mathbf{B}_{\mathbf{m}} * \mathbf{B}_{\mathbf{m}'}, \mathbf{A}_{\mathbf{m}} * \mathbf{B}_{\mathbf{m}'} + \mathbf{A}_{\mathbf{m}'} * \mathbf{B}_{\mathbf{m}})$	Multiply two ciphertexts
HERotate($\llbracket \mathbf{m} \rrbracket, r, \mathbf{evk}_{\text{rot}}^{(r)}$)	$\llbracket \mathbf{m} \ll r \rrbracket = \text{KeySwitch}(\psi_r(\mathbf{A}_{\mathbf{m}}), \mathbf{evk}_{\text{rot}}^{(r)}) + (\psi_r(\mathbf{B}_{\mathbf{m}}), \mathbf{0})$	Circular rotate elements left by r slots, ψ_r is an automorphism performed
HERescale($\llbracket \mathbf{m} \rrbracket$)	$\llbracket \Delta^{-1} \cdot \mathbf{m} \rrbracket = (\Delta^{-1} \mathbf{B}_{\mathbf{m}}, \Delta^{-1} \mathbf{A}_{\mathbf{m}})$	Restore the scale of a ciphertext from scale Δ^2 back to Δ

CHAPTER 2. BACKGROUND

The HEAdd operation is straightforward and adds the corresponding polynomials within the two ciphertexts. However, the HEMult and HERotate operations are computationally expensive, as they perform a key switch operation after the multiplication and automorph operations, respectively. In both the HEMult and HERotate implementations, there is an intermediate ciphertext with a decryption key that differs from the decryption key of the input ciphertexts. In order to transform this new decryption key back to the original decryption key, we perform a key switch operation. This operation takes in a switching key (either $\mathbf{evk}_{\text{mult}}$ or $\mathbf{evk}_{\text{rot}}^{(r)}$), a ciphertext and a ciphertext $\llbracket \mathbf{m} \rrbracket_s$ that is decryptable under a secret key s . The output of the key switch operation is a ciphertext $\llbracket \mathbf{m} \rrbracket_{s'}$ that encrypts the same message, but is decryptable under a different key s' .

Keyswitch operations itself are highly computationally expensive and it involves a sequence of sub-operations. To incur minimal noise growth during the key switch operation, the key switch operation requires that a polynomial be split into d_{num} digits, then raise the modulus before multiplying with the switching key, followed by a modulus down operation. The modulus raise and down operations operate on the coefficient representation of the polynomial, requiring us to perform expensive NTT and iNTT conversions. Moreover, the switching keys are the same size as the ciphertext itself, requiring us to fetch a large amount of data at once to multiply the switching keys with the ciphertext. Finally, there exists an operation known as bootstrapping [43] that needs to be performed frequently to de-noise the ciphertext. This bootstrapping operation is a sequence of the basic building blocks in the CKKS scheme, meaning that it suffers from the same compute and memory bottlenecks that exist in these building blocks, making it one of the most expensive operations. We list the parameters that define the CKKS FHE scheme in Table 2.2.

CHAPTER 2. BACKGROUND

Table 2.2: The CKKS parameters and descriptions.

Param	Description
N	Polynomial degree-bound
n	Length of the message. $n \leq \frac{N}{2}$
Q	Polynomial modulus
L	Maximum number of limbs in a ciphertext
$\mathcal{C}$	The set $\{q_0, q_1, \dots, q_L\}$ of prime factors of Q
ℓ	Number of limbs, number of factors in Q ;
dnum	Number of digits in the switching key
α	Number of limbs that comprise a single digit in the key-switching decomposition $\alpha = \lceil \frac{L+1}{\text{dnum}} \rceil$
P	Product of extension limbs added for raised modulus. Total extension limbs = $\alpha + 1$
fftIter	Multiplicative depth of bootstrapping linear transform
Δ	Scale multiplied during encryption
$\mathbf{m}$	A message vector of n slots
$\llbracket \mathbf{m} \rrbracket$	Ciphertext encrypting a message
$A_{\mathbf{m}}$	A randomly sampled polynomial from message $\mathbf{m}$
P	Encrypted message as a polynomial
P_m	Polynomial encrypting message m
$[P]_{q_i}$	q_i -limb of P
$\mathbf{evk}$	Evaluation key
$\mathbf{evk}_{rot}^{(r)}$	Evaluation key for <i>HE-Rotate</i> block with (r) rotations

Table 2.3: Practical parameters for our FHE operations.

$\log(q)$	N	$\log Q$	L	L_{boot}	dnum	fftIter	λ
54	2^{16}	1728	23	17	3	4	128

Chapter 3

Related Work

In this chapter, we review prior work related to the research areas covered in this dissertation. We first begin by discussing the state-of-the-art in GPU simulators and emulators. Next, we discuss related work on extending GPU architecture to accelerate FHE workloads. There is no directly related work of extending GPU architecture for FHE but the most closely related work focuses on Deep Learning (DL) workloads [47].

3.1 GPU Simulators

GPU simulators have been critical infrastructures that enable GPU architecture design and quantitative evaluation. To date, GPU architecture researchers have devoted major efforts to developing GPU simulators and emulators to support architectural exploration in both industry and academia.

3.1.1 Commercially-develop Simulators

GPU manufacturing companies, such as NVIDIA and AMD, have developed their own internal simulation tools. *NVArchSim (NVAS)* [28], developed by NVIDIA, is an advanced simulation framework for testing and refining their GPU designs. NVIDIA’s Architectural Simulator (NVArchSim or NVAS) focuses on building trustworthy simulation platforms with different levels of fidelity to achieve higher simulation speed. NVAS is a hybrid trace-driven and execution-driven multi-GPU simulator. NVAS is used to evaluate HPC and ML workloads with much higher performance as compared to most publicly available GPU simulators.

CHAPTER 3. RELATED WORK

AMD has developed their own GPU simulator based on the *gem5 simulator* [48]. The AMD *gem5 GPU model* [49], whose main target is the AMD APU devices, is a component added to the *gem5 simulator*. The new component is dedicated to the AMD GCN3 architecture. The AMD *gem5 GPU simulator* is an execution-driven single-GPU simulator.

3.1.2 Academic Research Simulators

Developing GPU research-related tools has been a focal point since the tremendous growth in using graphics processors for compute. Earlier tools include Barra [50] and GPU Ocelot [51, 52], which provide functional GPU emulation support, though do not support timing models. Over the past decade, many publicly available GPU simulators have been developed to capture not only functional, but also timing behavior, of various generations of GPUs. We present these GPU simulators based on the timing of their GPU target. Then we introduce several parallel GPU simulators.

For NVIDIA GPUs, GPGPUSim [53] is a GPU simulator that models the NVIDIA Fermi architecture and supports the CUDA 4.0 runtime API. GPGPUSim supports virtual ISA PTX, an intermediate language between the CUDA programming language and a subset of an older machine ISA `Source` and `ASSEMBLY` (SASS). GPGPUSim has been extended to support additional GPU features. MAFIA [54] extends GPGPUSim to support concurrent kernel execution and MOSAIC [55] extends MAFIA simulator to support virtual-to-physical address translation and a unified CPU-GPU address space. Accel-Sim [56] is another work extended from GPGPUSim. It is a trace-based GPU simulator with a flexible frontend enabling the support of modern SASS machine ISAs. Accel-Sim includes a trace-generation tool (built using the NVBit [57] binary instrumentation tool) to produce machine ISA instruction traces from any CUDA binary. They also make extensive modifications to GPGPUSim’s performance model to model features introduced in more modern GPUs. Accel-Sim also adds support for the NVIDIA Kepler, Pascal, Volta, and Turing architectures.

For AMD GPUs, Multi2Sim [58] is one of the early simulation tools that supported AMD GPU simulation. Multi2Sim is a simulator infrastructure capable of emulating various CPU and GPU architectures. Multi2Sim supports detailed timing simulation for CPUs operating on the X86 instruction set. In terms of GPUs, Multi2Sim initially modeled the AMD Evergreen and Southern Islands series GPUs. Notably, it has expanded its support to include the modeling of NVIDIA Kepler GPUs [59]. MGPUSim [20] is a high-performance parallel GPU simulator targeting the AMD GCN3 architecture. MGPUSim can run either emulation mode or timing simulation mode and provides native language-level support for multi-threaded programming. Moreover, MGPUSim provides a

CHAPTER 3. RELATED WORK

flexible multi-GPU simulation framework supporting both unified and discrete multi-GPU models.

For Intel GPUs, the Macsim [60] is a simulator framework capable of simulating a variety of architectures including x86, ARM64, NVIDIA PTX and Intel GEN GPU architecture. It models the detailed Intel's integrated GPU microarchitecture.

The parallel execution model of the GPU leads to a large number of units concurrently executing independently, which produces opportunities to use multiple CPU threads to simulate GPU execution. Barra [50] mainly focuses on parallel functional emulation. GPUTEjas [61] is a Java-based, trace-driven, parallel architectural simulator that can achieve high performance and scalability. Lee et al. proposed a parallel simulator framework [62, 63], which modifies GPGPUSim and only synchronizes when the processor accesses the memory system. MGPUsim also supports parallel simulation adopting a conservative parallel simulation approach which requires global synchronization after each cycle. Hughes et al. integrated GPGPUSim into a node mode in the SST [64] framework to present a parallelizable, multi-node, GPU simulator [65].

3.2 Extending GPU Architectures to Accelerate FHE

3.2.1 Extending/Modifying Current GPU Architectures

Researchers have been trying to address challenges in workload-related problems (i.e., acceleration of specific applications and concurrent executions of multiple applications) by designing GPU architecture extensions.

Marangoz et al. [66] propose an extension of the GPU memory architecture to effectively support bandwidth reservation and sharing with little overhead. They address the interference problem between the co-running applications which increases non-deterministic timing behaviors by designing a GPU memory bandwidth reservation scheme and implementing new hardware components in a GPU simulator built upon GPGPUSim.

Fu et al. [47] propose a Composable On-Package GPU (COPA-GPU) architecture to accommodate architectural requirements between the FP32 (or larger)-based HPC and FP16 (or smaller)-based DL workloads. By leveraging multi-chip module (MCM) [67] integration, they propose composable, semi-specialized designs that provide high levels of GPU design reuse across the HPC and DL domains. The paper presents the microarchitecture of the proposed memory systems, cost and benefits of the COPA-GPU. They evaluate the COPA-GPU design using a modified version of NVIDIA's NVArchSim.

CHAPTER 3. RELATED WORK

Braak et al. [68] propose R-GPU, a reconfigurable GPU architecture with communicating cores. R-GPU, as an extension to a GPU, enables communication between the cores in a Streaming Multiprocessor (SM, similar to CU) by adding configuration registers (CR) and a communication network with FIFO buffers. The paper introduces a programming model to assist the programmer in programming the R-GPU architecture and provides an estimation of the hardware costs in terms of area and power consumption. The idea of configuring cores in an SM in a network is similar to the first microarchitectural extension presented in GME [69], which builds a CU/SM-side network-on-chip that interconnects CU/SMs to support data movement between CU/SMs. They validate the performance improvements of R-GPU in GPGPUSim.

3.2.2 Accelerating FHE

Since the major challenge in deploying FHE in real-world systems is overcoming the high computational costs, many prior efforts have been invested in building efficient FHE implementations including libraries and frameworks.

For GPU implementation acceleration, Privft [70] provides a GPU implementation of the Cheon-Kim-Kim-Song (CKKS) FHE scheme that shows 1 to 2 orders of magnitude speedup. The work by Badawi et al. [71] aims to accelerate FHE using NVIDIA GPUs. 100x [72] aims at speeding up the end-to-end CKKS scheme including bootstrapping. They utilize memory-centric optimizations such as kernel fusion and reordering primary functions to optimize off-chip memory transactions.

Several prior works accelerate FHE by developing domain-specific custom accelerators. HEAX [73] is an FPGA-based accelerator that only speeds up CKKS encrypted multiplication, with the remainder offloaded to the host processor. Kim et al. [74] propose an FPGA-based implementation for HE operations with the bootstrappable parameters. It only focuses on optimizing Number Theoretic Transform (NTT), a fundamental operation in lattice-based cryptography, to support the bootstrapping process in FHE. FAB [75] is the most recent work and it proposes an FPGA-based accelerator for bootstrappable FHE for a secure and practical parameter set. FAB achieves performance comparable to the previous GPU implementation, 100x [72]. However, most of these prior proposals either do not cover all HE primitives, or only support a smaller parameter set that allow computation up to a multiplicative depth of 10, which is insufficient for bootstrapping or applications such as logistic regression training.

Although FPGAs show great potential for accelerating FHE workloads, they are limited

CHAPTER 3. RELATED WORK

to run at low operating frequencies and provide limited compute resources. Researchers have developed several ASIC-accelerators to accelerate FHE. F1 [76] proposes an ASIC implementation that makes use of small N and Q values, implementing only a single-slot bootstrapping. BTS [77] is the first ASIC proposal demonstrating the performance of a fully-packed CKKS bootstrapping. CraterLake [78] and ARK [79] further enhance the packed CKKS bootstrapping performance and demonstrate several orders of performance improvement across various workloads.

Although ASIC implementations deliver several orders of performance improvement, they are tailored for a single workload. With customized designs, they can adopt the latest memory technology. For example, ARK [79] integrates the latest HBM3 technology and CraterLake [78] incorporates extra physical layers to facilitate communication between DRAM and on-chip memory. Hence, they are able to enhance the available bandwidth for FHE workloads.

Chapter 4

NaviSim

The development and availability of architectural simulators has enable industry and research to aggressively perform design space exploration. Architectural simulation tools are vital as they provide quantitative feedback on design optimization and can aid in pre-silicon verification. A wide range of simulators have been developed to explore the design space of CPUs, GPUs, and customer accelerators. In this thesis, we attempt to characterize the process of developing a simulator, utilizing a number of design goals. To ground our discussion in reality, we will cover NaviSim, a novel GPU simulator that models the AMD RDNA and CDNA architectures. While NaviSim has been used to evaluate the performance of AMD CDNA GPUs, we will leverage here to explore the different simulator metrics developed in this thesis.

One of the key motivations for carefully considering how best to architect a new GPU simulator is the rapid rate of innovation occurring in GPU architectures. GPU vendors have introduced next-generation architectures and products at an unprecedented pace. Existing GPU simulators are typically designed to simulate a specific instruction set architecture, and are customized to capture a very specific GPU microarchitecture. This rigid design makes it difficult to adapt an existing simulator to support future architectures. This will impact the architecture community’s ability to properly assess future architectures.

Over the past five years, AMD announced two brand-new architectures — RDNA and CDNA, moving away from the GCN architecture which has been the dominant GPU architecture for AMD for over a decade. One unique goal of NaviSim is to model the two new architectures along with GCN3 and compare the differences between ISAs and microarchitectures.

4.1 Design Goals

An overall design goal of NaviSim is to achieve adaptability. We design NaviSim to anticipate multiple ISAs and microarchitectures, attempting to guard against the simulator becoming quickly outdated due to the pace of development in GPU technology. We breakdown adaptability further into two qualities: i) modularity and ii) extensibility.

Highly modular design. We separate the ISA emulation from the detailed timing simulation by adopting a stand-alone instruction emulator. As an execution-driven simulator, NaviSim recreates the execution results of GPU instructions during simulation with the help of an instruction emulator for individual ISAs. Our design only defines a high-level interface of the emulator and this allows users to easily swap the instruction emulator being used.

In addition, we break down our overall design into a set of isolated components, which serve as the basic simulation elements of the framework. Isolation here means each component should be a self-sufficient entity that maintains all the state required to perform tasks. More specifically, a component should not directly access fields of other components or call functions of other components. This is a form of design encapsulation, akin to how object-oriented programming language encapsulate data and options in a class. Moreover in our design, a simulator component cannot schedule an event for another component.

Ease of Design Extensions. When the research community uses an architectural simulator to perform their research studies, the most common step is to modify the simulator to fit the needs of their intended study. We follow the “Open-Closed Principle” [80] — one should be able to extend a simulator without modifying it. We avoid designing inter-dependent components, because modifying them tends to be non-trivial, and even more problematic than implementing them from scratch. Instead, we define clear interfaces and allow users to write new extensions to implement their own components. They can then plug the new extensions into the existing simulator without interfering with other parts.

Accuracy and Trustworthy. Accuracy is one of the three primary goals during the development of an architectural simulator [11]. While accuracy can be measured in comparison to existing silicon, trustworthiness reflects the ability of the simulator to model novel ideas not yet baked into silicon [28].

4.2 Simulator Development Methodology

To achieve the design goals we mention above, we leverage the Akita simulator framework [81] to enable the modular and extensible design of various GPU components. Akita is based on a discrete event simulation (DES) model and consists of the following key concepts:

1. The event-engine system
2. The component system
3. The message-port-connection system

Akita features an event-driven simulation engine which manages a queue of events for the entire simulation and executes events sequentially. An event indicates a state update of the simulator. Every entity of the simulated computer platform is a component [81]. The Message-Port-Connection system is used for communication between components. Two components can only communicate with each other by sending and receiving messages through a *connection*. A *connection* is an abstract structure that can model wires, buses, and intra-chip/inter-chip interconnect networks.

To fulfill the goal of designing an accurate GPU simulator, we use microbenchmarks to calibrate key parameters of target GPUs in order to evaluate the accuracy of NaviSim. We design a set of microbenchmarks (see Table 4.1) to evaluate individual GPU subsystems, such as the wavefront dispatcher, instruction pipelines, and memory hierarchy. At a high level, we repeat one operation thousands to millions of times to stress individual components of the GPU. Then we statistically analyze the latency of an operation, as we evaluate the throughput of a specific GPU component.

As an example of this process, we use the pointer-chasing benchmark to evaluate cache sizes and associated latencies. The host program first creates a region of memory (i.e., the pointer chasing region) of a given size. Next, the host program divides the region into 8-byte cells and fills each cell with an address that points to the next cell, located at a randomly assigned address within the region. There is no repetition in the addresses stored in the region so that the whole region can be traversed multiple times. The GPU will use a single thread to access the cells many times (at least several times more than the number of cells), following the addresses stored in the region. We eventually divide the kernel execution time by the number of accesses to calculate the average access latency.

In the following sections of this chapter, we will describe how we model RDNA architecture (ISA and microarchitecture) in NaviSim including the implementation details and validation process.

Table 4.1: The list of microbenchmarks used to calibrate NaviSim.

Microbenchmark	Parameter	Description
empty_kernel	Work-Group DispatchingSpeed	Executes a kernel, while varying the kernel and work-group sizes. We leave the kernel blank so that we can measure work-group dispatching latency.
empty_kernel_multi	Kernel Launching Overhead	Launches a given number of empty kernels. The kernel used is same as the one used in empty_kernel microbenchmark.
single_thread_loop	Instruction Pipeline Depth	Executes a kernel with only one thread. The kernel has a main loop, and each iteration of the loop executes a single single-precision floating point instruction.
multi_thread_loop	Instruction Pipeline Throughput	Executes the same kernel as single_thread_loop, but with a large number of threads that are large enough to fully occupy the GPU execution resources.
pointer_chasing_random	Cache Sizes and Cache Latencies	Runs the classic pointer chasing microbenchmark [82]. The pointers stored in the buffers are randomized. We only run 1 thread in this kernel.
pointer_chasing_linear	Cache Size and Cache Latencies	The same kernel as pointer_chasing_random, except that the pointers in the buffers always point to the next pointer (the last pointer points to the first one).
memory_copy	Bandwidth of Caches and DRAMs	Copies data from one buffer to another, using a given number of work-groups. When there are only 1-2 work-groups, we evaluate the L0 cache bandwidth. When there are enough work-groups to fill the whole GPU, this microbenchmark can also test the DRAM bandwidth.

We study the architectural differences between GCN3 and RDNA and build the RDNA model from scratch while reusing some components in MGPUSim [20]. Showcasing this process as a concrete example of the general design methodology we proposed will benefit the simulator community, not only from a simulation tool with a range of ease-of-use features but also the design philosophy of achieving a highly adaptable and accurate GPU simulator. We focus on our flexible emulator design which can be plugged into timing simulation modular, making it possible to mix and match instruction sets with different microarchitecture implementations. This flexibility enables the evaluation the performance impact of microarchitectural design features in the RDNA architecture.

4.3 RDNA Architecture

The AMD GCN architecture [2] has been the reference specification for AMD GPU design over the past decade. However, in recent years, the AMD GCN architecture faced critical scalability challenges, as it struggled to fully exploit the ever-increasing number of transistors provided by today’s photolithography technology [83]. In response, AMD developed a new family of RDNA architectures [3], introducing a major architectural overhaul as compared to the GCN architecture. We focus on these architectural differences between GCN and RDNA architectures, including the programming model, the compute unit (CU), and the memory hierarchy. The AMD RDNA Whitepaper [3] provides more detailed specifications of the design.

One of the most noteworthy changes is that the RDNA architecture reduces the size of the wavefront from 64 work-items to 32 work-items. By cutting the wavefront size in half, the CUs are expected to better cope with a higher degree of thread divergence in modern workloads. Additionally, as the wavefront size is smaller, fewer memory transactions are expected to be generated by a single load/store instruction (although the total number of transactions generated by the kernel is likely to remain unchanged), potentially reducing memory access latencies. These benefits should help to improve ALU utilization. We will further evaluate the effects of narrower wavefronts in the later part of this chapter.

A second major change in RDNA GPUs is the introduction of Dual Compute Units (DCUs, see Figure 4.1(d)), replacing the GCN CUs. A DCU contains 4 schedulers. Increasing the number of schedulers from 1 to 4 significantly increases the instruction issue rate. Rather than dispatching instructions across 4 SIMD units, as in the integrated GCN CU design, each RDNA scheduler in a DCU dispatches instructions to 1 SIMD unit. One SIMD unit in a DCU has 32 single-precision ALUs, doubling the number of ALUs in a CU. Working together with the narrower 32 work-item

CHAPTER 4. NAVISIM

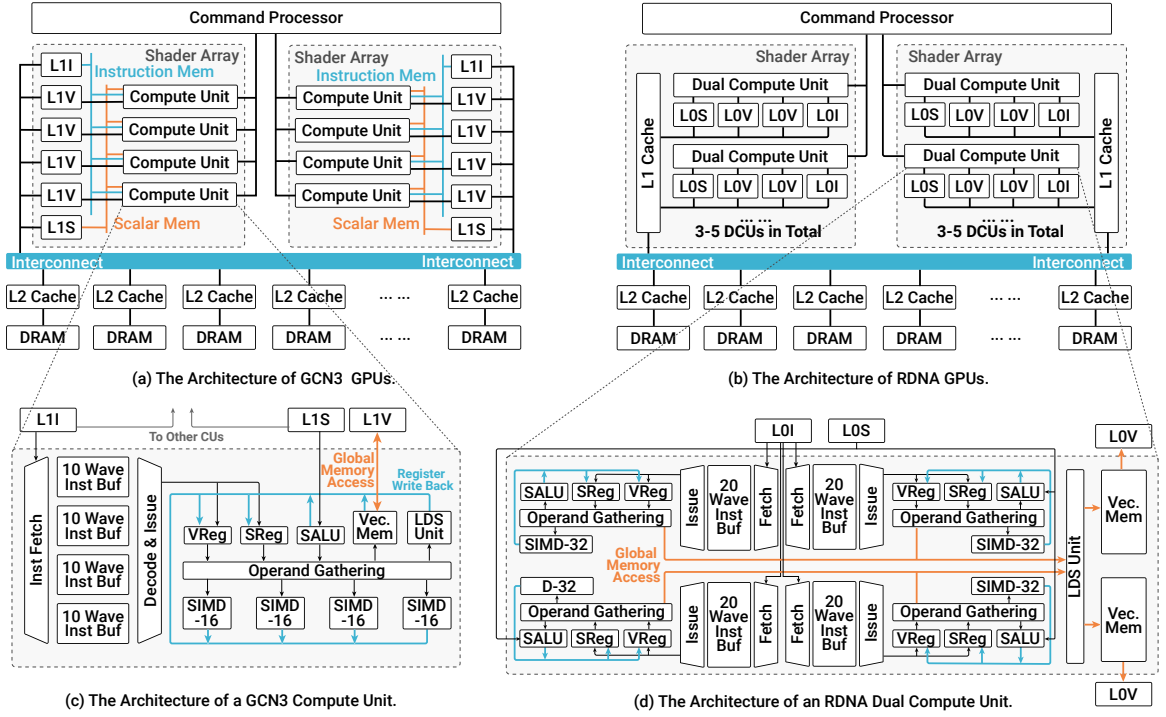


Figure 4.1: A Comparison between the GCN Architecture [2] and the RDNA Architecture [3].

wavefronts, each SIMD unit can finish executing one instruction in a single cycle, as compared to 4 cycles in a CU.

Third, the RDNA architecture redefines the cache hierarchy from a 2-layer structure to a 3-layer structure. The caches that are directly connected to the DCUs are renamed as L0 caches (versus L1 caches). Each read-only L0 instruction cache and read-only L0 scalar cache are no longer shared by multiple CUs, but dedicated to a DCU. Each DCU connects with 2 separate write-through L0 vector caches; a group of two schedulers and two SIMD units can use one L0 vector cache. Since now we have two L0 caches connected to one DCU, updating the data in one cache may render the data in the other L0 cache stale. This may cause coherence issues within a DCU and requires explicit cache invalidation instructions (as provided in the RDNA ISA, but not in the GCN ISA).

Additionally, an intermediate level of caching (i.e., the new write-evict L1 cache) is inserted. The L1 cache serves a group of DCUs (typically 4-5) in a Shader Array, and sits between the L0 and L2 caches. The L1 caches can reduce the number of requests arriving at the L2 caches (in the case of L1 hits) and reduce the amount of data that is transmitted across the chip (from L2 to L0), thereby increasing performance and lowering the power consumption caused by cross-chip transmissions.

Finally, the cache line size of the L0 vector caches, L1 caches, and the L2 caches is doubled from 64B to 128B, so that a cache line can deliver unique single-precision numbers for all 32 work-items in a wavefront ($4B \times 32 = 128B$).

4.4 Implementation of NaviSim

Next, we focus on the implementation details including ISA emulation, wavefront dispatching, DCU modeling, and memory hierarchy. We also discuss the user interface, the supported features, and simulation output formats.

4.4.1 ISA Emulation

NaviSim runs in either emulation mode or timing simulation mode. The core component in instruction emulation mode is an emulator which recreates execution results, without evaluating detailed timing information for the instruction pipelines, caches, and DRAM controllers. Timing simulation mode utilizes an external emulator to execute instructions while simulating the timing behavior of other components. We have developed RDNA and CDNA instruction emulators for NaviSim. Since these instruction emulators share the same interface with the GCN3 instruction emulator from MGPUSim, this allows users to swap the instruction emulator being used. The virtual driver (a set of APIs that connect the Go-coded host programs and the simulated GPUs) of NaviSim allows users to configure which ISA to emulate and load the corresponding GCN3/RDNA/CDNA kernel binaries.

Currently, NaviSim supports kernel written in either the OpenCL [10] or HIP [8, 9] programming languages. OpenCL kernels can be compiled by the AMD official `clang-ocl` compiler, which is a standard part of the AMD Radeon Open Compute (ROCm) platform [84]. HIP kernels can be compiled with the `hipcc` compiler, which also ships with the AMD ROCm platform. By using the `--genco` argument, `hipcc` ignores the host program and only generates kernel binaries. NaviSim supports loading kernel binaries compiled by either compiler and emulates the execution of the GPU kernels using actual input data sets.

4.4.2 Wavefront Dispatching

We model the wavefront dispatching process in the Command Processor. The modeled Command Processor maintains resource masks that keep track of which resources are occupied in

each CU/DCU, including the wavefront slot (wavefront-level resources, such as instruction buffers and the program counter register), scalar registers, vector registers, and LDS memory. The resource masks ensure that no hardware resources are oversubscribed. Since we mask the resources either at the register level (for vector and scalar registers) or at the byte level (for the LDS), we can also model register/LDS fragmentation issues in the CUs/DCUs [85].

To support the modeling of concurrent kernel execution [86], basically allowing multiple kernel executions simultaneously on one GPU, the modeled Command Processor has multiple wavefront dispatchers. Each wavefront dispatcher will manage the progress of the currently executing kernel's execution and dispatch a new wavefront when resources free up in the CUs/DCUs. By default, each Command Processor provides 8 dispatchers (the number is configurable), and hence, we allow up to 8 kernels to execute concurrently. The wavefront dispatchers compete for the resources and have equal opportunities to dispatch wavefronts to the CUs/DCUs, ensuring the fairness of the concurrently executing kernels.

4.4.3 DCU modeling

We develop a detailed architectural model of the DCU, which governs how instructions are executed in the simulator. Since the accurate modeling of the DCU is essential for simulation accuracy, NaviSim carefully models the pipeline (see Figure 4.2) with a multi-stage, multi-issue structure.

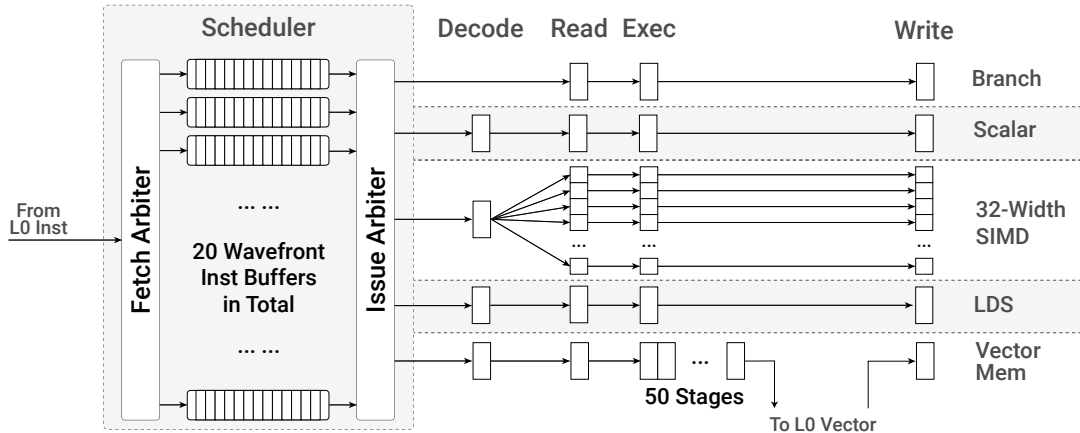


Figure 4.2: The instruction pipeline model in a NaviSim DCU. Each DCU has four copies of the structure shown.

The pipeline starts with the instruction fetch arbiter, which attempts to fetch instructions

CHAPTER 4. NAVISIM

for a wavefront that has vacant space in instruction buffers. When multiple wavefronts have space available, the wavefront that received instructions furthest in the past is selected. The issue arbiter monitors the instruction buffers and selects wavefronts that have instructions ready that they can be issued. The arbiter can issue at most 5 instructions per cycle, one instruction to each instruction pipeline.

The Branch, Scalar and LDS pipelines use a fairly regular 6-stage pipeline that includes fetch, issue, decode, read, execute, and write stages. Note that, for a single instruction, the decode stage is logically executed after the issue stage. This is because the issue arbiter can easily determine the type of each instruction by checking a few encoded bits in the instruction. The branch instruction has no decode stage, as the instructions are very simple. This is in line with publicly available documentation provided by AMD for the DCU architecture. The SIMD unit, which provides most of the computing power of the DCU, also uses a 6-stage pipeline design, but is capable of reading, executing, and writing 32 instructions in parallel. The vector memory has the most complex pipeline structure, adopting a 50-stage pipeline. This may sound unusual, but the model matches our microbenchmarking results. Since there is no public information on the exact function of each of the pipeline stage, we do not model the behavior of each stage, but only associate a latency value with each instruction.

4.4.4 Memory Hierarchy

We model the 3-level cache hierarchy in RDNA GPUs. L0 scalar and L0 instruction caches are read-only, L0 vector caches use a write-through policy, L1 caches use a write-evict policy, and L2 caches use a write-back policy. We also connect DRAMSim3 [87] to model GDDR5, GDDR5X, GDDR6, GDDR6X, HBM, and HBM2 DRAM controllers. We allow users to configure any number of caches in the hierarchy and support any combination of the available cache policies. Additionally, all of the parameters of a cache (e.g., set count, way associativity, cache line size, directory latency, storage access latency) can be fully configured by users.

4.4.5 User Interface

NaviSim adopts a similar user interface as used in MGPUSim. To run benchmarks, the user can compile either HIP or OpenCL kernels using the official AMD compiler. A host program written in Go is required to invoke a set of APIs to allocate/copy memory and launch kernels. The host APIs are compatible with MGPUSim and are similar to common GPU programming frameworks (e.g.,

Table 4.2: The configurations of validation platforms.

Parameter	Platform1	Platform2
GPU	Radeon RX 5500 XT	Radeon Pro W6800
GPU Core Freq	1845 MHz	2320 MHz
DCU Count	11	30
GPU Memory	GDDR6	GDDR6
Mem Bandwidth	224.0GB/s	512.0GB/s
CPU	AMD Ryzen Threadripper 2950X	AMD EPYC 7302P
OS	Linux Ubuntu 18.04	Linux Ubuntu 20.04
GPU Driver	AMD ROCm 5.0	AMD ROCm 5.1

CUDA, OpenCL, HIP). Users can specify either emulation mode (value emulation only) or timing mode (detailed timing simulation that involves instruction pipeline, caches, and memory controllers) as a command line option. Meanwhile, NaviSim uses a few configuration files that are written in Go to define the hardware configuration. Users can easily configure the hardware under simulation by modifying parameters and component connections in the configuration code. We use code for configuration because users can easily debug the configuration logic with debuggers.

4.5 Evaluation

Next, we describe the methodology of validating NaviSim against real GPU hardware platforms. We calibrate and validate the accuracy of the simulator using a set of microbenchmarks and full benchmarks. To demonstrate the model’s suitability to conduct a range of studies and analyses, we conduct additional experiments to evaluate the RDNA architecture design.

4.5.1 Simulator Validation

Any computer architecture simulator requires a rigorous validation process before it can serve as a baseline for future research. We present our validation work for NaviSim, comparing simulation results against GPU hardware execution.

Validation Platforms: We use an RX 5500 XT GPU and Radeon Pro W6800 (see Table 4.2) to validate our NaviSim GPU Model. RX 5500 XT GPU is an entry-level graphics card

CHAPTER 4. NAVISIM

which was announced along with the new RDNA architecture. Radeon Pro W6800 is a high-end workstation GPU, built on the RDNA2 architecture. The main differences between the RDNA and the RDNA2 lie in the memory/cache system (Radeon Pro W6800 has a 4-level cache hierarchy). Our platforms run the ROCm 5.0/5.1 software stacks on Linux Ubuntu 18.04/20.04 servers.

Table 4.3: Simulator configuration

Param	P1	P2	P3*
Base Model	RX5500XT	R9 Nano	-
# CU/DCU	11(DCU)	64(CU)	32(DCU)
Core Freq	1845MHz	1000MHz	1000MHz
TFLOPS	5.20	8.19	8.19
L0V \$ [@]	16KB	16KB	16KB
L0V \$ Assoc.	4-way	4-way	4-way
L0 Inst \$ [@]	32KB	32KB	32KB
L0 Scalar \$ [@]	16KB	16KB	16KB
L0I/L0S \$ Org.	Per DCU	4-CU shared	Per DCU
L1 \$	128KB	-	128KB
L1 \$ Assoc.	16-way	-	16-way
L2 \$	1MB	2MB	2MB
L2 \$ Assoc.	16-way	16-way	16-way
DRAM Tech	GDDR6	HBM	HBM
DRAM Size	4GB	4GB	4GB
Mem Freq	1750MHz	500MHz	500MHz
Mem Bus	128 bit	4096 bit	4096 bit
Mem BW	224 GB/s	512 GB/s	512 GB/s

* P3 is not an off-the-shelf GPU, but it is chosen specifically to analyze the performance impact of individual RDNA features.

[@] We will refer to the caches that are directly connected to a CU/DCU as the L0 cache. In P2, L0 caches connect to L2 caches directly.

Simulator Configuration: We select the baseline GPU configuration using publicly available information and calibrate our results using microbenchmarks. The default configuration of NaviSim for the RX 5500 XT GPUs is recorded as P1 in Table 4.3.

CHAPTER 4. NAVISIM

After validating the NaviSim GPU model, we use the model to conduct a series of use-case experiments to evaluate the performance impact of microarchitectural design features in the RDNA architecture. We compare our simulation results with the default configuration of MGPUSim for the R9 Nano GPUs (denoted as P2 in Table 4.3). Since the AMD RX 5500 XT and the R9 Nano GPUs belong to two very different markets, comparing them directly would not provide us with a lot of new insights. Therefore, we have configured a GPU as P3 in Table 4.3. We chose these GPU configurations so that they would have a comparable theoretical computing throughput (represented by TFLOPS) and a comparable memory hierarchy. In the experiments, rather than directly comparing P2 and P3, we gradually add features from P2 to build P3, so that we can evaluate the effect of each feature.

Full Benchmarks: We also utilize a list of full GPU benchmarks (see Table 4.4) selected from a wide range of benchmark suites including AMDAPPSDK [88], SHOC [89], HeteroMark [90], PolyBench [1], and DNNMark [91]. We use these benchmarks as a set of workloads that cover a wide range of applications to exercise the simulator with different arithmetic intensities, memory access patterns, and communication patterns. We start with the original OpenCL kernel implementations from the benchmark suite and compile the kernels with the original AMD ROCm compiler (applying default compiler optimizations). We also write host programs in Go, allowing the simulators to call the kernels. We ensure our host programs are equivalent to the original host program from the benchmark suite. For validation experiments, we vary the problem sizes of the benchmarks to make sure that NaviSim can recreate scaling trends. For performance evaluation, we use large problem sizes that are sufficient to stress the whole GPU.

Running Benchmarks: We run OpenCL implementations of the benchmarks, while varying the input size. We use kernel execution time as the performance metric and report the average execution time over 10 runs. The times obtained on the real GPU are recorded using OpenCL events [10].

We use kernel execution time as the primary metric to evaluate the accuracy of NaviSim, for two reasons. First, execution time is the most commonly reported metric when considering architectural tradeoffs. Second, the kernel execution time is a high-level metric that summarizes the impact of all features being simulated; we can only accurately capture execution time if the fidelity of all the components being modeled in the simulator is high.

Validation Steps: First, we verify the correctness (in terms of application outputs) of NaviSim in both emulation mode and timing simulation modes. To this aim, we compare every simulator-generated application output with its corresponding actual hardware execution output. We

Table 4.4: Full Benchmarks

Abbr.	Suite	Workload
ATAX	PolyBench	Matrix Transpose and Vector Multiplication
BICG	PolyBench	BiCGStab Linear Solver [92]
BS	AMDAPPSDK	Bitonic Sort
FIR	HeteroMark	Finite Impulse Response Filter
FLW	AMDAPPSDK	Floyd-Warshall Algorithm
FWT	AMDAPPSDK	Fast Walsh Transform
KM	HeteroMark	KMeans Clustering
MT	AMDAPPSDK	Matrix Transpose
ReLU	DNNMark	Rectified Linear Unit
SPMV	SHOC	Sparse Matrix-Vector Multiplication

observe that the results match exactly for all applications. The matching results of NaviSim when running in the emulation mode suggest that NaviSim faithfully emulates the RDNA instructions.

Second, to validate the timing model, we use both microbenchmarks and full application workloads to validate the accuracy. We configure our simulation using the publicly available parameters of the AMD RX 5500 XT GPU (see Table 4.2). We then use microbenchmarks (see Table 4.1) to help us reverse-engineer a wide range of undocumented parameters, such as the work-group dispatch overhead, instruction pipeline depth, cache latency at each level, and DRAM bandwidth and latency. A good example of our strategy is that we use our pointer chasing microbenchmark to figure out the size and latency of each level of the cache hierarchy. As we can see from the representative results of running the pointer chasing microbenchmark (see Figure 4.3), NaviSim is able to calibrate the parameters with extremely high accuracy. The calibration results of other microbenchmarks follow similar trends as the pointer chasing results.

With the fully calibrated parameters, we evaluate the accuracy of the simulator by validating it against AMD RX 5500 XT hardware using full benchmarks (see Table 4.4). Figure 4.4 shows the simulator accuracy (left y-axis), as well as the relative error, at each data point by the bar plot (right y-axis) for a range of problem sizes. The problem size ranges are specific to each benchmark. We observe that for some benchmarks the relative error increases as the problem size increases (e.g., ReLU), and for some other benchmarks the relative error decreases as the problem size increases (e.g., BS, SPMV). For each benchmark, in parenthesis, we also report the average error. As we can

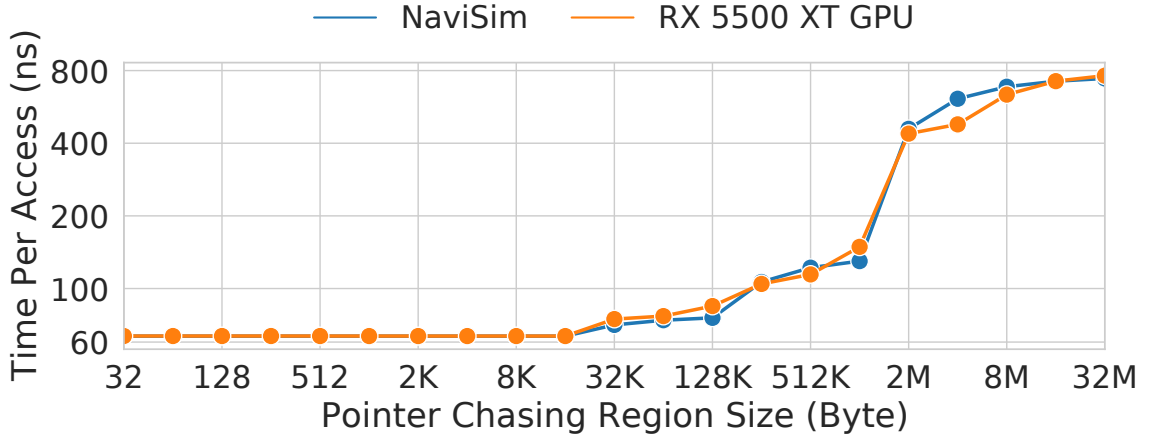


Figure 4.3: Comparison of the Pointer Chasing microbenchmark between NaviSim and the RX 5500 XT hardware.

see, the error in terms of modeled execution time, averaged across all benchmarks, is just 9.75%. Additionally, we not only model the execution with high fidelity, we clearly capture the patterns and the nuances of the GPU architecture. For example, in the FIR and ReLU benchmarks, we successfully capture the transition in the workload when the execution time starts to increase. This suggests that we properly model any bottlenecks due to resource contention (i.e., number of workgroups available) on the GPU.

As another example, we observe step-wise behavior in some benchmarks (e.g., BS before 32K, FWT after 16K and 32K, KM between 2K and 3K). As the observed step-wise behavior is caused by complex interactions between the instruction scheduler, cache hierarchy, and memory transaction handling, being able to model these steps demonstrates that NaviSim can model the subtle features in the RDNA architecture. We have also validated NaviSim against a second GPU model, the AMD Radeon Pro W6800 GPU (see Table 4.2), which is an RDNA2-based GPU. The average difference between the simulated and hardware measured execution times is 10.08%. Our validation results, which are shown in Figure 4.5, demonstrate that NaviSim can capture the microarchitectures in RDNA across different devices with high fidelity.

We are also aware of discrepancies in a few benchmarks, such as FW and KM. In general, these benchmarks are either short-running benchmarks (FW) or workloads that involve a large number of kernel launches (KM). The discrepancies suggest that NaviSim has difficulties in modeling GPU behavior during the kernel launch phase. In general, we believe this is not a big problem since the simulators are likely to be used to model large problem sizes, and the differences observed in the

kernel launch overhead should not impact the overall accuracy by much. We leave more detailed modeling of the kernel launch behavior as future work.

Memory Footprint and Simulation Speed: We also analyze the memory footprint and performance of NaviSim simulation. In the FIR benchmark (with a 4M problem size, as shown in Figure 4.4), we use 823MB of memory, which fits in the memory of most modern computers. In terms of simulation performance, on an Apple M1 Mac Mini, we achieve 43.5 KIPS and 89.5 KIPS in serial and parallel modes (NaviSim is multi-threaded), respectively. This performance is much faster than MGPUSim, which reported 27 KIPS parallel execution performance in their original paper. Thus, the memory consumption and performance of NaviSim are quite competitive.

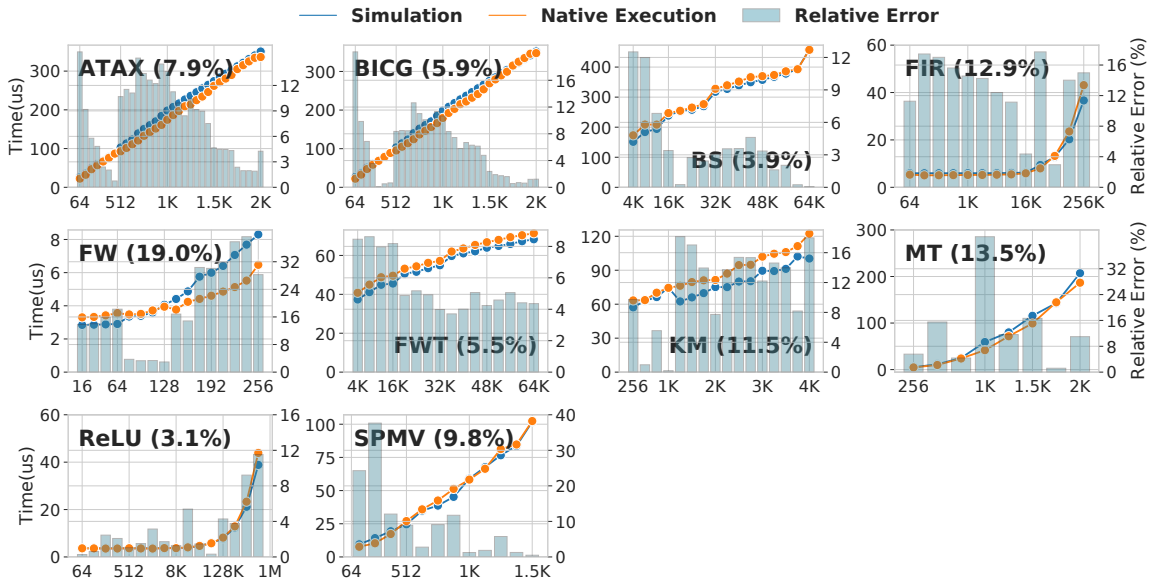


Figure 4.4: Simulator validation against the AMD RX 5500 XT GPU. The x-axis plots the problem size and the two y-axes plot the kernel execution time and relative error. The numbers in the parentheses represent the average difference between NaviSim simulation and the hardware execution for each application.

4.6 Case Studies: Understanding the RDNA Architecture Features

With a carefully validated simulator model, we next use NaviSim to perform a set of experiments to analyze the impact of RDNA features on application performance. In particular, we attempt to answer the following questions:

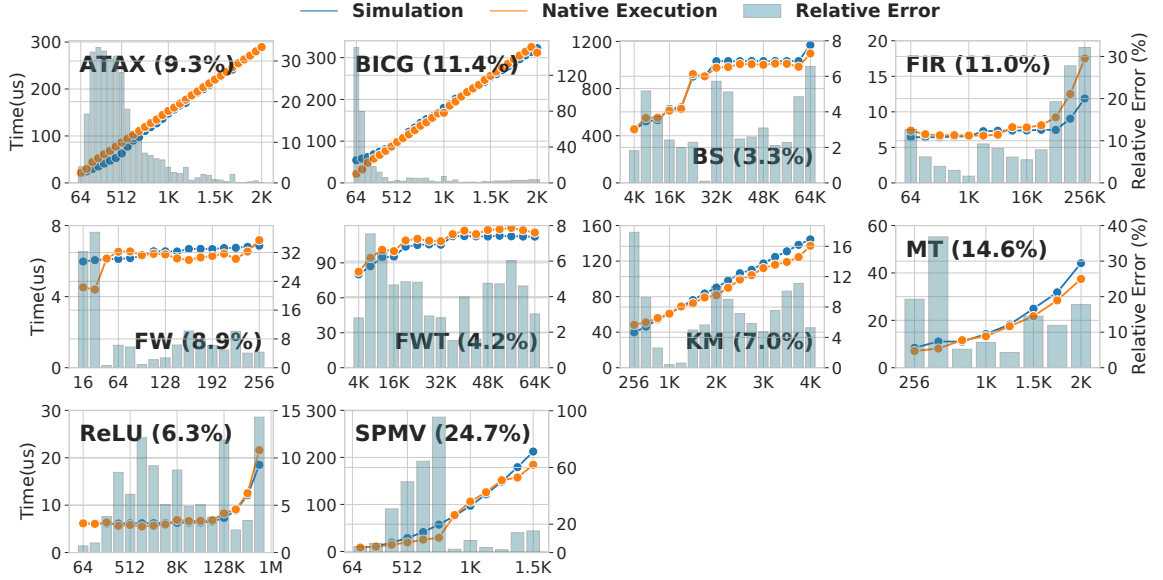


Figure 4.5: Simulator validation against the AMD Radeon Pro W6800 GPU. The x-axis plots the problem size and the two y-axes plot the kernel execution time and relative error. The numbers in the parentheses represent the average difference between NaviSim simulation and the hardware execution for each application.

1. How does changes in the ISA impact the overall performance and how does the DCU architecture impact performance? How is the instruction execution pipeline impacted by executing a different ISA?
2. What is the effect of the newly added L1 cache?
3. What percent of the overall performance increase can be attributed to changes in frequency (increased from 1 GHz to 1.845 GHz)?

While we study the impact of the features on the overall performance, we also use this study to demonstrate the utility of NaviSim. We showcase the flexibility and the configurability of NaviSim.

4.6.1 Impact of changes in the ISA and the introduction of the DCU

In the first set of experiments, we focus on question (1). We use the P2 configuration (see Table 4.3, denoted as CU+GFX803 in Figure 4.6) as the baseline. We have configurations where we either change the ISA to GFX1010 (CU+GFX1010) or change the core to DCU (DCU+GFX803, where the number of DCUs is half of the number in the CU, thus providing a fair comparison of

similar computing capabilities). We also provide a configuration that changes both the core and the ISA (DCU+GFX1010).

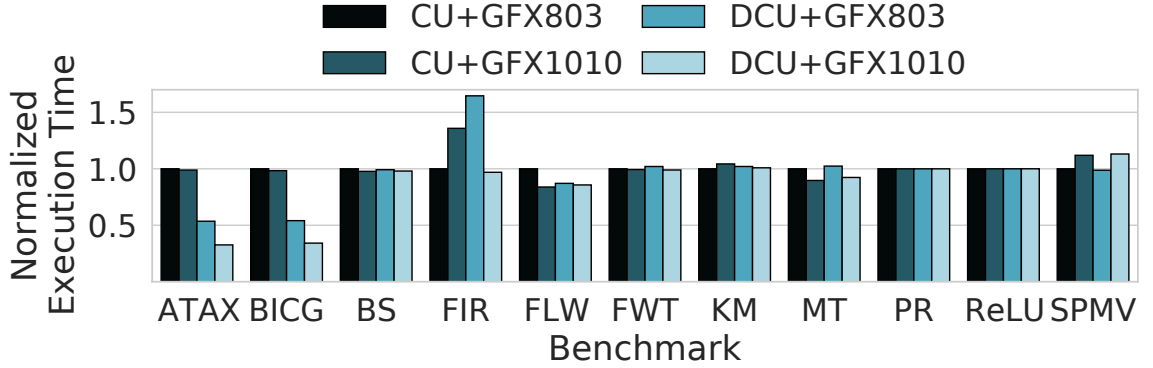


Figure 4.6: The effect of changing from CUs to DCUs and the ISA from GFX803 (GCN3) to GFX1010 (RDNA) on execution time.

At a high level, our results (see Figure 4.6) suggest that many benchmarks (e.g., BS, PR, ReLU) achieve the same performance for all four cases. This is understandable, as the memory system and the overall computing capabilities remain the same. However, we notice major performance differences in ATAX, BICG, FIR, and FLW benchmarks. These changes are caused by the differences in the ISA and the CU/DCU organization.

ATAX and BICG are good examples of how the CU/DCU organizer can impact the performance of workloads that are sensitive to memory bandwidth and latency. ATAX and BICG are workloads with limited parallelism and strong inter-work-item dependencies. Therefore, ATAX and BICG have large work-group sizes and a small number of work-groups in each kernel. In the CU configurations, the number of blocks cannot utilize all the CUs in the GPU, and hence, cannot fully utilize the bandwidth between the L0 caches and the CUs. On the contrary, the number of blocks in the ATAX and BICG can fully utilize the DCUs on the third and fourth configurations, since the count of DCUs is halved. As each DCU is connected with two vector L0 caches, the bandwidth between the core and the L0 caches is effectively doubled, resulting in a *2times* increase in the performance.

The most critical change in the ISA is the difference in wavefront size. Kernels compiled to the GFX1010 ISA always use a wavefront size of 32, which is not a perfect match for the CU architecture. When running wavefronts on CUs, because the CU scheduler can only issue 1 instruction to a SIMD unit every 4 cycles, the CU needs to spend 4 cycles to execute 1 instruction. We observe underutilization of the ALUs in the ATAX, BICG, FIR, and FLW benchmarks when

we run the GFX1010 ISA with the CU microarchitecture. Other benchmarks do not observe this issue because they are bound by memory bandwidth and are not sensitive to ALU utilization. In the FIR benchmark, we also see that running the GFX803 benchmark on the DCU microarchitecture causes a significant slowdown. The GFX803 ISA uses a wavefront size of 64 and it takes two cycles for the SIMD unit to execute, while the scheduler in the DCU schedule issue 1 instruction every cycle. Thus, the SIMD unit is not able to finish execution of the instructions in time to catch up with the instruction dispatch, leading to a significant number of pipeline stalls and even more slowdown. The problem does not exist when the DCU microarchitecture and GFX1010 ISA are used together, suggesting that the ISA and the microarchitecture are co-designed to achieve the best performance.

4.6.2 Impact of the L1 cache

Next, we focus on question (2) and try to understand the effect of the new L1 cache. We use the P3 configuration (see Table 4.2, denoted as P3 w/ L1 in Figure 4.7) as the baseline, and compare the performance with a configuration without an L1 cache (P3 w/o L1). The results are shown in Figure 4.7.

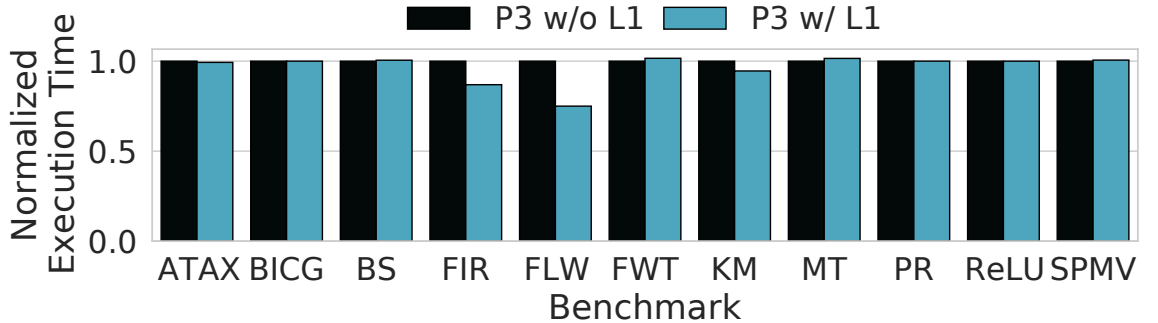


Figure 4.7: The effect of adding the new write-evict L1 cache on benchmark execution time.

Similar to the earlier experiments, as we only modify a small part of the configuration, most of the benchmarks that are either bounded by the available compute resources or by DRAM bandwidth do not observe a performance difference. However, we did observe performance improvements in the FIR and FLW benchmarks.

To understand why FIR and FLW can benefit from the added L1 cache, we plot the cache hit rate for each level of cache in Figure 4.8. In general, we see a high degree of diversity across the benchmarks, as these workloads have dramatically different levels of locality. For the BS, FIR, FLW

and FWT benchmarks, we observe a relatively high L1 cache hit rate (around 40%). However, the effect of the high L1 cache hit rate on the L2 caches differs across workloads. For BS and FWT, the L2 cache hit rate remains unchanged, while the L2 cache hit rate drops significantly for FIR and FLW (in a good way). These results suggest that the improved L1 cache hit rates lead to a reduction in the number of L2 cache transactions for the FIR and the FLW benchmarks. Because the FIR benchmark already has a high L0 hit rate, the speedup offered by the L1 cache is not as high as the benefits observed in the FLW benchmark.

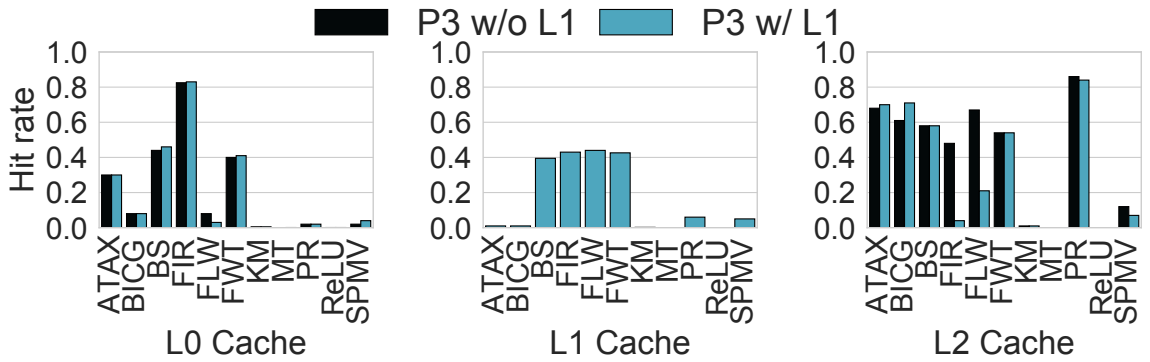


Figure 4.8: The cache hit rate of each level of cache, before and after the L1 cache is added.

4.6.3 Impact of increased frequency

One major change in RDNA GPUs is the increased frequency. While the increase is not an architectural feature, a higher frequency can impact other architectural changes, so we are eager to understand how much the increased frequency can improve performance.

Here, we use P3 as a baseline and increase the GPU core frequency to 1845MHz, which is the base frequency for the RX 5500 XT GPU. Note that as we increase the core frequency, we also increase the clock speed that controls the L0, L1, and L2 caches to 1845MHz. In AMD GPUs, the cores and the caches work in the same frequency domain. We do not change the DRAM configuration, so the DRAM latency and bandwidth remain unchanged.

Overall, we see the performance of all the benchmarks improve, with the rate of change ranging from $1.25\times$ to $1.84\times$ (see Figure 4.9). This suggests that the increase in frequency leads to a marked improvement in GPU performance. This is particularly evident in the PR and ReLU benchmarks, which experience a speedup which is closely correlated with the increase in the core frequency. This suggests that the kernel execution time of these two benchmarks is linearly

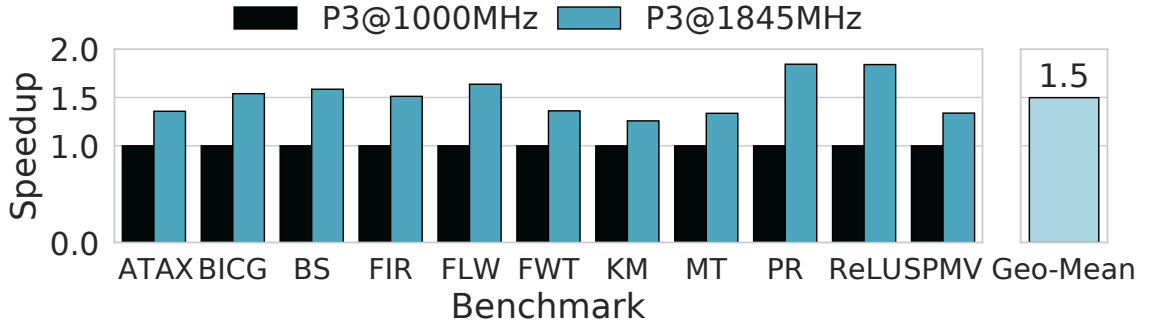


Figure 4.9: The effect of the increased core frequency on the overall performance. As we increase the core frequency by $1.845\times$, the performance improved by $\approx 1.5\times$.

dependent on the GPU frequency. More specifically, after deeper investigation with our visual simulation debugger (Daisen [93]), we find that the two workloads are dependent on the work-group dispatch rate, and have a rather short work-group execution time. The dispatcher cannot catch up with the work-group retirement rate. Because the CU can retire the work-groups faster than the rate that at which new work-groups arrive to start execution, the number of work-groups concurrently executing in each CU is limited. This also limits the number of concurrent in-flight memory transactions. Increasing the core frequency accelerates the dispatching speed and increases the occupancy of the CUs, allowing the CUs to better utilize the memory bandwidth. This effect is also observed in the case study discussed in the Daisen paper [93]. Meanwhile, we also notice that the average speedup (geometrical mean) is only $1.5\times$, suggesting that the increased core frequency still needs to be accompanied by improved DRAM performance.

Chapter 5

Design Space Exploration: Accelerating FHE with Microarchitectural Extensions (GME)

With a fully validated simulator model, we next use NaviSim to characterize application behavior and explore novel design ideas using NaviSim. We will target the performance issue of the FHE workload on GPUs. We will demonstrate the flexibility of NaviSim, attempting to accelerate a Fully Homomorphic Encryption (FHE) workload.

Our approach to accelerating the workloads starts by profiling the application on our simulator. Once we understand where performance bottlenecks occur, we focus on enhancing the microarchitecture of GPUs and we will explore three different microarchitecture extensions, as well as a compile-time optimization, to accelerate FHE execution on an AMD CDNA GPU.

To evaluate our design, we extend the implementation of NaviSim with a block-level directed acyclic compute graph simulator called *BlockSim* [69]. In addition, we conduct ablation studies on our micro-architectural feature implementations, enabling us to isolate each micro-architectural component and evaluate its distinct influence on the entire FHE workload.

5.1 Challenges of Accelerating CKKS FHE

The CKKS scheme, similar to other homomorphic encryption schemes, introduces computational overhead due to its unique properties. In order to perform computations on encrypted data,

large values of N and Q are used for encoding/decoding and encryption/decryption, which produce heavy workload demands. It requires a significant amount of computational resources to evaluate modular arithmetic expressions and also places high demands on memory bandwidth utilization. We first conduct a workload characterization of the GPU kernels that implement multiple FHE primitives. As an example of this process, we use the number-theoretic transform (NTT) kernels, which are essential building blocks of polynomial multiplication, to reveal the intrinsic bottlenecks of FHE workloads. Figure 5.1 provides profiling results for the NTT kernels for integer sizes of 32 bits and 64 bits. The high memory intensity reported indicates that a significant portion of the execution time is devoted to accessing memory, resulting in low ALU utilization. We observe that, not only NTT, but also all of the FHE operations exhibit low arithmetic intensity and require very large working sets, which indicates frequent main memory accesses.

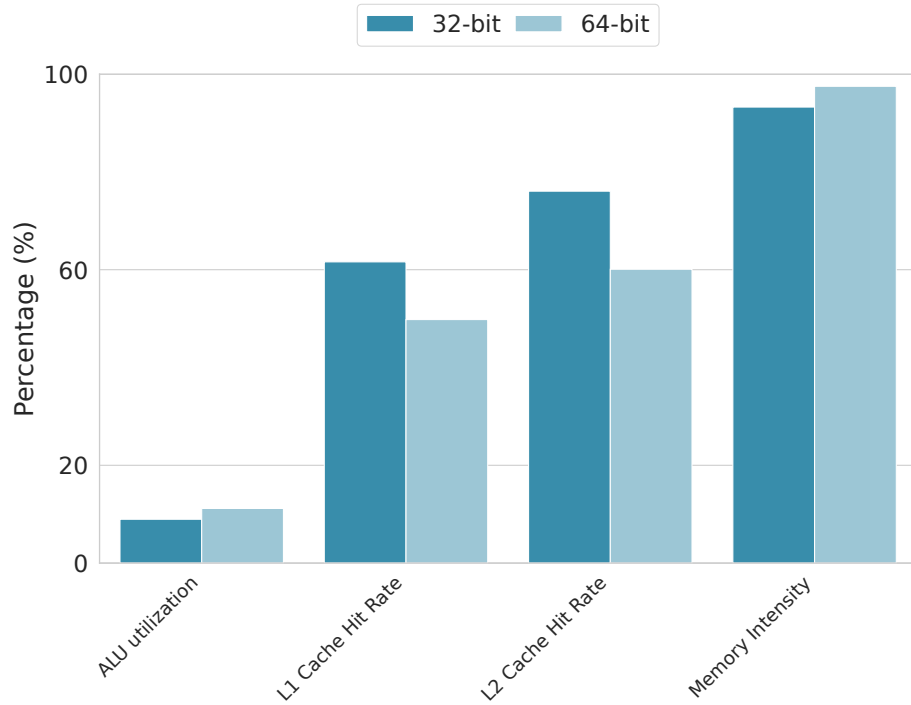


Figure 5.1: NTT kernels profile of ALU utilization, cache hit rates and memory intensity

Memory Bottlenecks: The basic datatype in CKKS is ciphertext, denoted as a vector of N elements, where each element is $\log Q$ bits in size. Thus, we can calculate the size of each ciphertext. We choose $N = 2^{16}$ and $\log Q = 1728$, meaning that our ciphertext size will be $2 * N * \log Q = 28.3$ MB, where each polynomial in the ciphertext is ~ 14 MB. After applying RNS

decomposition [45] on these polynomials using a word length of 54 bits, we get 32 limbs in each polynomial, where each limb is ~ 0.44 MB large.

For AMD’s MI100 GPUs, the size of the last level cache and local data share space are 8 MB and 7.5 MB, respectively. Thus we cannot accommodate even a single ciphertext in the on-chip memory. At most, we can fit ~ 18 limbs of a ciphertext polynomial, and as a result, we will have to perform frequent accesses to the main memory to operate on a single ciphertext.

Compute Bottlenecks: FHE operations involve complex mathematical computations, including modular arithmetic and polynomial multiplications. Modular arithmetic operations occur at the lowest level, which means every scalar addition and multiplication is followed by a modular reduction. Since division operation is involved and is not directly implemented on hardware, some algorithms have to be used to perform a modular reduction operation, so it becomes one of the compute bottlenecks. In addition, we need to use RNS decomposition to map the wide modular operations to the widths of the underlying computing platform (i.e., 32, 64, and 128-bit) and then perform operations on the divided smaller elements(RNS limbs). The limb bit width impact the number of modular addition and multiplication operations required. Currently, commercially available GPU architectures do not implement 64-bit and 128-bit wide modular arithmetic operations directly but can emulate them via multiple arithmetic instructions, which significantly increases the amount of computing required for these operations.

To address the previously identified bottlenecks, we focus on exploring new micro-architectural extensions with Navisim that can leverage the GPU’s on-chip memory space to minimize the accesses to the main memory and provide native modular arithmetic units to accelerate modular arithmetic operations.

5.2 Exploring Microarchitectural Extensions with NaviSim

In this work, we propose to re-architect the current GPU microarchitecture and also introduce novel microarchitectural extensions that enable improved utilization of GPU resources so as to maximize the performance of the FHE workloads running on the GPU. We propose a robust set of micro-architectural features targeting AMD’s CDNA architecture, unlocking the full potential of the GPU to accelerate FHE workloads.

We pinpoint critical bottlenecks encountered during FHE workload execution and address them progressively using three micro-architectural feature extensions. Our on-chip CU-side hierarchical network (**cNoc**) and the Locality Aware Block Scheduler (**LABS**) contribute to minimizing

the DRAM bandwidth bottleneck. Simultaneously, our implementation of native modular reduction (**MOD**) and wider multiply-accumulate units (**WMAC**) features improve the math pipeline throughput, ensuring a streamlined data flow with evenly distributed resource utilization.

5.2.1 cNoC: CU-side interconnect

Modern GPUs have a network-on-chip that interconnects the compute units(CUs) together with the memory partitions or memory banks (see Figure 2.1). On-chip communication occurs between the CUs and the memory banks, not necessarily between the CUs.

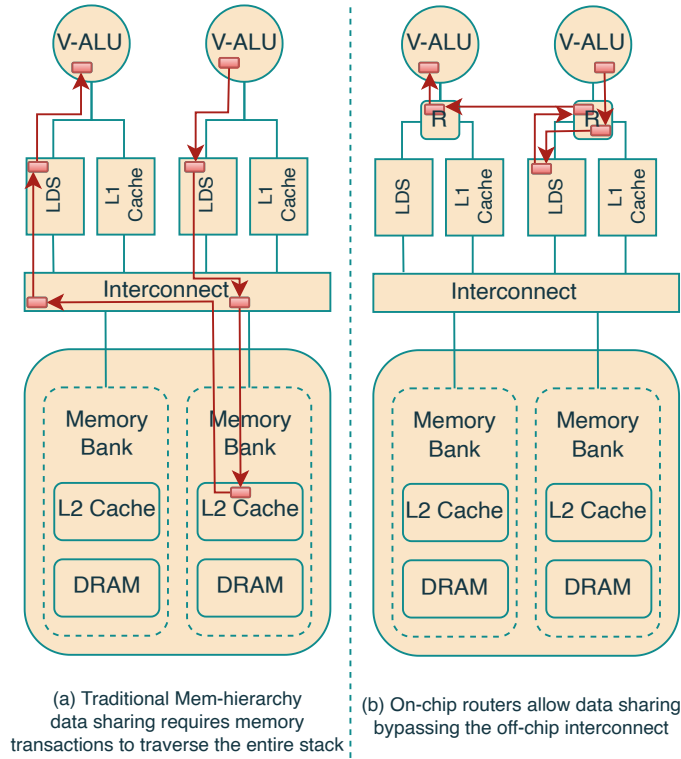


Figure 5.2: Inter-CU communication: Traditional (left) versus proposed on-chip network.

In current GPU microarchitecture design, we can utilize the Local Data Share (LDS) memory space dedicated to one compute unit to enable low-latency communication between threads in one work-group. However, the memory access pattern of the FHE workload requires data to be shared with a large group of threads whose size is much larger than the size of a work-group. Thus, it is not possible in the current GPU microarchitecture design to support data sharing between work-groups through CU-side memory space (i.e., LDS). We propose a novel CU-side network-on-chip

cNoc that interconnects the CUs together, a design that is inspired by the idea of the LDS. We enable a “global” LDS that can be shared between the CUs. We show how data is communicated in the traditional method and in the proposed new on-chip network (see Figure 5.2)

By exploiting the **cNoc**, the dedicated on-chip memory can be shared between compute units, thus reducing the number of redundant memory accesses to the off-chip memory. We also provide synchronization barriers of varying granularity to mitigate race conditions by modifying the logic of the kernel dispatcher. Since the LDS use is under programmer control, our approach does not incur the overhead associated with cache coherence and avoids redundant cache invalidations, but comes with some extra programmer effort. By implementing a global address space (GAS) in our GPU, we establish data sharing and form a unified GAS by combining all LDSs. The virtual address space is then mapped onto this unified GAS, with translation using a hash of the lower address bits.

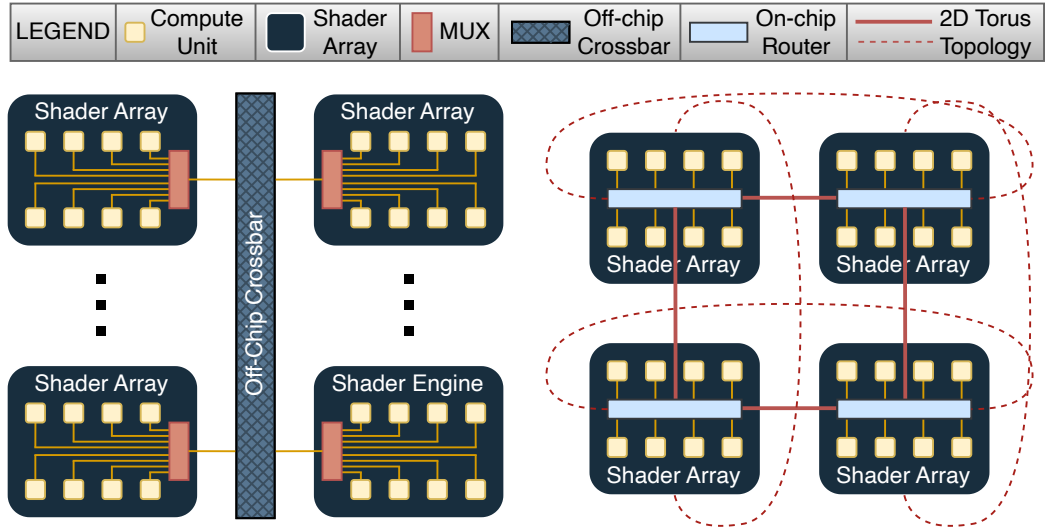


Figure 5.3: Proposed hierarchical on-chip network featuring a concentrated 2D torus topology.

Current GPUs are designed hierarchically (e.g., MI100 GPU comprises numerous compute units), with 8 CUs combined to form a *Shader Array*. There are 15 *Shader Arrays* in total, resulting in a total of 120 compute units. The proposed **cNoc** takes advantage of this hierarchy, utilizing a hierarchical on-chip network (see Figure 5.3) that features a single router for each *Shader Array*, connecting the eight compute units that make up a *Shader Array*. The routers are arranged in a 3×5 2D grid and interconnected through a torus topology. While this concentrated-torus topology [94, 95] can increase network complexity, it reduces the number of required routers (from 120 to 15), thereby minimizing the chip area needed for the network. In a concentrated-torus topology, all routers have

the same degree (number of ports), creating an edge-symmetric topology that is well-suited for the all-to-all communication patterns of FHE workload.

The on-chip network significantly enhances data reuse, reducing redundant memory transactions by 38%, and effectively supports the all-to-all communication pattern common in FHE workloads.

We implement the on-chip network, leveraging the Message-Port-Connection system in NaviSim [81]. We extend the shader array structure with an on-chip router, leveraging simple routing algorithms. We use a connection to connect CUs in the shader array with the router as a node in the 2D torus network. We reuse the communication protocols of accessing the LDS unit for read/write operations. The process of implementing the network does not require modifying files of existing components; only files with new extensions are added, as well as changes in configuration files for plugging the new extensions into the existing simulator to realize new configurations. To further experiment and validate the design, NaviSim allows users to easily configure the hardware under simulation by modifying parameters and component connections in the configuration code. NaviSim exhibits high adaptability in terms of effectively supporting extensions of the current microarchitecture implementations.

5.2.2 Enhancing the Vector ALU

Native modular reduction extension: The existing GPU arithmetic pipeline is highly optimized for data manipulation operations, such as *multiply*, *add*, *bit-shift*, and *compare*. A wavefront executing any of these instructions takes 4 clock cycles to execute in the SIMD units in a lock-step fashion. Conversely, operations such as *divide* and *modulus* are emulated using a series of native instructions, resulting in poorer performance compared to their native counterparts.

As stated before, the modular reduction operation, used for determining the remainder of a division, is performed after each addition or multiplication. As a result, optimizing modular reduction is crucial for speeding up FHE workloads. At present, the MI100 GPU executes a modular operation through a sequence of addition, multiplication, bit shift, and conditional operations, drawing on the conventional Barrett’s reduction algorithm. This process is considerably slow.

To demonstrate the flexibility of our NaviSim simulation framework, we will extend the vector ALU pipeline of the CDNA architecture to natively support modular reduction. We augment the CDNA instruction set architecture (ISA) with a collection of vector instructions designed to perform modular reduction operations natively, after executing an addition or multiplication operation.

The new native modular instructions proposed include:

- Native modular reduction:

$$\text{mod-red } \langle v0, s0 \rangle \mid \mathbf{V}_0 = \mathbf{V}_0 \bmod s_0$$

- Native modular addition:

$$\text{mod-add } \langle v0, v1, s0 \rangle \mid \mathbf{V}_0 = (\mathbf{V}_0 + \mathbf{V}_1) \bmod s_0$$

- Native modular multiplication:

$$\text{mod-mult } \langle v0, v1, s0 \rangle \mid \mathbf{V}_0 = (\mathbf{V}_0 \times \mathbf{V}_1) \bmod s_0$$

Modular reduction involves several comparison operations, resulting in branch divergence in GPUs. Our implementation is derived from an improved Barrett reduction algorithm [16]. This approach minimizes the number of comparison operations to one per modular reduction operation, significantly reducing the number of branch instructions and enhancing compute unit utilization.

Wider multiply-accumulate units In the CKKS FHE scheme [13], we can choose to perform operations on 32, 64, or 128-bit wide RNS limbs for a ciphertext. This limb bit width governs the operand size for the vector ALUs, impacting the number of modular addition and multiplication operations required.

We are faced with an algorithm-level performance versus precision decision when selecting the bit width. If we opt for 32-bit wide RNS limbs, we will have numerous limbs to work with, increasing the available levels, while simultaneously reducing the achievable precision of the application. Conversely, if we select 128-bit RNS limbs, we will have fewer limbs to work with, resulting in a decrease in the number of available levels, but in high precision for an application. With our chosen parameters, using 128-bit wide RNS limbs would leave us with an insufficient number of limbs to perform a single bootstrapping operation, which is a sequence of the building block operations Table 2.1 that reduces the noise within the ciphertext to an acceptable level. To strike a balance between performance and precision, we choose to use 64-bit wide RNS limbs in this work.

Most GPUs on the market natively support 16-, 32-, and 64-bit floating point computations, as well as 4-, 8-, 32-bit integer computations. Unfortunately, they lack dedicated hardware support for 64-bit integer operations, the most commonly used format in FHE workloads. Instructions for processing 64-bit integer operands are emulated using multiple 32-bit integer instructions, making them significantly slower.

To complement our native modular reduction, which relies on 64-bit integer operations, we add support for a hardware-backed 64-bit integer multiplier and accumulator. We also widen the

Table 5.1: Cycle counts for 64-bit modulus instructions comparing MOD and WMAC features

μ -arch. Feature	mod-red (cycles)*	mod-add (cycles)*	mod-mul (cycles)*
Vanilla MI100 [†]	46	62	63
MOD^Δ	26	18	38
MOD+WMAC	17	7	23

[†] Refers to the unmodified CDNA architecture of MI100 GPUs.

*Cycle count is averaged over 10,000 modulus instructions, computed on cached data (using LDS cache) and rounded to the nearest integer.

^ΔModular operation is implemented using various compile-time prime constants as the modulus.

register-file size to accommodate the large ciphertexts. Table 5.1 reports the decrease in total cycles for each of our proposed native modular instructions, in comparison to the MI100 GPU-emulated instructions in the baseline configuration.

Modular reduction involves executing several comparison operations, resulting in branch divergence in GPUs. Our implementation is derived from an improved Barrett reduction algorithm [16]. This approach minimizes the number of comparison operations to one per modular reduction operation, significantly reducing the number of branch instructions and enhancing compute unit utilization.

5.2.3 LABS: Locality-Aware Block Scheduler

So far, our micro-architectural extensions primarily focused on optimizing individual FHE blocks. To better leverage these new features, we focus next on inter-block optimization opportunities, targeting the work-group dispatcher within the CDNA architecture. GPU scheduling is typically managed using streams of blocks that are scheduled on compute units in a greedy manner [8]. The presence of large GPU register files allows the scheduler to oversubscribe blocks to each compute unit. However, the existing scheduler within the CDNA architecture is not cognizant of inter-block data dependencies, forcing cache flushes when transitioning from one block to the next.

We propose a Locality-Aware Block Scheduler (LABS) designed to schedule FHE blocks with shared data together, thus avoiding redundant on-chip cache flushes, specifically in the LDS. The LABS further complements our set of micro-architectural enhancements, which relax the operational constraints during block scheduling and create new opportunities for optimization. For example, the **cNoc** feature enables LDS data to be globally accessible across all CUs, thereby allowing the

scheduler to assign FHE blocks to any available CU.

To develop LABS, we employ a well-known graph-based mapping solution and frame the problem of block mapping to CUs as a compile-time Graph Partitioning Problem (GPP) [96].

Graph Partitioning Problem: We formulate our Graph Partitioning Problem (GPP) by introducing a cost function Φ . For a graph G , if it is partitioned such that E_c denotes the set of edge cuts, then Φ can be expressed as $|E_c|$, the sum of the individual cut-edge weights. Here (v, w) represents the edge-weight of the edge connecting node v to node w .

$$\Phi = |E_c| = \sum_{(v,w) \in E_c} |(v,w)|$$

In this equation, $|(v,w)|$ signifies the data transferred between FHE blocks. Φ is the cost function that reflects the communication overhead associated with assigning FHE blocks to separate compute units. The goal of the graph partitioning problem is to discover a partition that evenly distributes the load across each compute unit, while minimizing the communication cost Φ .

To partition the compute graph and prepare it for mapping onto the architecture graph, we utilize a multilevel mesh partitioning technique. For readers interested in gaining further insights into our graph partitioning implementation of the multi-level mesh partitioning algorithm, we recommend referring to the work of Walshaw and Cross [96].

Architecture-aware mapping: We use two graphs in our locality-aware block scheduler. Let $G = G(V, E)$ represent a directed acyclic compute graph with vertices V and edges E . Here V corresponds to FHE blocks and E indicates the data dependencies of the FHE blocks. Similarly, let $G_a = G_a(V_a, E_a)$ denote an undirected graph, with vertices V_a and edges E_a . Here, V_a represents GPU compute units, and E_a illustrates the communication links between compute units. Both edge sets, E and E_a , are assumed to be weighted, with edge weights of E signifying the size of data transferred between related blocks, and E_a representing the latency of communication between corresponding compute units. We can then define $\pi : V \rightarrow V_a$ as a mapping of V into V_a disjoint subsets.

Our objective is to find a mapping π that minimizes communication overhead between compute units. We focus on mapping our partitioned subgraphs onto the set of compute units V_a , where communication costs (both latency and bandwidth) are not uniformly distributed across the network. To uniformly distribute the communication overheads across the network, we introduce a network cost function Γ . Formally, Γ is described as:

$$\Gamma = \sum_{(v,w) \in E_c} |(v,w)| \cdot |(\pi(v), \pi(w))|$$

Here, Γ is defined as the product of individual cut-weights and their corresponding edge-weights in the architecture graph, when mapped using a mapping function π . In this equation, $\pi(v)$ represents the mapping of block v to a compute unit from the set V_a , after applying the mapping function π . Additionally, $|(\pi(v), \pi(w))|$ represents the communication latency between compute units $\pi(v)$ and $\pi(w)$. Similar to our analysis with Φ , our goal is to minimize Γ . To accomplish this, we use a compile-time optimization by applying *simulated annealing* [96], alongside mesh partitioning for efficient mapping of FHE blocks onto compute units.

5.3 Evaluation

5.3.1 Methodology

We evaluate the three microarchitecture extensions using NaviSim, demonstrating its flexibility and extensibility to support architectural design space exploration. To evaluate the overall design including the inter-FHE-block optimization, a block-level directed acyclic compute graph simulator called BlockSim (developed by Kaustubh Shivdikar) is introduced on top of NaviSim. BlockSim [69] is designed to enable us to identify inter-kernel optimization opportunities. With an adjustable sampling rate for performance metrics, BlockSim accelerates simulations, facilitating more efficient design space exploration. BlockSim generates analytical models of the FHE Blocks to provide estimates for run times of various GPU configurations. When the best design parameters are identified, NaviSim is then employed to generate cycle-accurate performance metrics. NaviSim is highly configurable using a few configuration files that are written in Go to define hardware configuration. As a good example of NaviSim’s degree of reconfigurability, BlockSim enables the restructuring of the kernel dispatcher by modifying the dispatching logic to integrate compile-time optimizations obtained from LABS. Besides supporting FHE workloads, BlockSim serves as an essential component of NaviSim by abstracting low-level implementation details from the user, allowing them to focus on entire workloads rather than individual kernels. We utilize AMD’s CDNA architecture-based MI100 GPU (see table 5.2) to set the baseline GPU configuration for FHE application evaluations.

We iteratively introduce micro-architectural extensions and evaluate the performance benefits of each enhancement. We first evaluate our three micro-architectural extensions (**cNoC**,

Table 5.2: MI100 GPU Parameters

Parameter	Value
GPU Core Freq	1502 MHz
Process Size	7 nm
Compute Units	120
TFLOPS	23.07
Register File	15 MB
CU count	120
L1 Vector Cache	16 KB per CU
L1 Scalar Cache	16 KB
L1 Inst Cache	32 KB
Shared L2	8 MB
LDS	7.5 MB
GPU Memory	32 GB HBM2
Mem Bandwidth	1229 GB/s
Host CPU	AMD EPYC 7002
Host OS	Ubuntu 18.04
GPU Driver	AMD ROCm 5.2.5

Table 5.3: Performance of various FHE building blocks

	CMult	HE-Add	HE-Mult	Rotate	Rescale
Baseline MI100 (μs)	178	217	4012	3473	681
GME* (μs)	22	28	464	364	69
Speedup over Baseline	$8.1\times$	$7.8\times$	$8.6\times$	$9.5\times$	$9.9\times$

*The values displayed here exclude contributions from the LABS optimization, as LABS is an *inter-block* optimization, and the metrics provided are intended for individual blocks.

MOD, **WMAC**), then evaluate our compile-time optimization **LABS**, and conclude with a memory size exploration to determine the impact of on-chip memory size on FHE workloads. We evaluate these micro-architectural enhancements and compiler optimization using NaviSim and BlockSim.

We analyze the performance of three workloads: bootstrapping [43], HE-based logistic regression (HELR) [97] and encrypted ResNet-20 [98] utilizing the CIFAR-10 dataset. For all three workloads, we evaluate the contributions of each individual FHE building block (see Table 2.1) that make up the respective workload. In addition, for these workloads, we report the performance benefits achieved by employing each of the proposed micro-architectural enhancements.

5.3.2 Results

Performance of FHE Building Blocks: We begin by showing the performance of individual FHE blocks. Since these are individual FHE blocks, the reported metrics do not account for our inter-block LABS compiler optimization. We find that `HEMult` and `HERotate` are the most expensive operations, as they require key switching operations that involve the most data transfers from the main memory. The next most expensive operation is `HERescale`, where the runtime is dominated by the compute-intensive NTT operations.

Across the five FHE blocks mentioned in Table 5.3, we achieve an average speedup of $8.8\times$ compared to the MI100 baseline execution. In particular, we see a substantial performance improvement in the most expensive operations, namely `HEMult` and `HERotate`, as our proposed micro-architectural enhancements reduce the data transfer time by $12\times$ for both blocks. For `HERescale`, we manage to decrease the average memory transaction latency by $13\times$ using our micro-architectural enhancements to the on-chip network, cNoC.

Impact of Micro-architectural Extensions: Figures 5.4 and 5.5 highlight the impact

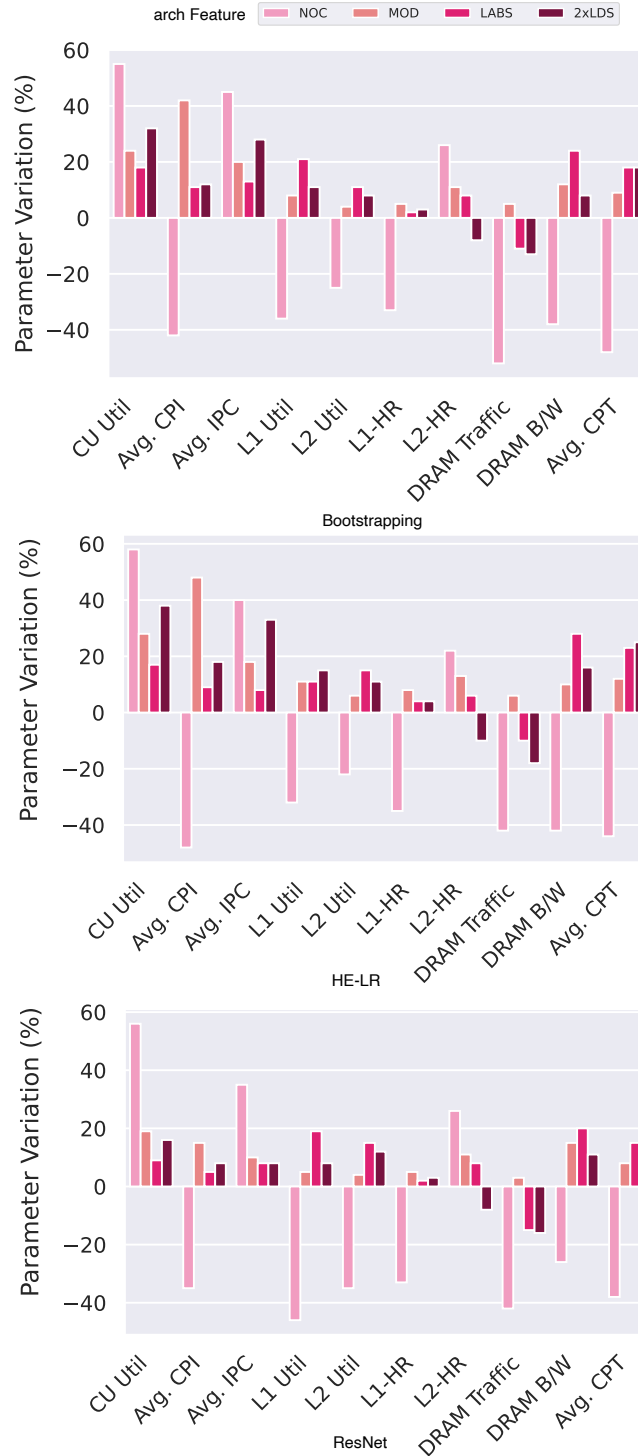


Figure 5.4: Influence of individual proposed micro-architectural extension on architectural performance metrics. Metrics illustrate a cumulative profile where each enhancement builds upon the preceding set of improvements

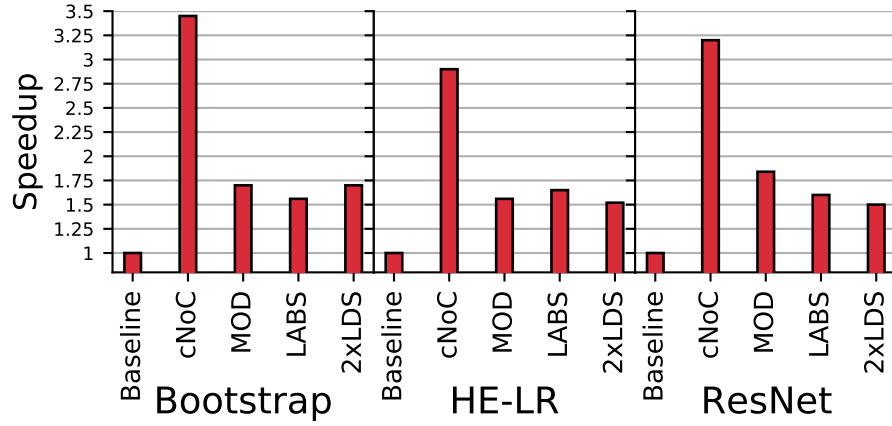


Figure 5.5: Speedup achieved from each micro-architectural extension. The baseline refers to a vanilla MI100 GPU. The reported speedup is cumulative, with each micro-architectural enhancement building upon the previous ones

of each of our proposed micro-architectural extensions, as well as our compile-time optimizations, across three different workloads, i) bootstrapping, ii) HE-LR, and iii) ResNet-20.

First, our proposed concentrated 2D torus network enables ciphertexts to be preserved in on-chip memory across kernels, leading to a significant increase in compute unit utilization across workloads, thereby reducing the average cycles consumed per memory transaction (see Avg. CPT in Figure 5.4). In fact, when comparing the average number of cycles spent per memory transaction (average CPT), we observe that the ResNet-20 workload consistently achieves a lower average CPT value compared to the HE-LR workload. This indicates a higher degree of data reuse within the ResNet-20 workload across FHE blocks, as opposed to the HE-LR workload. With the **cNoC** enhancement, as the data required from previous kernels is retained in the on-chip memory, CUs are no longer starved for data and this also results in a substantial decrease in DRAM bandwidth utilization and DRAM traffic (the total amount of data transferred from DRAM). The L1 cache utilization decreases notably across all three workloads for the **cNoC** micro-architectural enhancement. This is due to the fact that the LDS bypasses the L1 cache, and memory accesses to the LDS are not included in the performance metrics of the L1 cache.

The proposed MOD extension enhances the CDNA ISA by adding new instructions. These new instructions are complex instructions that implement commonly used operations in FHE, like `mod-red`, `mod-add`, and `mod-mult`. As these instructions are complex (composed of multiple sub-instructions), they consume a higher number of cycles as compared to simpler instructions such

as `mult` or `add`. This is the reason for the increase in the average cycles per instruction (CPI) reported in Figure 5.4.

The compile-time **LABS** optimization in our approach further removes redundant memory transactions by scheduling blocks that share data together, thus reducing total DRAM traffic and enhancing CU utilization. **LABS** takes advantage of on-chip ciphertext preservation feature enabled by our **cNoC** micro-architectural enhancement. As Figure 5.5 shows, across bootstrapping, HE-LR, and ResNet-20 workloads, **LABS** consistently delivers an additional speedup of over $1.5\times$ on top of **cNoC** and **MOD**.

On-chip Memory Size Exploration: Finally, we explore the impact of on-chip memory (LDS) size on FHE workload efficiency, as shown in Figure 5.6. By increasing the total LDS size from 7.5MB (which is the current LDS size on MI100 GPU) to 15.5MB, we achieve speedups of $1.74\times$, $1.53\times$, and $1.51\times$ for Bootstrapping, HE-LR, and ResNet-20 workloads, respectively. However, increasing the LDS size beyond 15.5 MB does not result in substantial speedup, as the DRAM bandwidth becomes a bottleneck.

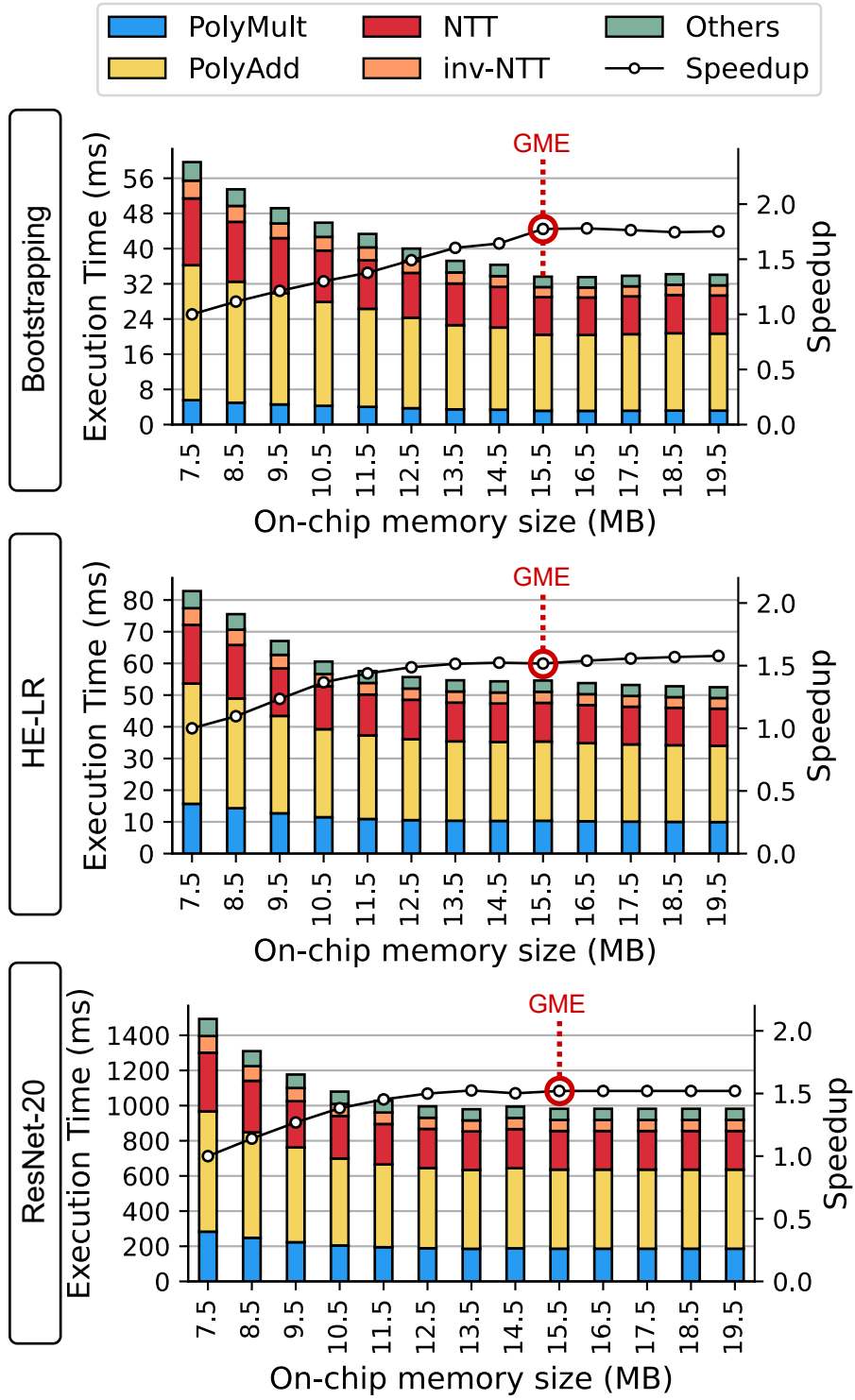


Figure 5.6: Exploring the impact of on-chip memory size on FHE workload performance.

Chapter 6

Simulation Performance Evaluation and Optimization

So far in this dissertation we have focused on designing and building highly adaptable simulators. In this chapter, we have considered simulation performance as the main design criteria. When referring to the term performance throughout the remainder of this chapter, we are referring to the speed of the simulation.

Cycle-level architectural simulators tend to be slow due to their nature of capturing the detailed timing behavior of the underlying hardware. The required level of detail comes at the cost of speed [22]. Multi2Sim [58] is reported to be $44,000\times$ slower than native execution time. Malhotra et al. [61] report the slowdown over the native execution time of GPGPUSim is $480,000\times$. Based on our evaluation, MGPUSim experiences a slowdown of 5 to 6 orders of magnitude compared to native execution time. The rapid evolution of GPU architectures, which has been driven by HPC and ML workloads, requires GPU architects to rapidly evaluate the impact of a variety of new features with performant simulators. This challenge drives the motivation for designing high-performance simulators.

The excessively slow simulation speed motivates us to reconsider overall simulator design, with a focus on performance. Our goal is to discover patterns in the design process that contribute to simulation overhead. Some of the key factors include the degree of abstraction selected for modeling, the software engineering approach employed, the efficiency of the parallel simulation engine, and the impact of applying certain simulation strategies (i.e., DES) have on the simulation performance. To design a high-performance simulator, understanding how these factors influence the simulation

speed and exploring possible strategies to mitigate some overhead are essential steps.

6.1 Methodology

Next, we cover our methodology for examining how different aspects of the simulator design affect simulation speed. Designing a high-performance simulator is an extremely challenging task because it involves multiple complex steps, including: selecting a simulation framework, deciding on the best abstraction level for the simulator models, and adopting a simulator architecture that will drive the implementation of the model. To design a high-performance simulator, it makes sense to exploit multi-threading to accelerate architectural simulation, especially given the number of cores available on today's CPUs. Being able to run a simulation in parallel becomes a key factor in achieving fast and accurate simulation results.

Given all these considerations, we narrow our discussion to focus on the key design decisions that simulator developers need to consider to minimize unnecessary simulation overhead and improve parallel simulation efficiency. Our goal is to develop best practices that can help guide future simulator developers.

6.1.1 General simulator design

Since discrete event simulation [33] (DES) is one of the most commonly and widely used methods of simulation in the field of computer architecture, our discussion focuses on the design of event-based simulators. Discrete event simulation models the system as a series of events occurring at specific moments in time. The behavior of the system is modeled through handling events.

Retry mechanism: Scheduling events to perform work in the future is a flexible and convenient way to model the system behavior. While this approach to simulation has a number of advantages, it still may encounter significant delays. For example, scheduling events that are needed to retry a blocked operation is a common mechanism, and managing this through events works efficiently in most cases. However, in situations where the rate of requests exceeds the rate of request consumption, especially as we scale input data set sizes, this retry mechanism can cause significant overhead. Since message congestion cannot be resolved immediately, continuing to generate additional events to attempt the same operation impact simulator performance and fail to make any forward simulation progress.

CHAPTER 6. SIMULATION PERFORMANCE EVALUATION AND OPTIMIZATION

To avoid this pitfall, we need to consider how to handle potential congestion resulting from a large volume of requests. We need to design a specific mechanism to address it at the consumer end. Predicting all possible scenarios that could lead to congestion can be challenging. To address this issue effectively, we need to design our simulator to monitor the total number of active events and identify events that result in multiple retry operations.

Emulation: When designing simulators that support both functional (i.e., for emulation) and timing simulation, a key design consideration is whether to create a standalone emulation model or to develop a unified model that serves both emulation and timing simulation. While a custom emulation model may be more performant, developing a single model that can be used for both emulation and timing simulation will support developing reusable code and increase maintainability of the simulator code. However, it could introduce performance drawbacks during emulation.

For event-driven simulation, the granularity of events defined by the simulator greatly impacts the simulation speed. Compared to timing simulation, which generally requires more fine-grained event definitions, emulation can leverage a coarser grained event definition that can boost performance. From a performance perspective, a separate emulation model offers advantages over a unified model.

Level of abstraction: When designing a simulator, a critical decision is to decide the level of the detail we should use when simulating the hardware. This decision is critical since it can impact both the simulation speed and the accuracy. The goal of the architectural simulator is to enable performance modeling of the hardware behavior. A thorough understanding of the factors that impact the underlying hardware performance is crucial for determining the appropriate level of abstraction selected for the simulator. Developers must navigate the trade-off between simulator performance and accuracy to optimize their design decisions.

A necessary step is to examine the performance after the simulator is built and validated. We will undoubtedly find that some of our decisions lead to notable performance bottlenecks. Can we raise our level of abstraction by stripping away certain details in our model while still maintaining accuracy? The answer depends on whether the details being removed significantly impact the performance. One approach is to develop multiple models, each possessing a different level of detail and perform experiments to decide which one works best, in terms of accuracy and performance.

6.1.2 Parallel simulation scalability

Modern computer systems are inherently parallel. Given the growing number of cores in current CPUs, we should explore how best to utilize these cores to run simulation efficiently. This may enable us to simulate the execution of our designs in parallel. Although the parallel simulation mechanism can improve performance as compared with serial simulation, the performance scalability issue becomes a concern.

Parallel simulation engine: A parallel simulation engine allows multiple simulation tasks to run concurrently, distributing the workload across multiple processors or cores. For event-driven simulators, parallelism can be introduced by processing multiple events concurrently. In the parallel engine, we aggressively manage the event scheduling and exploit multiple threads, but we still need to synchronize shared data access and ensure the correct order of operations. The efficiency of the parallel simulation engine is crucial for achieving high scalability when performing parallel simulation.

Synchronization: When implementing components as part of a multi-threaded program, we need to carefully examine the different scenarios where shared resources are modified and properly manage the resources (with locks or semaphores) to prevent data races. However, synchronization overhead can significantly degrade performance. Finding a balance between an efficient locking mechanism and scalability is the key to designing a parallel simulation framework. We believe having a highly modular design for each component can keep the locking mechanism simple and clean. We also believe separating the initialization of the shared resources from accessing them can reduce the synchronization requirements.

Connections: The design of the communication system is essential for the scalability of parallel simulation. If the messages exchanged between components cannot be transferred in parallel, the communication system becomes a major bottleneck for all threads. A straightforward connection design is having a universal communication system connecting all components, which could cause significant synchronization overhead because different messages interfere with each other by sharing the same connection. A more parallel-friendly communication system should have individual connections linking two ports (owned by components or switches) and use multiple switches to manage routing.

6.1.3 Performance Metrics

The most common metric to measure simulation speed is overall simulation execution time (i.e., *wall-clock time*). To better measure the simulation overhead, we introduce a set of performance metrics, that offer a comprehensive view of the measurement of the simulation overhead.

Slowdown: Slowdown indicates how many times slower the simulation is as compared to native execution. This is the most intuitive method of measuring the simulation overhead.

$$\text{Slowdown} = \frac{\text{simulation execution time}}{\text{native execution time}}$$

KIPS: Kilo-simulated-instructions per second (KIPS) indicates how many seconds are spent to simulate the execution of 1000 instructions. The simulated instruction count is an important metric that architectural simulators provide, in addition to the predicted kernel execution time [58].

$$\text{KIPS} = \frac{\text{simulated instruction counts}}{\text{simulation execution time} * 1000}$$

CIPGI: CPU instructions per simulated GPU instruction (CIPGI) indicates how many instructions are executed on the host system to simulate one GPU instruction.

$$\text{CIPSHI} = \frac{\text{CPU instruction counts}}{\text{simulated GPU instruction counts}}$$

6.2 Evaluation

To explore the various factors that influence simulator performance, and develop an engineered process for tuning the performance of the simulator, we use NaviSim to ground our discussion. We conduct a thorough performance analysis of NaviSim while running simulations in both emulation mode and timing simulation mode. We profile NaviSim as an application to find bottlenecks and performance issues. We discuss our findings and attempt to pinpoint bottlenecks and inefficiencies in the execution of the simulator code. Our goal is to build a process that leverages the execution profile of NaviSim to help guide a set of software-level optimizations to improve simulation speed.

6.2.1 Experimental Setup and Profiling Methods

We evaluate a set of GPU benchmarks (see Table 6.1) selected from a wide range of benchmark suites and supported by NaviSim. We elect to use the default configuration of the R9

CHAPTER 6. SIMULATION PERFORMANCE EVALUATION AND OPTIMIZATION

Nano GPU as our simulator configuration (details are provided in Table 6.2). Our experiments run in two different simulation modes: i) emulation mode and ii) timing simulation mode, on Linux platforms (see Table 6.3).

Table 6.1: Benchmarks

Abbr.	Suite	Workload
ATAX	PolyBench	Matrix Transpose and Vector Multiplication
BS	AMDAPPSDK	Bitonic Sort
FWT	AMDAPPSDK	Fast Walsh Transform
FIR	HeteroMark	Finite Impulse Response Filter
FLW	AMDAPPSDK	Floyd-Warshall Algorithm
KM	HeteroMark	KMeans Clustering
MM	AMDAPPSDK	Matrix Multiplication
MT	AMDAPPSDK	Matrix Transpose
PR	HeteroMark	Page Rank
ReLU	DNNMark	Rectified Linear Unit
SPMV	SHOC	Sparse Matrix-Vector Multiplication

To profile NaviSim’s execution, we utilize the Go performance profiling tools, which include **pprof** and **trace** [39], to perform a thorough analysis as we identify performance bottlenecks. We use the Go profiling tool **pprof** to collect profiling data and visualize the different profiles, including heap usage, CPU usage, thread blocking profiles and execution trace data. The CPU profile identifies the code sections that dominate overall execution time. The CPU profile provides both a textual and graphics-based visualization of the profile data to help identify the most expensive portions of the simulation. The Go profiling tool **trace** provides execution details of each Go’s lightweight thread (called goroutine) by tracing the runtime events within an interval. The goroutine analysis view displays information about each set of goroutines that share the same function. Since NaviSim is an event-driven simulator, we also explore and analyze the event information (i.e., event name, event time, and execution time) of all the events being executed in the simulation, inspecting them in time order. This helps us identify efficiencies during event handling and parallel event execution.

Table 6.2: Simulator configuration

Param	Large
Base Model	R9 Nano
# CU	64
Core Freq	1000MHz
TFLOPS	8.19
L1V	16KB
L0V \$ Assoc.	4-way
L1 Inst \$ [@]	32KB
L1 Scalar \$ [@]	16KB
L2 \$	2MB
L2 \$ Assoc.	16-way
DRAM Tech	HBM
DRAM Size	4GB
Mem Freq	500MHz
Mem Bus	4096 bit
Mem BW	512 GB/s

Table 6.3: Evaluation platform

Parameter	Platform1	Platform2
CPU	AMD Ryzen	AMD EPYC 7742
	Threadripper 2950X	64-Core Processor
OS	Linux Ubuntu 18.04	CentOS 7 Linux
Cores	16C/32T	64C/128T
Max Freq	3.5 GHz	2.25GHz
L1	32KB(D) + 64KB(I)	32KB(D) + 32KB(I)
L2	512KB	512K
L3	8MB	16MB
DRAM	64GB	1TB

6.2.2 Performance Analysis

In this section, we take a deeper look at the performance of NaviSim. This analysis helps to ground our methodology, which will be used to make key design decisions that simulator developers need to consider in order to tune simulation performance and improve parallel simulation efficiency.

6.2.2.1 Overall Performance analysis

As we mentioned in our methodology, examining the performance after the simulator is built and validated is a necessary step to optimize the design decisions. With the assistance of the textual and graphics-based visualization of the profiling results, we identify that a key bottleneck involves the built-in `copy` function, which internally copies values from one buffer to another by calling the `runtime.memmove` function in Go’s runtime. Figure 6.1 is a partial CPU profile generated from running the Go `pprof` tool on top of running the FIR benchmark. We find both emulation and timing simulation encounter this bottleneck.

We are able to trace the origin of this behavior to locations in the simulator where copy functions are called, discovering that are tied to accesses in the `scratchpad` and `scratchpadpreparer` data structures. The `scratchpad` is a data structure, defined in the implementation of the ALU component used for the ALU to store input and output data. The `scratchpadpreparer` is a data structure that prepares the operands by reading from or writing into the register files before ALU starts to access these operands. The `scratchpad` is an internal storage space in the simulator to model some hardware logic used for temporarily storing operands for the ALU.

This performance bottleneck poses a question of whether we could raise the level of abstraction when we model the ALU. We offer some potential solutions in section 6.3.

6.2.2.2 Parallel Simulation Scalability

NaviSim supports simulating hardware operations in parallel by leveraging the Akita simulator framework [81]. Akita employs a parallel simulation engine to support parallel simulation. The parallel engine triggers events that are scheduled at the same time in parallel. This strategy follows a conservative parallel discrete-event simulation methodology (PDES) [99], which maintains the chronological order of the events and will not impact simulation correctness.

We evaluate the efficiency of the Akita parallel simulation engine used in NaviSim. First, we run a set of benchmarks, while leveraging the cores on our multicore CPU used for simulation.

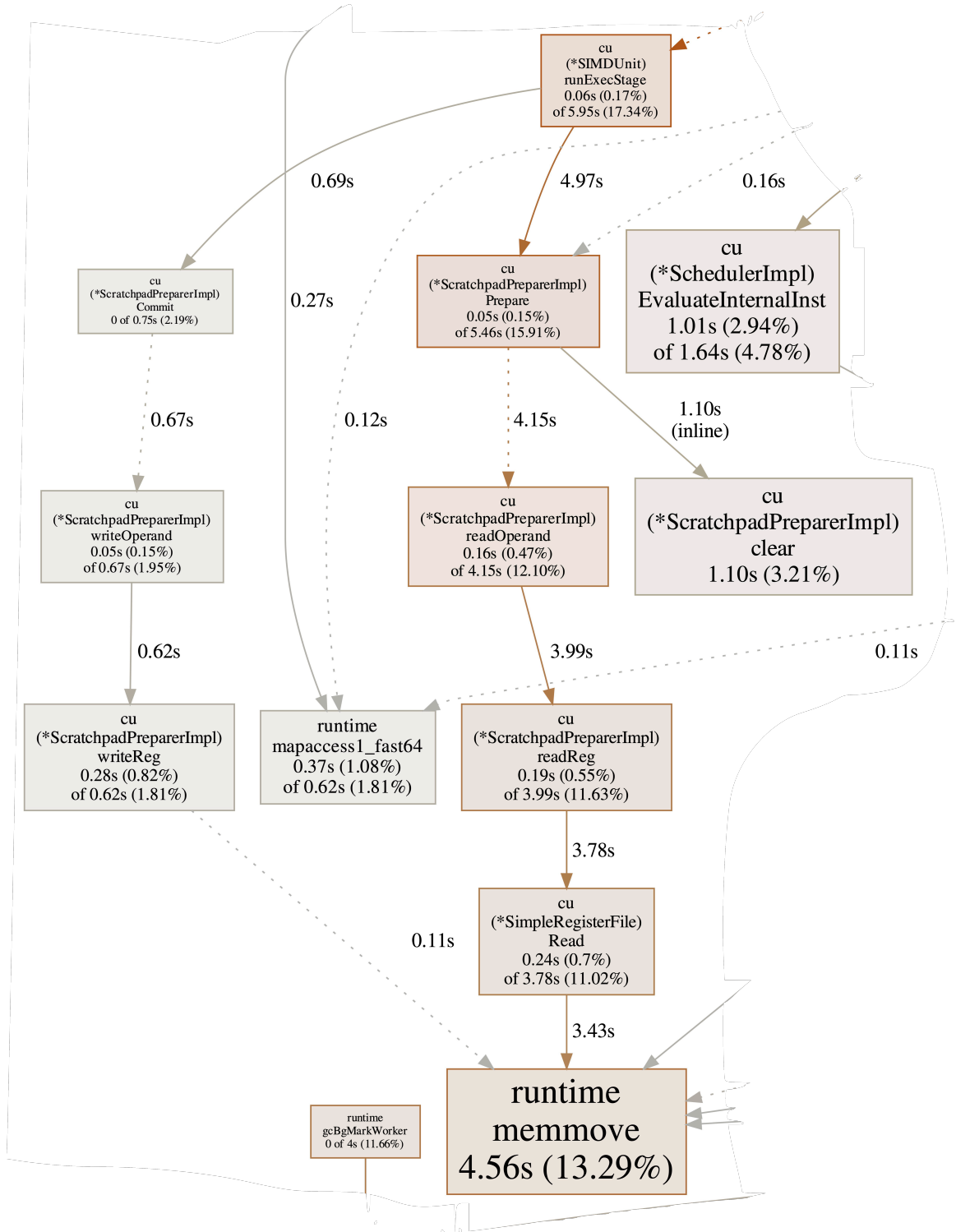


Figure 6.1: The CPU profile from the Go pprof tool, while running the FIR benchmark timing simulation.

CHAPTER 6. SIMULATION PERFORMANCE EVALUATION AND OPTIMIZATION

We set the number of CPU cores to be 1, 4, 8, and 16, which is controlled using the environment variable `GOMAXPROCS`.

Emulation: Figure 6.2 shows the speedup achieved through parallel simulation when running emulation mode. We normalize the simulation execution time of running with 4, 8, and 16 cores against 1 core to show the speedup. The average speedups for 8 and 16 cores are 2.64 and 2.44. For most benchmarks, running with 8 cores gives the best performance, and when increasing to 16 cores, we either obtain the same speedup or performance degrades.

Clearly, there are some performance issues with the parallel emulation. Our approach to address these performance issues is to study the event list during emulation, without applying a parallel engine. First, we analyze the proportion of events that do the actual instruction emulation, because these are the events that can be handled in parallel. We find that only a portion of the total time is spent in executing events that are related to emulation. The rest of the events are created and handled by simulating other parts of the system (i.e. driver, dispatcher). We believe that the Workgroup Dispatcher is the most inefficient component, and there are opportunities to streamline its functionality so as to reduce some unnecessary events.

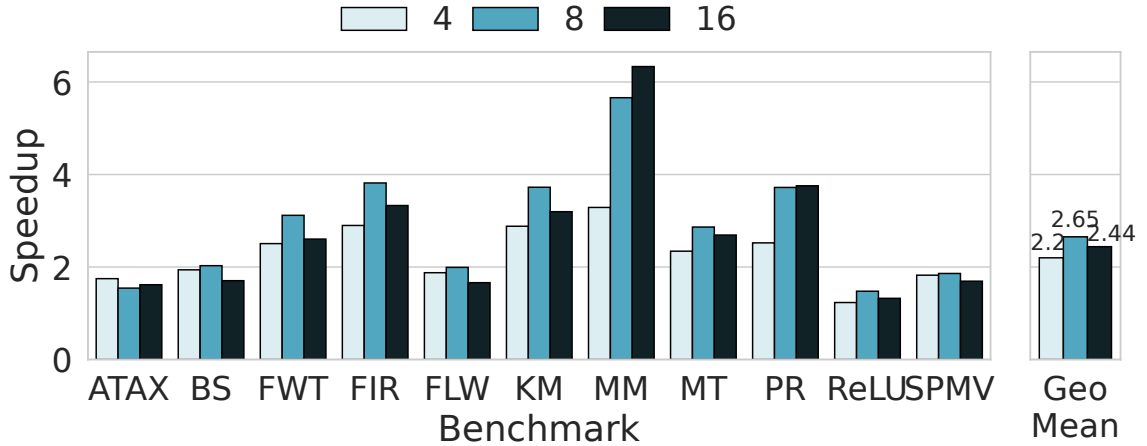


Figure 6.2: The speedup achieved through parallel simulation when running emulation mode.

Apart from analyzing the event list, we use the Go tool **trace** to further investigate the behavior of the goroutines. Figure 6.3 shows a table of specific goroutine instances, with various execution statistics. Figure 6.4 shows a portion of a synchronization blocking profile. The profiling results shown in Figure 6.3 and Figure 6.4 are generated when running the ReLU benchmark emulation in parallel with `GOMAXPROCS=16`. The time interval we look at here is 100ms. The full

CHAPTER 6. SIMULATION PERFORMANCE EVALUATION AND OPTIMIZATION

synchronization blocking profile can be found in Figure A.1.

The goroutine analysis view indicates only half of the total profiling time is spent on execution for all goroutines, and about 20% of the time is spent as Block time caused by the `sync` package. The synchronization blocking profile further confirms that these goroutines are competing for shared resources (i.e., they are accessing the same data structure simultaneously). The `sync` package provides locks to avoid race conditions. As a result, these requests must be executed sequentially, which introduces waiting/blocking time. The graph of the profile data, shown in Figure 6.4, visualizes where this blocking occurs. Each box in the graph corresponds to a single function, and the boxes are sized according to the length of the time in which the function spends running. The largest box is the `Lock` function from the `sync` package. We see that synchronization blocking is implemented in three functions — `find`, `getTable` and `createOrGetStorageUnit`. `createOrGetStorageUnit` is the function that spends the most time waiting.

Goroutine	Total		Execution time	Block time (sync)	Block time (syscall)	Sched wait time	Syscall execution time	Unknown time
91	100.199617ms		47.954555ms	18.110863ms	0s	34.020279ms	0s	113.92µs
85	100.199617ms		44.368583ms	19.435784ms	0s	36.252274ms	0s	142.976µs
96	100.199617ms		45.382724ms	18.928963ms	0s	35.72249ms	0s	165.44µs
89	100.199617ms		46.285062ms	18.819978ms	0s	34.962289ms	0s	132.288µs
98	100.199617ms		54.094009ms	15.684618ms	0s	30.278782ms	0s	142.208µs
94	100.199617ms		45.18432ms	20.029839ms	0s	34.73125ms	0s	254.208µs
84	100.199617ms		48.905918ms	16.79304ms	0s	34.409651ms	0s	91.008µs
87	100.199617ms		49.323455ms	16.390547ms	0s	34.218543ms	0s	267.072µs
93	100.199617ms		48.996868ms	16.689414ms	0s	34.412023ms	0s	101.312µs
86	100.199617ms		49.354107ms	17.938125ms	0s	32.663289ms	0s	244.096µs
95	100.199617ms		48.154244ms	18.28775ms	0s	33.714487ms	0s	43.136µs
90	100.199617ms		47.636099ms	17.632591ms	0s	34.799343ms	0s	131.584µs
88	100.199617ms		46.37632ms	18.572368ms	0s	35.057777ms	0s	193.152µs
83	100.199617ms		49.732733ms	17.812752ms	0s	32.438196ms	0s	215.936µs
97	100.199617ms		48.991436ms	17.982732ms	0s	32.953385ms	0s	272.064µs
92	100.199617ms		48.845949ms	18.026819ms	0s	33.244993ms	0s	81.856µs

Figure 6.3: Execution time breakdown in goroutine analysis view, using the Go trace tool.

Timing simulation: Figure 6.5 shows the speedup achieved through parallel simulation when running in timing simulation mode. Similar to the emulation results, we normalize the simulation execution time when running with 4, 8, and 16 cores against the time for a single core, reporting simulation speedup. The average speedups for 8 and 16 cores are 2.12 and 1.97, respectively. We are encountering a similar issue as we found in the emulation results. However, the underlying cause of the performance scalability issue for simulation is not the same as the emulation.

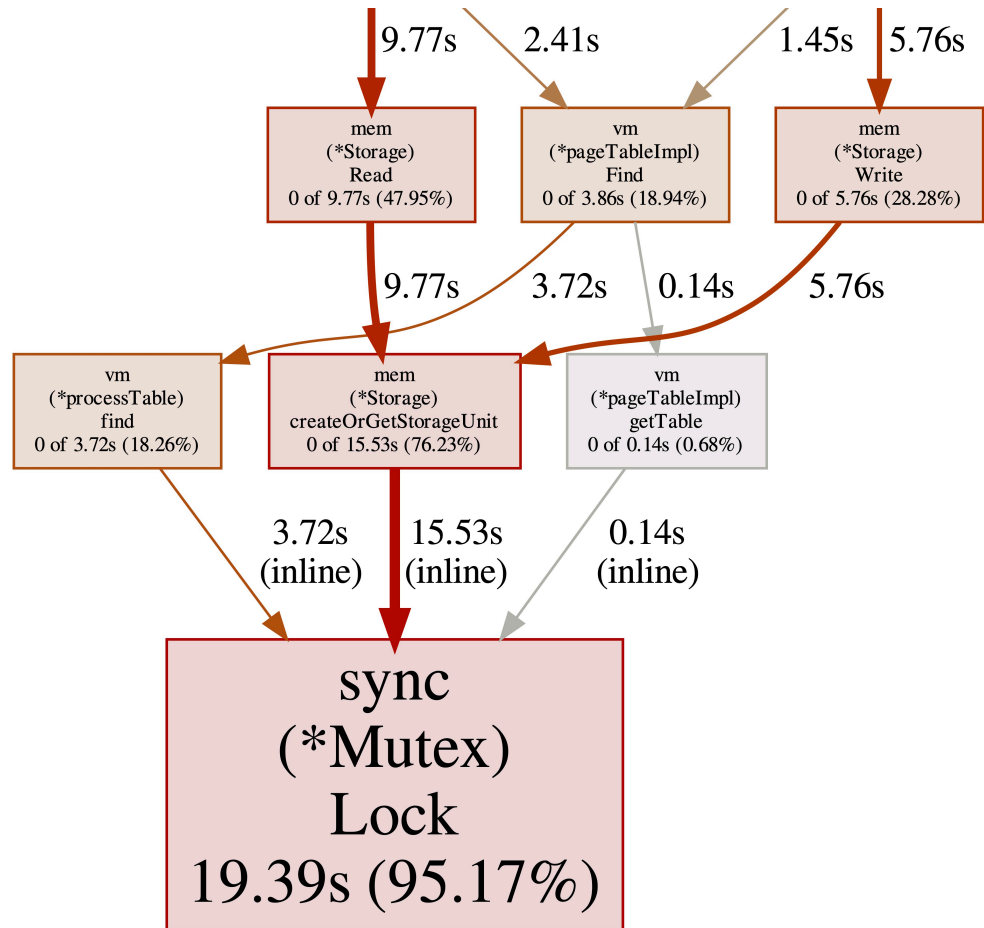


Figure 6.4: The synchronization blocking profile from the Go trace tool, while running the ReLU benchmark parallel emulation.

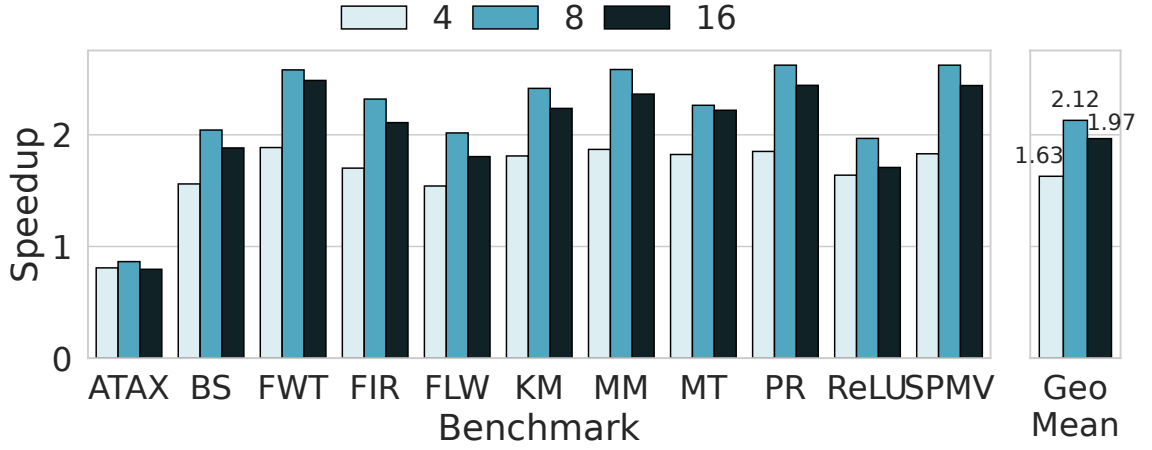


Figure 6.5: The speedup achieved through parallel simulation when running timing simulation.

Compared to emulation, the number of events handled in timing simulation grows exponentially. The memory subsystem simulation generates a significant number of events during timing simulation, which differs in contrast to emulation mode, which models an ideal memory since emulation is only concerned with correctness.

Figure 6.6 is a portion of the synchronization blocking profile generated when running the ReLU benchmark timing simulation in parallel with `GOMAXPROCS=16`. The full synchronization blocking profile can be found in the Figure A.2.

One of the key interfaces of the simulation engine is *Schedule*. *Schedule* registers events by inserting them into an event queue. *Run* is another key interface that extracts events from the event queue and lets the events be handled. To support concurrent event handling, we provide several event queues in the parallel simulation engine. The engine manages them through `Channel` which is a built-in data type in Go for concurrency and is used for communication between goroutines. To avoid race conditions, modifying an event queue on both ends (i.e., inserting into the queue (initiated by *Schedule*), and extracting from the queue (signaled by *Run*) simultaneously, is not allowed. A `Channel` is used to control when the event queues can be modified, enabling the goroutine to schedule events, while keeping the available event queues in the channel. The profile indicates that the goroutines are waiting to receive an event queue from the channel `queueChan` in the *Schedule* function, while trying to insert an event into the queue to complete scheduling that event. We observe that the first element in the queue array always receives more events than the other queue array entries. As the position to enter the queue becomes deeper in the queue array, the fewer events can

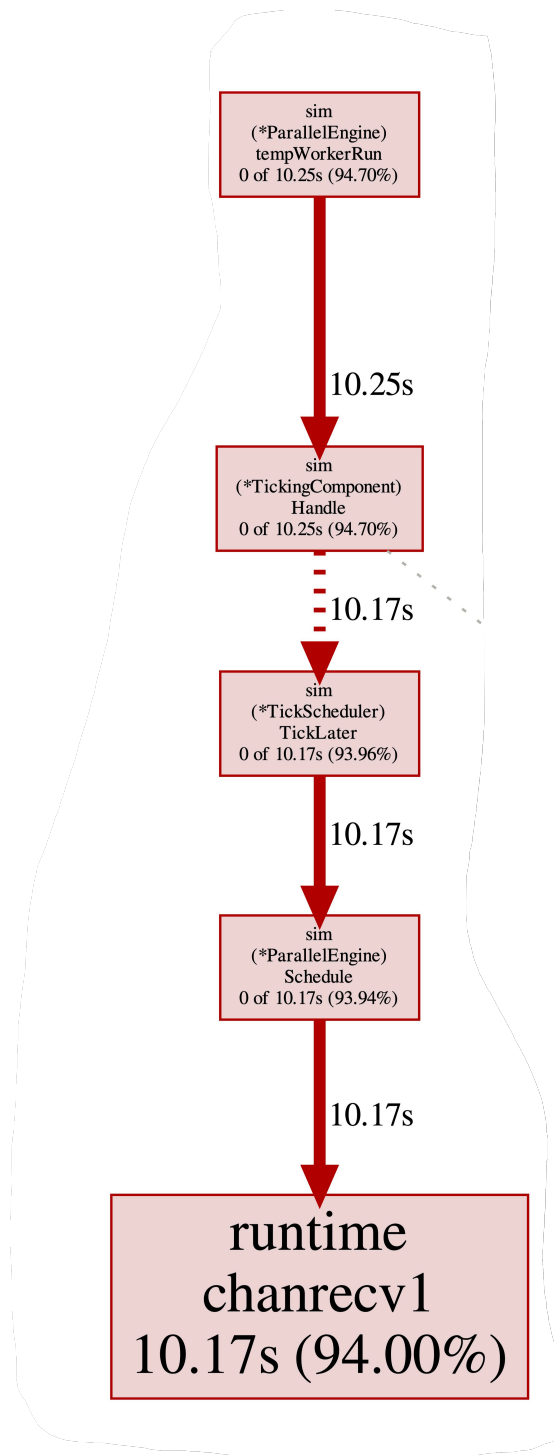


Figure 6.6: The synchronization blocking profile from Go trace tool for ReLU benchmark parallel timing simulation

be received. This is because the scheduling policy always tries to schedule events in the earliest queue array entry. The large number of events assigned to the first queue array entry impacts the wait time for the next round of executions, since all goroutines have to wait before this number of events are launched, in order to have access to the first available queue. To lower the number of events scheduled in the first queue array entry in each round, we utilize a strategy by swapping the position of the entries in the queue array. The last queue array entry assigned in the previous round becomes the first queue array entry in the current round. This can reduce the queue processing time at the beginning of each round.

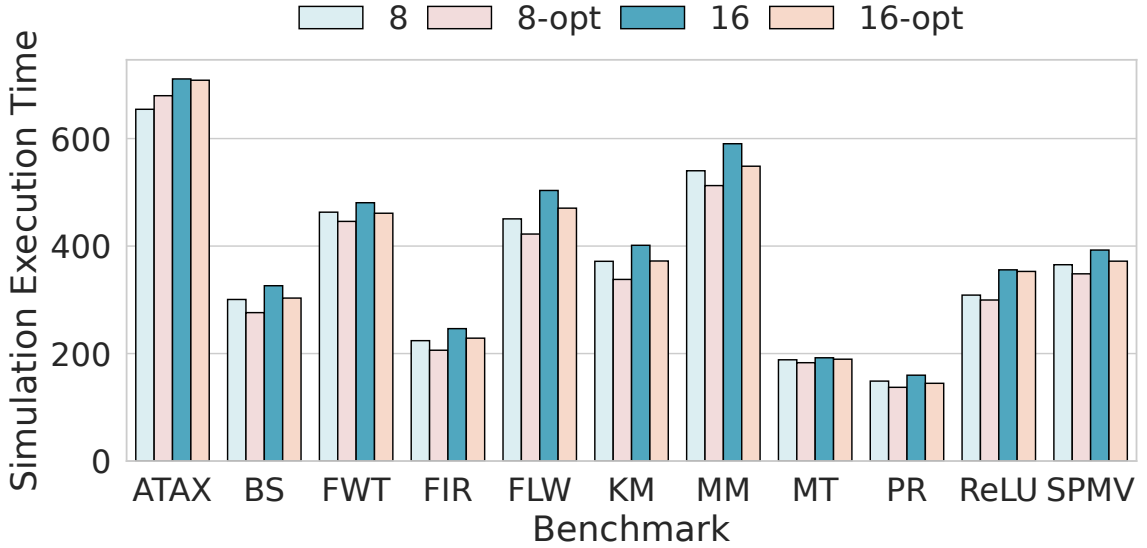


Figure 6.7: Execution performance comparison comparing the baseline and improvements made to the parallel engine.

Figure 6.7 shows a comparison between the simulation execution time results of running parallel timing simulation with 8 and 16 cores. The baseline is the results running with the original implementation of the parallel simulation engine. We observe a slight execution time decrease, indicating this strategy successfully reduces the blocking time, but it does not help with the scalability issue. To investigate further, we generated a new synchronization blocking profile given this strategy applied. A portion of the profile is shown in Figure 6.8. The full synchronization blocking profile can be found in the Figure A.3. This profile verifies that the channel blocking bottleneck has been eliminated. This experiment demonstrates that the design for concurrent event execution, along with the synchronization mechanism plays an important role in the efficiency of the parallel simulation

engine.

However, the profile suggests a different bottleneck. We can see that goroutines are waiting to get the lock to execute the `Send` function of the `Connection`, which is a basic data structure defined in Akita [81]. NaviSim leverages the Akita simulator framework which provides a port-connection-message system to support communication between components, as we mentioned in section 4.2. The Akita simulator architecture offers a single connection to link multiple ports, although each port can only be served by a single connection. When connecting multiple ports, the connection functions as a round-robin arbitrator, managing both the sender and receiver side buffers of each port. This eliminates the need to design separate switches and simplifies the configuration process. However, it requires a mutual exclusion lock to protect the `Send` function, because the `Send` function manipulates the buffers of all ports. When `Send` functions are called from different ports in multiple goroutines simultaneously, these operations must be serialized. A significant amount of blocking time occurs, as can be seen in Figure 6.8. This blocking time is caused by the connection that connects all L2 cache partitions to the L1 scalar, instruction and vector caches from all Compute Units. The R9 Nano GPU configuration (see Table 6.2) has 16 L2 cache partitions and 16 Shader Arrays (each has 4 Compute Units with 6 L1 caches), resulting in 96 L1 caches. The fact that 16 writeback L2 caches are sending messages to 96 L1 caches concurrently through a single connection significantly increases the chance that multiple goroutines call the `Send` function simultaneously.

6.3 Performance Improvement Results

Based on our profiling analysis, we have identified several bottlenecks in both emulation and timing simulation. In this section, we apply software-level optimizations to mitigate these bottlenecks to improve simulation speed. We focus on the emulation performance and show the optimization results.

Performance Debugging We observe ReLU and PR take much more time to finish their emulation than other benchmarks. After checking the event list when executing ReLU, we notice a large number of events scheduled and handled in the Compute Units. We pinpoint the place where these events are created and find that most of the events are created due to the retry mechanism, basically scheduling new events to retry sending a request to the Workgroup Dispatcher. The failure to send the request causes the scheduling of the next event to retry.

This indicates we encounter the retry mechanism pitfall we mentioned in subsection 6.1.1. We can avoid this by adjusting the way events are handled at the consumer end. Specifically for

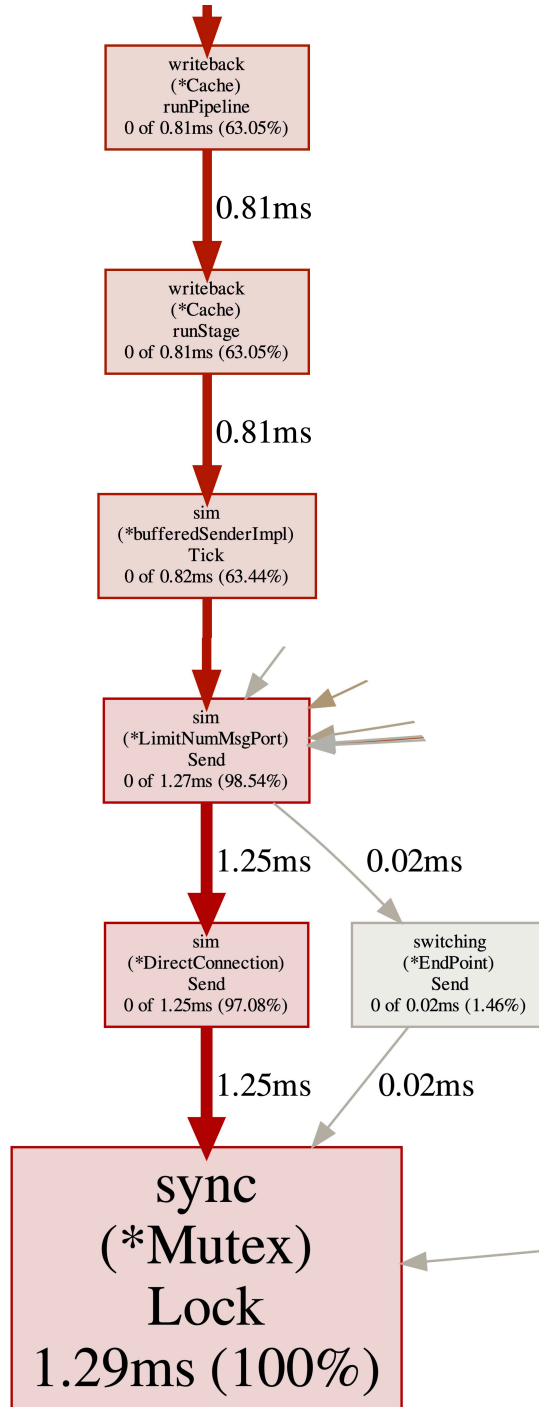


Figure 6.8: The synchronization blocking profile after applying the reversing-order strategy.

CHAPTER 6. SIMULATION PERFORMANCE EVALUATION AND OPTIMIZATION

NaviSim, we modify how we handle `WGCompleteEvent` in our Compute Unit implementation during emulation. We use the original implementation of NaviSim as the baseline and we have profiled the baseline code in subsection 6.2.2 to identify the performance issues. In the baseline implementation, if a workgroup to the Workgroup Dispatcher operation fails, a retry will be generated, creating a new `WGCompleteEvent`. This retry will generate a new request to the Workgroup Dispatcher and repeat this retry process. However, the Workgroup Dispatcher, which is designed to process one workgroup at a time struggles to process a large number of workgroups that are completed within a very short time frame. As a result, the number of events and requests increases rapidly.

To avoid negatively impacting the timing simulation, which could occur if we modified the Workgroup Dispatcher, we opted to change the way we handle `WGCompleteEvent` in the Compute Unit, given that both emulation and timing simulation use the same dispatcher implementation. The modified implementation waits until all workgroups finish and then sends a single request. We also modify the Workgroup Dispatcher accordingly to handle this new form of request.

According to our observations (see subsubsection 6.2.2.2) we found that the Workgroup Dispatcher does not run emulation efficiently, we propose and implement a new Workgroup Dispatcher tailored for emulation. The emulation dispatcher and timing simulation dispatcher share the same interface, but have their own implementations. The new emulation Workgroup Dispatcher removes timing-related modeling and this allows for the simultaneous dispatch of all workgroups in a kernel, also reducing the number of events used for dispatching.

Figure 6.9 shows the execution time for the simulator for the three cases: the baseline, the change that modified the Compute Unit, and the new Workgroup Dispatcher dedicated to emulation. Both resolve the emulation speed issue for the PR and ReLU benchmarks, reducing the execution time significantly. The new Workgroup Dispatcher, which is custom-designed for emulation, improves the efficiency of dispatching workgroups for emulation and reduces the emulation time slightly for most benchmarks.

Overall Performance Based on the profiling results we collected, which were shown in subsubsection 6.2.2.1, we propose a new model of the ALU that does not model the hardware logic used for temporarily storing operands for the ALU. This new model maintains a lower level of detail in ALU, allowing us to remove the `scratchpad` data structure and other related data structures. We reimplement some core functions of the ALU, including: accessing operands from the register files and eliminating the intermediate data structure `scratchpad` for storing input and output data. The redundant memory copy operations between buffers have been removed. We eliminate the

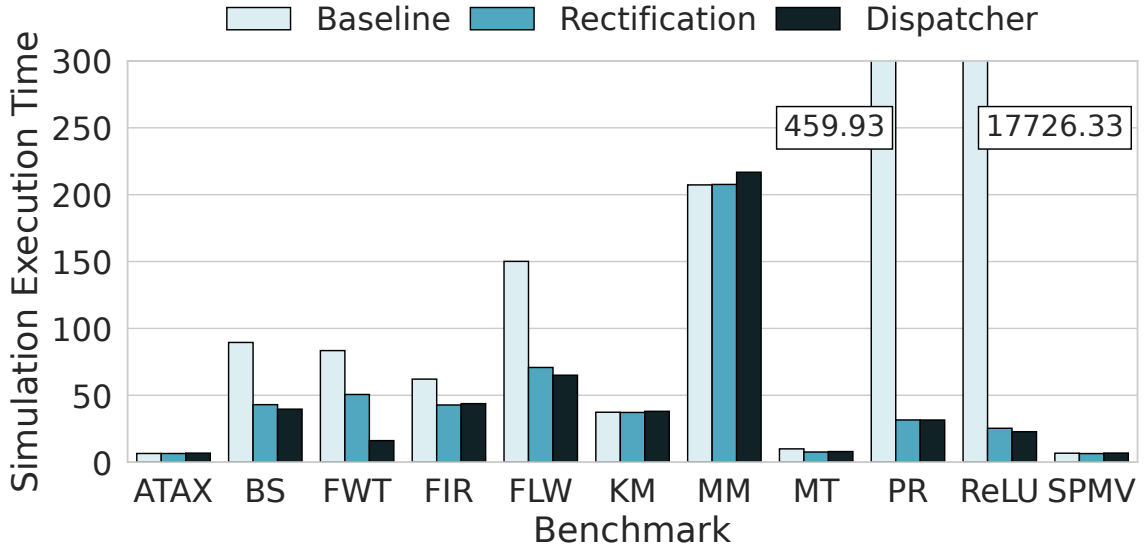


Figure 6.9: Performance comparison between the baseline, the modified Compute Unit and the new Workgroup Dispatcher.

bottleneck by raising the level of abstraction in modeling the ALU.

We use the case where the new Workgroup Dispatcher is applied as our new baseline simulator and provide a performance comparison between the original ALU and the new ALU implementations. The average speedup gain is 2.54.

The CPU profile generated by **pprof** indicates timing simulation suffers from a similar performance issue caused by the use of the `scratchpad` and `scratchpadpreparer` data structures. We modified the Compute Units by utilizing the new ALU component for emulation mode optimization. The performance comparison and speedup are shown in Figure 6.11. The average speedup is 1.1. Our results show the kernel execution time predicted by NaviSim remains accurate when applying the new, less detailed ALU implementation.

In summary, we introduce a new Workgroup Dispatcher for emulation, which addresses a key simulator bottleneck and improves overall performance. We then proposed a new ALU model by reimplementing the instruction emulation to remove both `scratchpad` and `scratchpadpreparer` data structures, which involved extensive code modification because every instruction has to change the way they access operands. Applying the above optimizations, we managed to achieve an average speedup of $4.6\times$, reducing the Slowdown from 71,405 to 19,078, increasing the KIPS by $4\times$, and lowering the CIPGI by 55.43%.

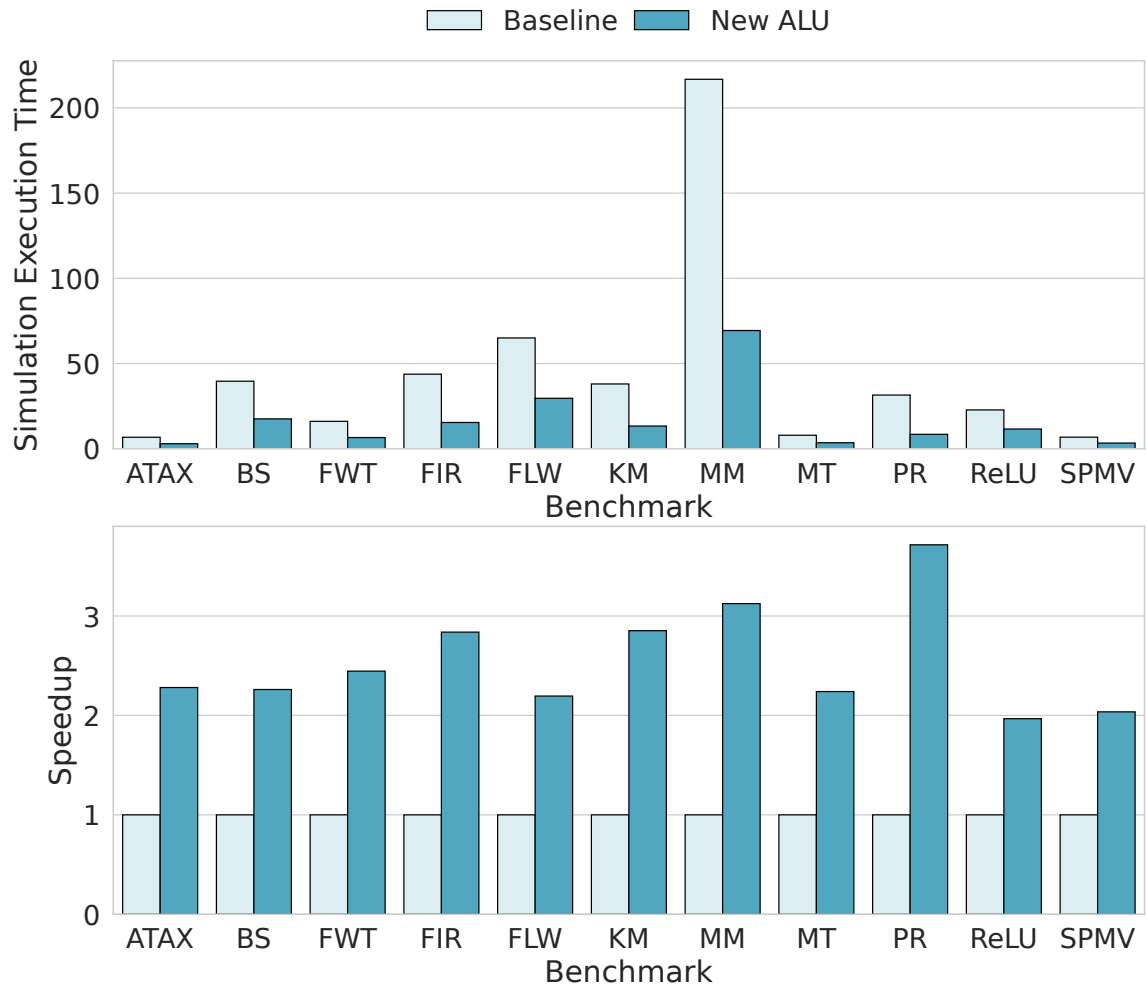


Figure 6.10: Emulation performance comparison between the original ALU and the new ALU implementations.

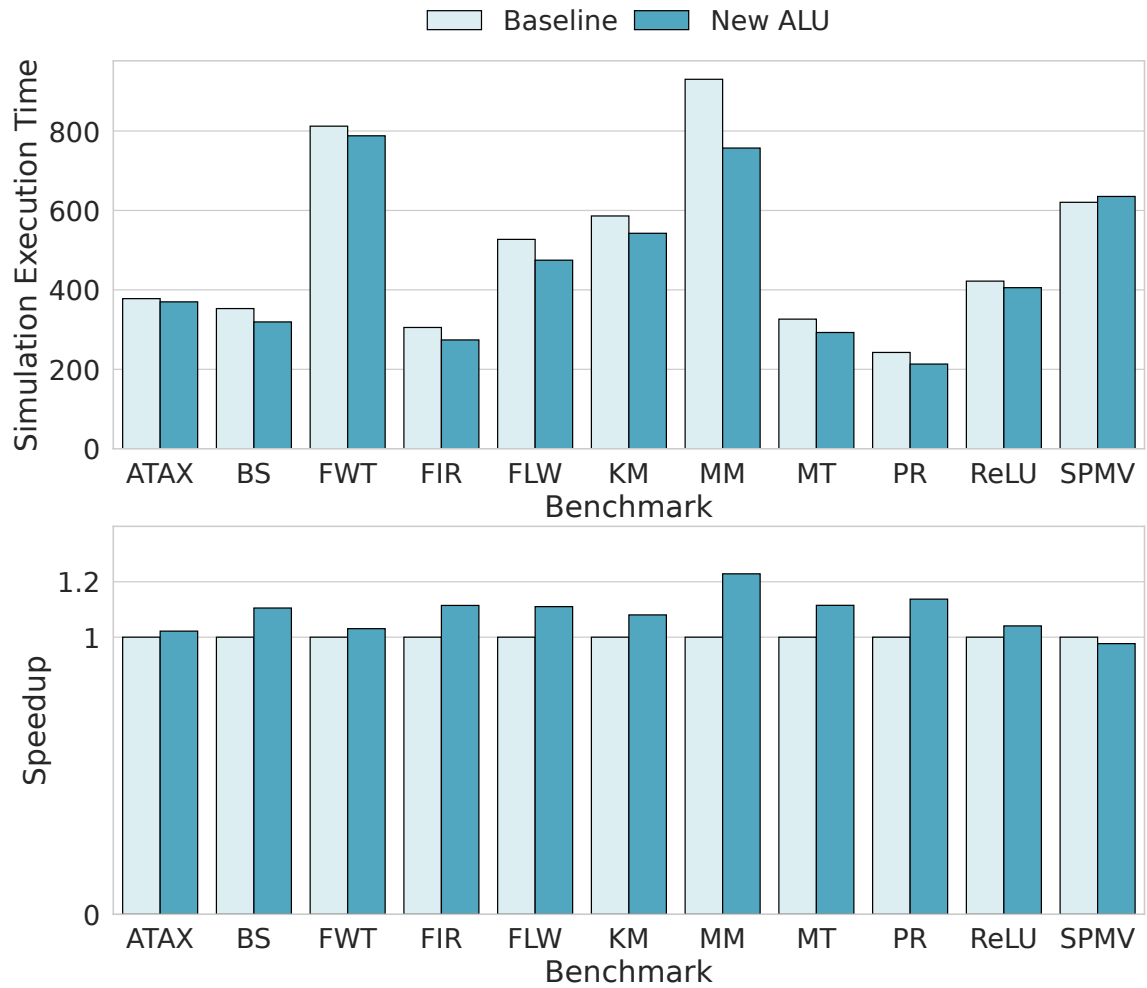


Figure 6.11: Timing performance comparison between the original ALU and the new ALU implementations.

Parallel Scalability Issue To mitigate the synchronization blocking problem, we examined the logic used for locking that protects shared data structures in the `createOrGetStorageUnit` function and other related functions. We are looking for opportunities to relax some synchronization requirements, while maintaining correctness. The shared data structure is `Storage`, which contains its addresses and corresponding `StorageUnit`. We find that every function call to `createOrGetStorageUnit` is serialized, because most of the function code is treated as a critical section and is protected by a mutual exclusion lock. However, this strict form of locking is not necessary when multiple goroutines are reading the shared data structure simultaneously.

Since function `createOrGetStorageUnit` includes both cases of reading and writing, which achieves the initialization of the `Storage`, using a mutual exclusion lock is necessary to ensure correctness. To relax this synchronization requirement, we modified the implementation to have a function named `createStorageUnit` for initialization separately, and it is guaranteed to be called by a single goroutine. We have another function named `getStorageUnit` to replace the `createOrGetStorageUnit` and apply a reader/writer mutual exclusion lock. The lock can be held by an arbitrary number of readers or a single writer. We take the same approach to address similar issues in `processTable` and `pageTableImpl` (see Figure 6.4).

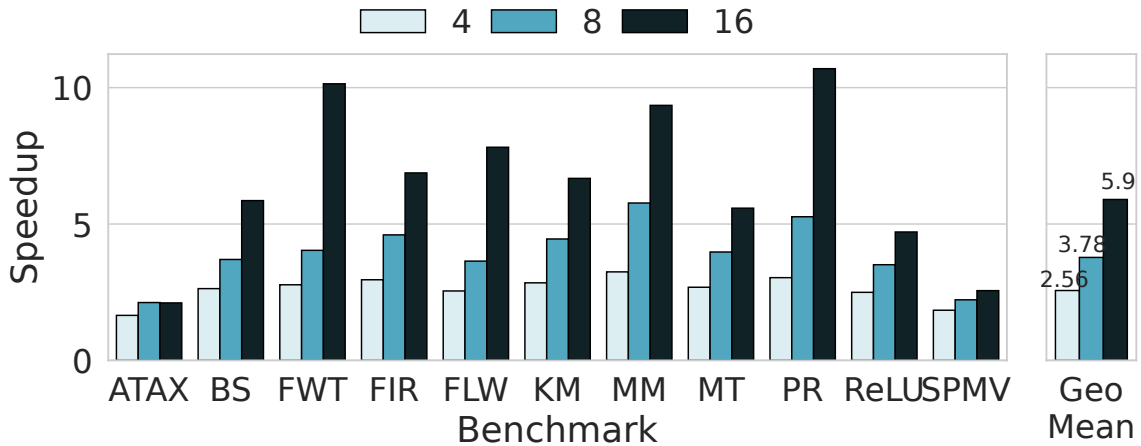


Figure 6.12: The speedup achieved through parallel simulation when running emulation mode on platform1.

Figure 6.12 shows the improved speedup achieved through parallel simulation when running emulation mode. We normalize the simulation execution time of running with 4, 8, and 16 cores against the time for a single core. The average speedups for 8 and 16 cores are 3.78 and

5.9, respectively. Compared to the original performance results (see Figure 6.2), we can see a clear improvement in the scalability issue. Figure 6.12 shows the improvement when running 16 cores gives the best performance, while in the original case, increasing to 16 cores did not improve scalability as compared to 8 cores. Although the speedup we achieve does not scale linearly with the number of cores, we are able to improve the parallel simulation time and achieved an average speedup of $5.9\times$, as well as a maximum speedup of $10.65\times$ when running 16 cores.

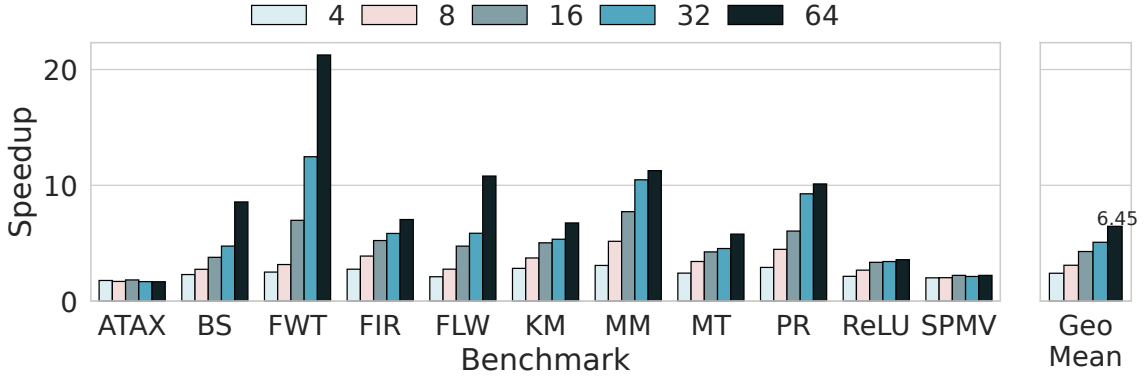


Figure 6.13: The speedup achieved through parallel simulation when running emulation mode on platform2.

We evaluate our optimizations on a different machine with 64 cores (see Figure 6.13). The average speedups for 32 and 64 cores are 5.1 and 6.45, respectively. Except for ATAX, ReLU and SPMV, all benchmarks exhibit an improvement in speed as we increase the number of cores. With 64 cores, we achieved an average speedup of $6.45\times$ and a maximum speedup of $20.26\times$.

6.4 Summary

Throughout our evaluation and performance improvement results, we elaborate on various important design decisions that could lead to significant simulation overhead and impact the parallel simulation scalability. Using NaviSim as an example, we apply our methodology for designing high-performance simulators to characterize some general simulator design decisions and identify key factors that restrict parallel simulation scalability. Simulator developers can follow our methodology when designing a high-performance simulator to minimize unnecessary simulation overhead and improve parallel simulation efficiency. We provide a detailed methodology, describing the steps to design and optimize a simulator (See Table 6.4). This is an iterative process. After following

CHAPTER 6. SIMULATION PERFORMANCE EVALUATION AND OPTIMIZATION

Table 6.4: Steps to tune the performance of a simulator

1	Evaluate the simulation execution of the simulator in the mode (e.g., emulation, timing, etc.) you plan to optimize and collect an appropriate set of performance metrics that should include execution time, Slowdown, KIPS and CIPGI as the baseline measurement set.
2	Profile the simulation execution with a performance profiling tool that can identify hotspots where the simulation spends the most time.
3	Identify the reason why the simulation spends the most time in any one function or component. • Collect event information (i.e., function/component name, the virtual time in the simulator, the execution time) for all events or events related to the bottleneck. Inspect them in time order. Focus on the earliest time-consuming event. • Revisit the modeling logic, looking for opportunities to raise the abstraction level, while maintain simulation accuracy. • Examine implementation details (i.e., data structures and built-in/library functions) by evaluating the time complexity.
4	Redesign or modify the related parts in the simulator to mitigate the cause of the bottleneck.
5	Rerun the simulation and collect the performance metrics. Compare the results with the evaluation results from Step 1 to test the impact of the changes made in Step 4.
Parallel Simulation Tuning	
1	Evaluate the parallel simulation scalability of the simulator when leveraging multiple threads. Collect the execution time when modifying the number of threads. Compare the results and analyze speedup gained through parallelism.
2	Profile the parallel simulation execution with a performance profiling tool that can identify blocked threads due to synchronization operations (i.e., locks or semaphores). Try to utilized advanced tools that can show a trace of each thread's execution over some time.
3	Review the event information at various timestamps in the simulator to ensure there are a large enough number of events to fully exploit parallel execution. It is essential to understand how the parallel simulation engine manages event scheduling and how concurrent events are handled. Make sure threads are using synchronization properly to ensure the correct order of operation.
4	Inspect accesses to shared resources performed by each thread, identifying resources that are accessed heavily during simulation.
5	Relax the synchronization requirements if possible. If we cannot relax these requirements, then consider rearchitecting the management of shared resources in a distributed manner.
6	Reevaluate the parallel simulation scalability of the simulator to test the impact of the changes made in Step 5.

CHAPTER 6. SIMULATION PERFORMANCE EVALUATION AND OPTIMIZATION

these steps, developers can revisit profiling results and continue refining the simulator based on the observed behavior. We demonstrate a thorough performance analysis of NaviSim by profiling both emulation and timing simulation execution running in serial and parallel. Based on the profiling results, we focused on improving emulation performance by proposing new models for the Workgroup Dispatcher and ALU. We also mitigated the synchronization blocking bottlenecks, improving the parallel emulation scalability.

Chapter 7

Conclusion and Future Work

7.1 Conclusion

This dissertation has focused on addressing the challenges involved when designing an architectural simulation framework that features high adaptability and high performance. We have developed a novel design methodology for producing a highly adaptable simulator. We have introduced a methodology for examining the key simulator design decisions that impact the simulation performance. We presented NaviSim, a AMD GPU simulator that was designed, implemented, and validated, following our principles. NaviSim is able to accommodate new features and design extensions, allowing the user to explore a wide range of GPU architecture questions. We perform performance analysis and tuning on NaviSim, following our methodology of designing high-performance simulators. We summarize the contributions of this dissertation as follows:

- We present the design goals and development methodology for building a highly adaptable and accurate GPU simulator. To build a GPU simulator that can support multiple ISAs and microarchitectures, and be easily extended to support future incoming architectures, we demonstrate our approach by presenting NaviSim [21]. NaviSim models the state-of-the-art AMD RDNA and CDNA architectures, providing both high modularity and extensibility. Through our intensive and rigorous validation methodology, which included developing several microbenchmarks, NaviSim achieves a small 9.85% average simulation error, as compared to hardware execution. We followed our simulator development methodology. Our approach can be generalized to design any architectural simulator, producing a toolset designed with high modularity, extensibility, and high fidelity. We conducted a performance study of the impact

CHAPTER 7. CONCLUSION AND FUTURE WORK

of individual architecture features of AMD RDNA architecture, showcasing the high flexibility and configurability of NaviSim.

- We present a series of GPU-based microarchitectural extensions (GME) [69] as a use case of design space exploration using NaviSim. We demonstrate the high degree of flexibility and adaptability of NaviSim. First, NaviSim serves as a performance analysis tool by identifying the microarchitecture-level bottlenecks. Next, we demonstrate the high flexibility of NaviSim, which enables users to quickly explore the design space of new architectural features of future GPUs. Users can easily extend NaviSim with new components (e.g., a new on-chip network and a new modular operations unit), and connect them to the existing components without modifying the rest of the simulator. We were able to marry NaviSim to BlockSim, allowing us to perform hybrid simulation. This use case demonstrates the high configurability of NaviSim. Architectural parameters, such as the size of LDS and register files, can be changed effortlessly. Last but not least, NaviSim, as an execution-driven simulator, can support the decode and execution of newly designed instructions, which enables the full support of GPU architecture design, including both ISA and microarchitecture.
- We present a methodology for examining the key simulator design decisions that simulator developers need to consider to minimize unnecessary simulation overhead and improve parallel simulation efficiency. We introduce a set of performance metrics for the architectural simulation community, so that they can utilize standard metrics when evaluating the execution performance of any architectural simulator. Using NaviSim as an example, we applied our methodology to the design of our high-performance simulator. We were able to provide guidance on simulator design decisions and identify key factors that impact parallel simulation scalability. Our methodology can be generalized to shape the design process of any architectural simulator in order to achieve high-performance. Next, we conducted a thorough performance analysis of NaviSim to find bottlenecks and performance issues. We built a process that leverages the execution profile of NaviSim to help guide a set of software-level optimizations to improve simulation speed.

7.2 Future Work

The main goal of this dissertation was to propose a design methodology for producing highly-adaptable and high-performance simulation frameworks. We leveraged the Akita simulator

CHAPTER 7. CONCLUSION AND FUTURE WORK

framework to enable modular and extensible designs as well as parallel simulation to achieve high performance. Akita has a rich set of features, one of which is the support of parallel simulation. Akita employs a parallel simulation engine to achieve parallel simulation. Through our evaluation of running NaviSim on multiple cores and further investigation of simulation scalability, we found that Akita has known limitations in its parallel simulation support. One fruitful direction for future work would be to improve the parallel simulation support of Akita.

A key limitation in Akita scalability is rooted in its port-connection-message system, which is one of Akita's core concepts to enable the communication between components, while keeping each component fully encapsulated. This type of universal communication system of connecting all components leads to significant synchronization overhead, as different messages interfere with each other by sharing the same connection. We would need to redesign the communication system to reduce the synchronization overhead. To achieve better parallel simulation scalability, further opportunities can be exploited by redesigning the parallel simulation engine to have a more aggressive way of managing event scheduling and utilization of multiple threads.

Bibliography

- [1] L.-N. Pouchet *et al.*, “Polybench: The polyhedral benchmark suite,” 2012.
- [2] A. Inc., “Amd graphics core next architecture,” url <https://www.techpowerup.com/gpu-specs/docs/amd-gcn1-architecture.pdf>, 2012.
- [3] AMD Inc., “Introducing rdna architecture, the all new radeon gaming architecture powering “navi”,” 2019. [Online]. Available: <https://www.amd.com/system/files/documents/rdna-whitepaper.pdf>
- [4] Y. Lecun, L. Bottou, Y. Bengio, and P. Haffner, “Gradient-based learning applied to document recognition,” *Proceedings of the IEEE*, vol. 86, no. 11, pp. 2278–2324, 1998.
- [5] W. Liu, Z. Wang, X. Liu, N. Zeng, Y. Liu, and F. E. Alsaadi, “A survey of deep neural network architectures and their applications,” *Neurocomputing*, vol. 234, pp. 11–26, 2017. [Online]. Available: <https://www.sciencedirect.com/science/article/pii/S0925231216315533>
- [6] Top500, <https://www.top500.org/lists/top500/>.
- [7] NVIDIA, “Compute unified device architecture programming guide,” 2007.
- [8] A. Inc., “Hip programming guide,” https://rocmdocs.amd.com/en/latest/Programming_Guides/HIP-GUIDE.html, 2022.
- [9] Y. Sun, T. Baruah, and D. Kaeli, “Accelerated computing with hip,” 2023.
- [10] D. R. Kaeli, P. Mistry, D. Schaa, and D. P. Zhang, *Heterogeneous computing with OpenCL 2.0*. Burlington,MA,USA: Morgan Kaufmann, 2015.
- [11] T. Nowatzki, J. Menon, C.-H. Ho, and K. Sankaralingam, “Architectural simulators considered harmful,” *IEEE Micro*, vol. 35, no. 6, pp. 4–12, 2015.

BIBLIOGRAPHY

- [12] A. Inc., “Introducing cdna architecture, the all-new AMD GPU architecture for the modern era of hpc & ai,” <https://www.amd.com/system/files/documents/amd-cdna-whitepaper.pdf>, 2020.
- [13] J. H. Cheon, A. Kim, M. Kim, and Y. Song, “Homomorphic encryption for arithmetic of approximate numbers,” in *Advances in Cryptology—ASIACRYPT 2017: 23rd International Conference on the Theory and Applications of Cryptology and Information Security, Hong Kong, China, December 3-7, 2017, Proceedings, Part I 23*. Springer, 2017, pp. 409–437.
- [14] I. Chillotti, N. Gama, M. Georgieva, and M. Izabachène, “Tfhe: Fast fully homomorphic encryption over the torus,” *Journal of Cryptology*, vol. 33, 04 2019.
- [15] W. Jung, E. Lee, S. Kim, J. Kim, N. Kim, K. Lee, C. Min, J. H. Cheon, and J. H. Ahn, “Accelerating fully homomorphic encryption through architecture-centric analysis and optimization,” *IEEE Access*, vol. 9, pp. 98 772–98 789, 2021.
- [16] K. Shivdikar, G. Jonatan, E. Mora, N. Livesay, R. Agrawal, A. Joshi, J. L. Abellán, J. Kim, and D. Kaeli, “Accelerating polynomial multiplication for homomorphic encryption on GPUs,” in *2022 IEEE International Symposium on Secure and Private Execution Environment Design (SEED)*. IEEE, 2022, pp. 61–72.
- [17] N. Livesay, G. Jonatan, E. Mora, K. Shivdikar, R. Agrawal, A. Joshi, J. L. Abellán, J. Kim, and D. Kaeli, “Accelerating finite field arithmetic for homomorphic encryption on GPUs,” *2023 IEEE MICRO*, 2023.
- [18] A. Al Badawi, B. Veeravalli, J. Lin, N. Xiao, M. Kazuaki, and A. K. M. Mi, “Multi-GPU design and performance evaluation of homomorphic encryption on GPU clusters,” *IEEE Transactions on Parallel and Distributed Systems*, vol. 32, no. 2, pp. 379–391, 2020.
- [19] S. Williams, A. Waterman, and D. Patterson, “Roofline: An insightful visual performance model for multicore architectures,” *Commun. ACM*, vol. 52, no. 4, p. 65–76, apr 2009. [Online]. Available: <https://doi.org/10.1145/1498765.1498785>
- [20] Y. Sun, T. Baruah, S. A. Mojumder, S. Dong, X. Gong, S. Treadway, Y. Bao, S. Hance, C. McCardwell, V. Zhao, H. Barclay, A. K. Ziabari, Z. Chen, R. Ubal, J. L. Abellán, J. Kim, A. Joshi, and D. Kaeli, “Mgpusim: Enabling multi-gpu performance modeling and optimization,” in *Proceedings of the 46th International Symposium on Computer Architecture*,

BIBLIOGRAPHY

- ser. ISCA '19. New York, NY, USA: Association for Computing Machinery, 2019, p. 197–209. [Online]. Available: <https://doi.org/10.1145/3307650.3322230>
- [21] Y. Bao, Y. Sun, Z. Feric, M. T. Shen, M. Weston, J. L. Abellán, T. Baruah, J. Kim, A. Joshi, and D. Kaeli, “Navisim: A highly accurate gpu simulator for amd rdna gpus,” in *Proceedings of the International Conference on Parallel Architectures and Compilation Techniques*, ser. PACT '22. New York, NY, USA: Association for Computing Machinery, 2023, p. 333–345. [Online]. Available: <https://doi.org/10.1145/3559009.3569666>
- [22] J. J. Y. Timothy Sherwood, “Computer architecture simulation and modeling,” *IEEE Computer Society*, vol. 26, pp. 5–7, July/August 2006.
- [23] D. Sanchez and C. Kozyrakis, “Zsim: Fast and accurate microarchitectural simulation of thousand-core systems,” in *Proceedings of the 40th Annual International Symposium on Computer Architecture*, ser. ISCA '13. New York, NY, USA: Association for Computing Machinery, 2013, p. 475–486. [Online]. Available: <https://doi.org/10.1145/2485922.2485963>
- [24] A. Mohammad, U. Darbaz, G. Dozsa, S. Diestelhorst, D. Kim, and N. S. Kim, “dist-gem5: Distributed simulation of computer clusters,” in *2017 IEEE International Symposium on Performance Analysis of Systems and Software (ISPASS)*, 2017, pp. 153–162.
- [25] R. Wunderlich, T. Wenisch, B. Falsafi, and J. Hoe, “Smarts: accelerating microarchitecture simulation via rigorous statistical sampling,” in *30th Annual International Symposium on Computer Architecture, 2003. Proceedings.*, 2003, pp. 84–95.
- [26] C. Avalos Baddouh, M. Khairy, R. N. Green, M. Payer, and T. G. Rogers, “Principal kernel analysis: A tractable methodology to simulate scaled gpu workloads,” in *MICRO-54: 54th Annual IEEE/ACM International Symposium on Microarchitecture*, ser. MICRO '21. New York, NY, USA: Association for Computing Machinery, 2021, p. 724–737. [Online]. Available: <https://doi.org/10.1145/3466752.3480100>
- [27] C. Liu, Y. Sun, and T. E. Carlson, “Photon: A fine-grained sampled simulation methodology for gpu workloads,” in *Proceedings of the 56th Annual IEEE/ACM International Symposium on Microarchitecture*, ser. MICRO '23. New York, NY, USA: Association for Computing Machinery, 2023, p. 1227–1241. [Online]. Available: <https://doi.org/10.1145/3613424.3623773>

BIBLIOGRAPHY

- [28] O. Villa, D. Lustig, Z. Yan, E. Bolotin, Y. Fu, N. Chatterjee, N. Jiang, and D. Nellans, “Need for speed: Experiences building a trustworthy system-level gpu simulator,” in *2021 IEEE International Symposium on High-Performance Computer Architecture (HPCA)*, IEEE. Seoul, Korea (South): IEEE, 2021, pp. 868–880.
- [29] M. Auguin, F. Boeri, J. Dalban, and A. Vincent-Carrefour, “Experience using a simd/spmd multiprocessor architecture,” *Microprocessing and Microprogramming*, vol. 21, no. 1, pp. 171–177, 1987, microcomputers: Usage, Methods and Structures. [Online]. Available: <https://www.sciencedirect.com/science/article/pii/0165607487900342>
- [30] J. Kim and Y. Kim, “Hbm: Memory solution for bandwidth-hungry processors,” in *2014 IEEE Hot Chips 26 Symposium (HCS)*, IEEE. Cupertino, CA: IEEE, 2014, pp. 1–24.
- [31] J. JESD250, “Graphics double data rate 6 (gddr6) sgram standard,” JEDEC Solid State Technology Association, 2017.
- [32] A. Akram and L. Sawalha, “A survey of computer architecture simulation techniques and tools,” *IEEE Access*, vol. 7, pp. 78 120–78 145, 2019.
- [33] J. Banks, J. S. Carson, B. L. Nelson, and D. M. Nicol, *Discrete-Event System Simulation*. Prentice Hall Publishing, 1995. [Online]. Available: <https://api.semanticscholar.org/CorpusID:122566976>
- [34] R. M. Fujimoto, “Parallel discrete event simulation,” *Commun. ACM*, vol. 33, no. 10, p. 30–53, oct 1990. [Online]. Available: <https://doi.org/10.1145/84537.84545>
- [35] C. Dufour, C. Andrade, and J. Bélanger, “Real-time simulation technologies in education: a link to modern engineering methods and practices,” in *Proceedings of the 11th International Conference on Engineering and Technology Education INTERTECH*, 2010.
- [36] T. Gleixner and I. Molnar, “perf,” 2008. [Online]. Available: <https://github.com/torvalds/linux>
- [37] S. L. Graham, P. B. Kessler, and M. K. Mckusick, “Gprof: A call graph execution profiler,” in *Proceedings of the 1982 SIGPLAN Symposium on Compiler Construction*, ser. SIGPLAN ’82. New York, NY, USA: Association for Computing Machinery, 1982, p. 120–126. [Online]. Available: <https://doi.org/10.1145/800230.806987>
- [38] Microsoft, “Windows performance toolkit,” 2022. [Online]. Available: <https://github.com/microsoft/microsoft-performance-toolkit-sdk>

BIBLIOGRAPHY

- [39] Google, “pprof.” [Online]. Available: <https://github.com/google/pprof>
- [40] I. C. Laboratory, “Papi,” 2009. [Online]. Available: <https://icl.utk.edu/papi/>
- [41] P. Paillier, “Public-key cryptosystems based on composite degree residuosity classes,” in *Advances in Cryptology — EUROCRYPT ’99*, J. Stern, Ed. Berlin, Heidelberg: Springer Berlin Heidelberg, 1999, pp. 223–238.
- [42] R. L. Rivest, A. Shamir, and L. Adleman, “A method for obtaining digital signatures and public-key cryptosystems,” *Commun. ACM*, vol. 21, no. 2, p. 120–126, feb 1978. [Online]. Available: <https://doi.org/10.1145/359340.359342>
- [43] C. Gentry, “Fully homomorphic encryption using ideal lattices,” in *Proceedings of the forty-first annual ACM symposium on Theory of computing*, 2009, pp. 169–178.
- [44] D. Micciancio and O. Regev, “Lattice-based cryptography,” *Post-quantum cryptography*, pp. 147–191, 2009.
- [45] V. Shoup, *A computational introduction to number theory and algebra*. Cambridge University Press, 2009.
- [46] A. J. Rashmi Agrawal, *On Architecting Fully Homomorphic Encryption-based Computing Systems*. Springer Cham, 2023. [Online]. Available: <https://doi.org/10.1007/978-3-031-31754-5>
- [47] Y. Fu, E. Bolotin, N. Chatterjee, D. Nellans, and S. W. Keckler, “Gpu domain specialization via composable on-package architecture,” *ACM Trans. Archit. Code Optim.*, vol. 19, no. 1, dec 2021. [Online]. Available: <https://doi.org/10.1145/3484505>
- [48] N. Binkert, B. Beckmann, G. Black, S. K. Reinhardt, A. Saidi, A. Basu, J. Hestness, D. R. Hower, T. Krishna, S. Sardashti, R. Sen, K. Sewell, M. Shoaib, N. Vaish, M. D. Hill, and D. A. Wood, “The gem5 simulator,” *SIGARCH Comput. Archit. News*, vol. 39, no. 2, p. 1–7, aug 2011. [Online]. Available: <https://doi.org/10.1145/2024716.2024718>
- [49] A. Gutierrez, B. M. Beckmann, A. Dutu, J. Gross, M. LeBeane, J. Kalamatianos, O. Kayiran, M. Poremba, B. Potter, S. Puthoor *et al.*, “Lost in abstraction: Pitfalls of analyzing gpus at the intermediate language level,” in *2018 IEEE International Symposium on High Performance Computer Architecture (HPCA)*, IEEE. Vienna, Austria: IEEE, 2018, pp. 608–619.

BIBLIOGRAPHY

- [50] C. Collange, M. Daumas, D. Defour, and D. Parello, “Barra: A parallel functional simulator for gpgpu,” in *2010 IEEE International Symposium on Modeling, Analysis and Simulation of Computer and Telecommunication Systems*. Miami, Florida, USA: IEEE, 2010, pp. 351–360.
- [51] N. Farooqui, A. Kerr, G. Damos, S. Yalamanchili, and K. Schwan, “A framework for dynamically instrumenting gpu compute applications within gpu ocelot,” in *Proceedings of the Fourth Workshop on General Purpose Processing on Graphics Processing Units*. Newport Beach, CA: ACM, 2011, pp. 1–9.
- [52] A. Kerr, G. Damos, and S. Yalamanchili, “Gpu application development, debugging, and performance tuning with gpu ocelot,” in *GPU Computing Gems Jade Edition*. Amsterdam, Netherlands: Elsevier, 2012, pp. 409–427.
- [53] A. Bakhoda, G. L. Yuan, W. W. Fung, H. Wong, and T. M. Aamodt, “Analyzing cuda workloads using a detailed gpu simulator,” in *2009 IEEE International Symposium on Performance Analysis of Systems and Software*, IEEE. Boston, MA USA: IEEE, 2009, pp. 163–174.
- [54] H. Wang, F. Luo, M. Ibrahim, O. Kayiran, and A. Jog, “Efficient and fair multi-programming in gpus via effective bandwidth management,” in *2018 IEEE International Symposium on High Performance Computer Architecture (HPCA)*, IEEE. Vienna, Austria: IEEE, 2018, pp. 247–258.
- [55] R. Ausavarungnirun, J. Landgraf, V. Miller, S. Ghose, J. Gandhi, C. J. Rossbach, and O. Mutlu, “Mosaic: a gpu memory manager with application-transparent support for multiple page sizes,” in *Proceedings of the 50th Annual IEEE/ACM International Symposium on Microarchitecture*. Cambridge, MA, USA: IEEE, 2017, pp. 136–150.
- [56] M. Khairy, Z. Shen, T. M. Aamodt, and T. G. Rogers, “Accel-sim: An extensible simulation framework for validated gpu modeling,” in *2020 ACM/IEEE 47th Annual International Symposium on Computer Architecture (ISCA)*, IEEE. Valencia, Spain: IEEE, 2020, pp. 473–486.
- [57] O. Villa, M. Stephenson, D. Nellans, and S. W. Keckler, “Nvbit: A dynamic binary instrumentation framework for nvidia gpus,” in *Proceedings of the 52nd Annual IEEE/ACM International Symposium on Microarchitecture*, ser. MICRO ’52. New York, NY, USA: Association for Computing Machinery, 2019, p. 372–383. [Online]. Available: <https://doi.org/10.1145/3352460.3358307>

BIBLIOGRAPHY

- [58] R. Ubal, B. Jang, P. Mistry, D. Schaa, and D. Kaeli, “Multi2sim: A simulation framework for cpu-gpu computing,” in *Proceedings of the 21st International Conference on Parallel Architectures and Compilation Techniques*, ser. PACT ’12. New York, NY, USA: Association for Computing Machinery, 2012, p. 335–344. [Online]. Available: <https://doi.org/10.1145/2370816.2370865>
- [59] X. Gong, R. Ubal, and D. Kaeli, “Multi2sim kepler: A detailed architectural gpu simulator,” in *2017 IEEE International Symposium on Performance Analysis of Systems and Software (ISPASS)*, IEEE. Santa Rosa, CA: IEEE, 2017, pp. 269–278.
- [60] H. Kim, J. Lee, N. B. Lakshminarayana, J. Sim, J. Lim, and T. Pho, *Macsim: A cpu-gpu heterogeneous simulation framework user guide*, Georgia Institute of Technology, Atlanta, GA, 2012.
- [61] G. Malhotra, S. Goel, and S. R. Sarangi, “Gputejas: A parallel simulator for gpu architectures,” in *2014 21st International Conference on High Performance Computing (HiPC)*, 2014, pp. 1–10.
- [62] S. Lee and W. W. Ro, “Parallel gpu architecture simulation framework exploiting work allocation unit parallelism,” in *2013 IEEE International Symposium on Performance Analysis of Systems and Software (ISPASS)*, 2013, pp. 107–117.
- [63] —, “Parallel gpu architecture simulation framework exploiting architectural-level parallelism with timing error prediction,” *IEEE Transactions on Computers*, vol. 65, no. 4, pp. 1253–1265, 2016.
- [64] R. Murphy, A. F. Rodrigues, P. Kogge, and K. D. Underwood, “The structural simulation toolkit : A tool for bridging the architectural/microarchitectural evaluation gap.” 12 2004. [Online]. Available: <https://www.osti.gov/biblio/1088092>
- [65] C. Hughes, S. D. Hammond, M. Zhang, Y. Liu, T. Rogers, and R. J. Hoekstra, “Sst-gpu: A scalable sst gpu component for performance modeling and profiling.” Sandia National Lab.(SNL-NM), Albuquerque, NM (United States), Tech. Rep., 2021.
- [66] E. C. Marangoz, K.-D. Kang, and S. Shin, “Designing gpu architecture for memory bandwidth reservation,” in *2021 IEEE International Symposium on Performance Analysis of Systems and Software (ISPASS)*, 2021, pp. 87–89.

BIBLIOGRAPHY

- [67] A. Arunkumar, E. Bolotin, B. Cho, U. Milic, E. Ebrahimi, O. Villa, A. Jaleel, C.-J. Wu, and D. Nellans, “Mcm-gpu: Multi-chip-module gpus for continued performance scalability,” in *2017 ACM/IEEE 44th Annual International Symposium on Computer Architecture (ISCA)*, 2017, pp. 320–332.
- [68] G.-J. V. D. Braak and H. Corporaal, “R-gpu: A reconfigurable gpu architecture,” *ACM Trans. Archit. Code Optim.*, vol. 13, no. 1, mar 2016. [Online]. Available: <https://doi.org/10.1145/2890506>
- [69] K. Shivdikar, Y. Bao, R. Agrawal, M. Shen, G. Jonatan, E. Mora, A. Ingare, N. Livesay, J. L. AbellÁN, J. Kim, A. Joshi, and D. Kaeli, “Gme: Gpu-based microarchitectural extensions to accelerate homomorphic encryption,” in *Proceedings of the 56th Annual IEEE/ACM International Symposium on Microarchitecture*, ser. MICRO '23. New York, NY, USA: Association for Computing Machinery, 2023, p. 670–684. [Online]. Available: <https://doi.org/10.1145/3613424.3614279>
- [70] A. Al Badawi, L. Hoang, C. F. Mun, K. Laine, and K. M. M. Aung, “Privft: Private and fast text classification with homomorphic encryption,” *IEEE Access*, vol. 8, pp. 226 544–226 556, 2020.
- [71] A. Al Badawi, B. Veeravalli, C. F. Mun, and K. M. M. Aung, “High-performance fv somewhat homomorphic encryption on GPUs: An implementation using cuda,” *IACR Transactions on Cryptographic Hardware and Embedded Systems*, pp. 70–95, 2018.
- [72] W. Jung, S. Kim, J. H. Ahn, J. H. Cheon, and Y. Lee, “Over 100x faster bootstrapping in fully homomorphic encryption through memory-centric optimization with GPUs,” *IACR Transactions on Cryptographic Hardware and Embedded Systems*, pp. 114–148, 2021.
- [73] M. S. Riazi, K. Laine, B. Pelton, and W. Dai, “Heax: An architecture for computing on encrypted data,” in *Proceedings of the Twenty-Fifth International Conference on Architectural Support for Programming Languages and Operating Systems*, 2020, pp. 1295–1309.
- [74] S. Kim, K. Lee, W. Cho, Y. Nam, J. H. Cheon, and R. A. Rutenbar, “Hardware architecture of a number theoretic transform for a bootstrappable RNS-based homomorphic encryption scheme,” in *2020 IEEE 28th Annual International Symposium on Field-Programmable Custom Computing Machines (FCCM)*. IEEE, 2020, pp. 56–64.

BIBLIOGRAPHY

- [75] R. Agrawal, L. de Castro, G. Yang, C. Juvekar, R. Yazicigil, A. Chandrakasan, V. Vaikuntanathan, and A. Joshi, “Fab: An fpga-based accelerator for bootstrappable fully homomorphic encryption,” in *2023 IEEE International Symposium on High-Performance Computer Architecture (HPCA)*. IEEE, 2023, pp. 882–895.
- [76] N. Samardzic, A. Feldmann, A. Krastev, S. Devadas, R. Dreslinski, C. Peikert, and D. Sanchez, “F1: A fast and programmable accelerator for fully homomorphic encryption,” in *MICRO-54: 54th Annual IEEE/ACM International Symposium on Microarchitecture*, 2021, pp. 238–252.
- [77] S. Kim, J. Kim, M. J. Kim, W. Jung, J. Kim, M. Rhu, and J. H. Ahn, “BTS: An accelerator for bootstrappable fully homomorphic encryption,” in *Proceedings of the 49th Annual International Symposium on Computer Architecture*, 2022, pp. 711–725.
- [78] N. Samardzic, A. Feldmann, A. Krastev, N. Manohar, N. Genise, S. Devadas, K. Eldefrawy, C. Peikert, and D. Sanchez, “Craterlake: a hardware accelerator for efficient unbounded computation on encrypted data,” in *Proceedings of the 49th Annual International Symposium on Computer Architecture*, 2022, pp. 173–187.
- [79] J. Kim, G. Lee, S. Kim, G. Sohn, M. Rhu, J. Kim, and J. H. Ahn, “Ark: Fully homomorphic encryption accelerator with runtime data generation and inter-operation key reuse,” in *2022 55th IEEE/ACM International Symposium on Microarchitecture (MICRO)*. IEEE, 2022, pp. 1237–1254.
- [80] R. C. Martin, *Agile Software Development: Principles, Patterns, and Practices*. USA: Prentice Hall PTR, 2003.
- [81] Y. Sun, *Enabling Collaborative Heterogeneous Computing*. Northeastern University, 2020. [Online]. Available: <https://books.google.com/books?id=dMeqzgEACAAJ>
- [82] V. Volkov and J. W. Demmel, “Benchmarking gpus to tune dense linear algebra,” in *SC’08: Proceedings of the 2008 ACM/IEEE conference on Supercomputing*, IEEE. Austin, TX, USA: IEEE, 2008, pp. 1–11.
- [83] T.-B. Chiou, A. C. Chen, M. Dusa, and S.-E. Tseng, “Impact of euv patterning scenario on different design styles and their ground rules for 7nm/5nm node beol layers,” in *Design-Process-Technology Co-optimization for Manufacturability X*, vol. 9781, International Society for Optics and Photonics. Bellingham, Washington USA: SPIE, 2016, p. 978107.

BIBLIOGRAPHY

- [84] Y. Sun, S. Mukherjee, T. Baruah, S. Dong, J. Gutierrez, P. Mohan, and D. Kaeli, “Evaluating performance tradeoffs on the radeon open compute platform,” in *2018 IEEE International Symposium on Performance Analysis of Systems and Software (ISPASS)*. Los Alamitos, CA, USA: IEEE Computer Society, apr 2018, pp. 209–218. [Online]. Available: <https://doi.ieeecomputersociety.org/10.1109/ISPASS.2018.00034>
- [85] J. J. K. Park, Y. Park, and S. Mahlke, “Dynamic resource management for efficient utilization of multitasking gpus,” in *Proceedings of the Twenty-Second International Conference on Architectural Support for Programming Languages and Operating Systems*, ser. ASPLOS ’17. New York, NY, USA: Association for Computing Machinery, 2017, p. 527–540. [Online]. Available: <https://doi.org/10.1145/3037697.3037707>
- [86] L. Wang, M. Huang, and T. El-Ghazawi, “Exploiting concurrent kernel execution on graphic processing units,” in *2011 International Conference on High Performance Computing & Simulation*, IEEE. Istanbul, Turkey: IEEE, 2011, pp. 24–32.
- [87] S. Li, Z. Yang, D. Reddy, A. Srivastava, and B. Jacob, “Dramsim3: a cycle-accurate, thermal-capable dram simulator,” *IEEE Computer Architecture Letters*, vol. 19, no. 2, pp. 106–109, 2020.
- [88] A. Staff, “Opencl and the amd app sdk v2. 4,” 2014.
- [89] A. Danalis, G. Marin, C. McCurdy, J. S. Meredith, P. C. Roth, K. Spafford, V. Tipparaju, and J. S. Vetter, “The scalable heterogeneous computing (shoc) benchmark suite,” in *Proceedings of the 3rd Workshop on General-Purpose Computation on Graphics Processing Units*, ser. GPGPU-3. New York, NY, USA: Association for Computing Machinery, 2010, p. 63–74. [Online]. Available: <https://doi.org/10.1145/1735688.1735702>
- [90] Y. Sun, X. Gong, A. Ziabari, L. Yu, X. Li, S. Mukherjee, C. Mccardwell, A. Villegas, and D. Kaeli, “Hetero-mark, a benchmark suite for cpu-gpu collaborative computing,” in *2016 IEEE International Symposium on Workload Characterization (IISWC)*. Los Alamitos, CA, USA: IEEE Computer Society, sep 2016, pp. 1–10. [Online]. Available: <https://doi.ieeecomputersociety.org/10.1109/IISWC.2016.7581262>
- [91] S. Dong and D. Kaeli, “Dnnmark: A deep neural network benchmark suite for gpus,” in *Proceedings of the General Purpose GPUs*, ser. GPGPU-10. New York,

BIBLIOGRAPHY

- NY, USA: Association for Computing Machinery, 2017, p. 63–72. [Online]. Available: <https://doi.org/10.1145/3038228.3038239>
- [92] H. A. Van der Vorst, “Bi-cgstab: A fast and smoothly converging variant of bi-cg for the solution of nonsymmetric linear systems,” *SIAM Journal on scientific and Statistical Computing*, vol. 13, no. 2, pp. 631–644, 1992.
- [93] Y. Sun, Y. Zhang, A. Mosallaei, M. D. Shah, C. Dunne, and D. Kaeli, “Daisen: A framework for visualizing detailed GPU execution,” *Eurographics Conference on Visualization*, vol. 40, no. 3, pp. 239–250, 2021.
- [94] N. E. Jerger, T. Krishna, and L.-S. Peh, “On-chip networks,” *Synthesis Lectures on Computer Architecture*, vol. 12, no. 3, pp. 1–210, 2017.
- [95] J. Balfour and W. J. Dally, “Design tradeoffs for tiled cmp on-chip networks,” in *ACM International conference on supercomputing 25th anniversary volume*, 2006, pp. 390–401.
- [96] C. Walshaw and M. Cross, “Multilevel mesh partitioning for heterogeneous communication networks,” *Future generation computer systems*, vol. 17, no. 5, pp. 601–623, 2001.
- [97] K. Han, S. Hong, J. H. Cheon, and D. Park, “Logistic regression on homomorphic encrypted data at scale,” in *Proceedings of the AAAI conference on artificial intelligence*, vol. 33, no. 01, 2019, pp. 9466–9471.
- [98] E. Lee, J.-W. Lee, J. Lee, Y.-S. Kim, Y. Kim, J.-S. No, and W. Choi, “Low-complexity deep convolutional neural networks on fully homomorphic encryption using multiplexed parallel convolutions,” in *International Conference on Machine Learning*. PMLR, 2022, pp. 12 403–12 422.
- [99] R. M. Fujimoto, “Parallel discrete event simulation,” *Communications of the ACM*, vol. 33, no. 10, pp. 30–53, 1990.

Appendix A

Full Synchronization Blocking Profiles

Figure A.1: The synchronization blocking profile from Go trace tool for ReLU benchmark parallel emulation.



APPENDIX A. FULL SYNCHRONIZATION BLOCKING PROFILES

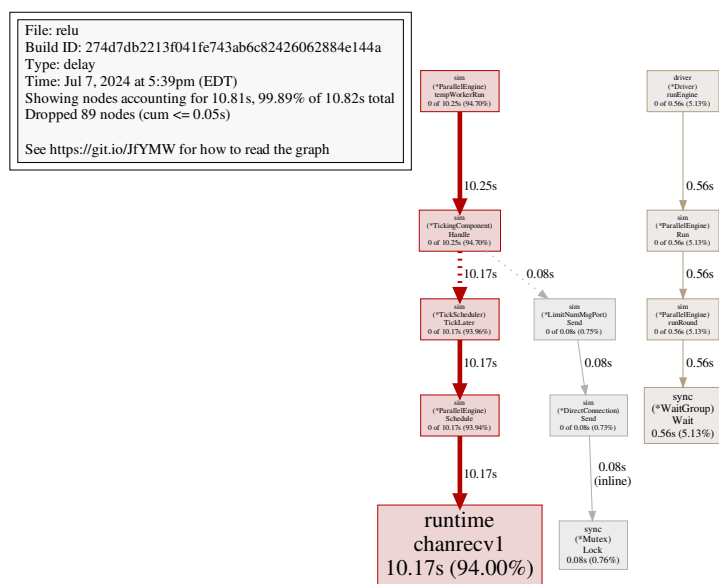


Figure A.2: The synchronization blocking profile from Go trace tool for ReLU benchmark parallel timing simulation.

APPENDIX A. FULL SYNCHRONIZATION BLOCKING PROFILES

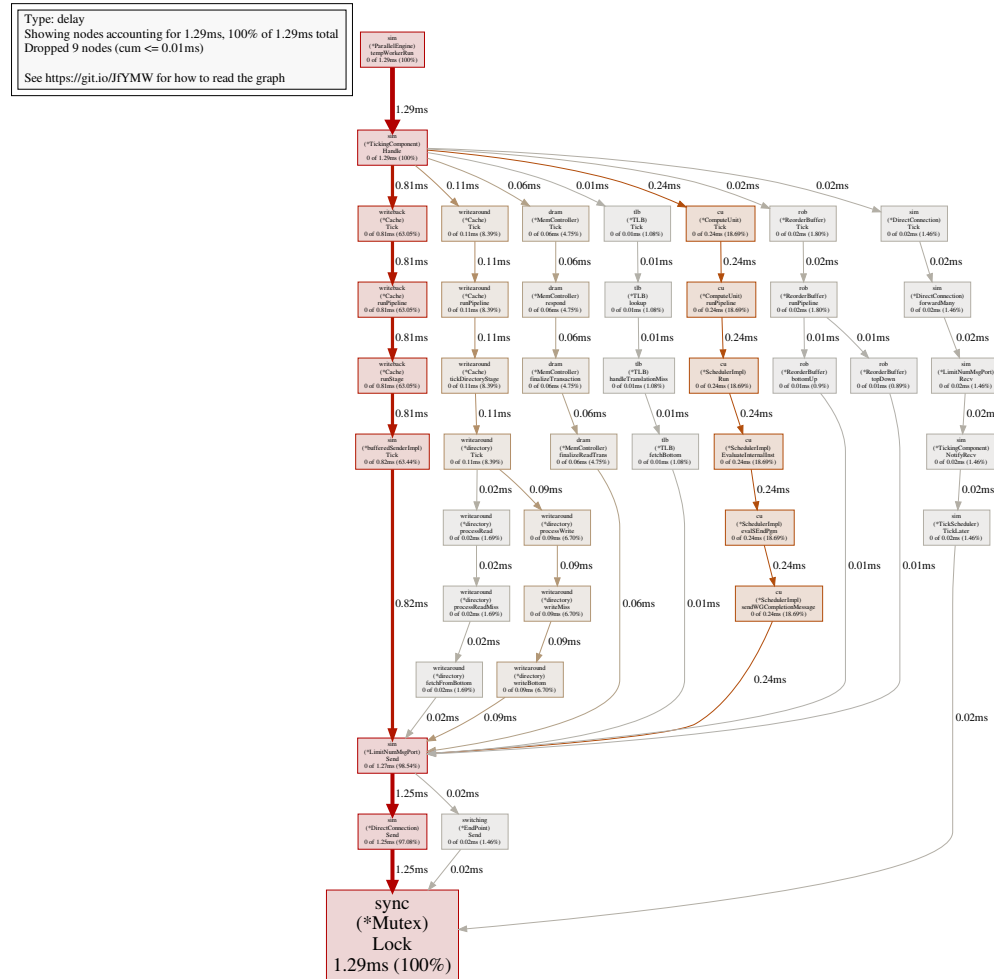


Figure A.3: The synchronization blocking profile after applying the reversing-order strategy.